AI-200 Comprehensive Course Notes

Containers on Azure, Azure Kubernetes Service, Azure Cosmos DB for NoSQL, Azure Database for PostgreSQL, Azure Managed Redis, backend integration, secrets and configuration, and observability for AI solutions

These notes combine the course material for Learning Paths 1 to 9 into a single reference. Each module explains the concepts, lists the key points, gives a practical AI-oriented example, and ends with the main takeaways.

Learning path	Modules covered
LP-1: Implement container application hosting on Azure	Store and manage containers in Azure Container Registry; Deploy containers to Azure App Service
LP-2: Deploy and manage apps on Azure Container Apps	Deploy containers; Manage containers; Scale containers
LP-3: Deploy and monitor applications on Azure Kubernetes Service	Deploy applications; Configure applications; Monitor and troubleshoot applications
LP-4: Develop AI solutions with Azure Cosmos DB for NoSQL	Build queries; Implement vector search; Optimize query performance
LP-5: Develop AI solutions with Azure Database for PostgreSQL	Build and query with Azure Database for PostgreSQL; Implement vector search; Optimize vector search
LP-6: Enhance AI solutions with Azure Managed Redis	Implement data operations; Implement event messaging; Implement vector storage
LP-7: Integrate backend services for AI solutions	Queue and process AI operations with Azure Service Bus; Develop event-driven AI workflows with Azure Event Grid; Build serverless AI backends with Azure Functions
LP-8: Manage application secrets and configuration for AI solutions	Manage application secrets with Azure Key Vault; Manage application settings with Azure App Configuration
LP-9: Observe and troubleshoot apps on Azure	Instrument an app with OpenTelemetry; Analyze app telemetry with logs and metrics

LP-1: Implement container application hosting on Azure 2
Module 1: Store and manage containers in Azure Container Registry 2
Module 2: Deploy containers to Azure App Service 6
LP-2: Deploy and manage apps on Azure Container Apps 9
Module 1: Deploy containers to Azure Container Apps 9
Module 2: Manage containers in Azure Container Apps 13
Module 3: Scale containers in Azure Container Apps 18
LP-3: Deploy and monitor applications on Azure Kubernetes Service 22
Module 1: Deploy applications to Azure Kubernetes Service 22
Module 2: Configure applications on Azure Kubernetes Service 26
Module 3: Monitor and troubleshoot applications on Azure Kubernetes Service 30
LP-4: Develop AI solutions with Azure Cosmos DB for NoSQL 34
Module 1: Build queries for Azure Cosmos DB for NoSQL 34
Module 2: Implement vector search on Azure Cosmos DB for NoSQL 37
Module 3: Optimize query performance for Azure Cosmos DB for NoSQL 42

LP-1: Implement container application hosting on Azure

Module 1: Store and manage containers in Azure Container Registry

1.1 Introduction to Azure Container Registry (ACR)

Azure Container Registry (ACR) is a managed, private container registry service in Azure. It lets you securely store, manage, and distribute the container images used to deploy applications.

- **Image storage:** ACR stores Docker and OCI-compliant images, which contain everything needed to run an application in a container.
- **Azure integration:** Azure Kubernetes Service (AKS), Azure Container Apps, and Azure App Service can pull images directly from ACR.
- **Supported content:** Docker images, Helm charts, and other OCI artifacts used in cloud-native deployments.

- **Service tiers:** Basic suits learning and small workloads, Standard suits higher storage and throughput needs, and Premium suits enterprise requirements.
- **Geo-replication (Premium only):** Replicates registry content across Azure regions so applications can pull images from a location closer to them.

Practical example: You build an AI-powered application and package it as a Docker image. You push the image to ACR, and AKS pulls it to deploy and run the application.

1.2 Registries, repositories, and artifacts

ACR organizes container content in a hierarchy of registries, repositories, and artifacts.

- **Registry:** The top-level resource that stores and manages images (for example, myregistry.azurecr.io).
- **Repository:** A named collection of related images within a registry (for example, inference-api).
- **Artifact:** The content stored in a repository, such as a container image, Helm chart, or other OCI content.
- **Namespaces:** Help organize repositories by application or environment (for example, production/inference-api).

Tags, layers, and manifests

- **Tags:** Mutable, human-readable labels for image versions, such as v1.2.0 or latest (written as repo:tag). Several tags can point to the same artifact, and a tag can be moved to a different image.
- **Layers:** Images are built from reusable layers. ACR is content-addressable and shares common layers between images, which reduces storage duplication.
- **Manifests:** Describe an image's configuration and layers. Each manifest is identified by an immutable SHA-256 digest (sha256:abc...).

How to reference an image

- **By tag:** myregistry.azurecr.io/api:v1.2.0. It is easy to read, but the tag can change.
- **By digest:** myregistry.azurecr.io/api@sha256:abc... It identifies exact content and is preferred when you need reproducible deployments.

Practical example: You deploy an AI inference API. You push its image to the production/inference-api repository and tag it v1.2.0. Your AKS deployment can pull the image by that tag or by its immutable digest.

1.3 Build and run images with ACR Tasks

ACR Tasks is a cloud-based build service. It lets you build, test, and push container images without managing your own build infrastructure.

- **Quick tasks:** Build and push an image with a single Azure CLI command (az acr build).
- **Source context:** The build can use source code from a local directory, a Git repository, or a tar archive.
- **Cloud-based builds:** The image is built in Azure and pushed to the specified registry.
- **Automatic triggers:** Tasks can run on a source code commit, a base image update, or a schedule (cron).
- **Multi-step tasks:** Define build, test, and push workflows in a YAML file.

Example: create a task triggered by source code

```
az acr task create \
  --registry myregistry \
  --name build-inference-api \
  --image inference-api:{{.Run.ID}} \
  --context https://github.com/myorg/inference-api.git#main \
  --file Dockerfile \
  --git-access-token $PAT
```

- **--registry:** The target Azure Container Registry.
- **--name:** The task name.
- **--image:** The image repository and tag. {{.Run.ID}} adds a unique build-run identifier.
- **--context:** The Git repository and branch that contain the source code.
- **--file:** The Dockerfile used to build the image.
- **--git-access-token:** A personal access token (PAT) for a private Git repository, when required.

Practical example: A team develops an AI inference API. Whenever developers push code to the main branch, an ACR Task builds a new image and pushes it to ACR. The team then deploys the image to Azure Container Apps or AKS.

1.4 Tag and version images

A clear versioning strategy lets you identify releases, track builds, and roll back deployments when needed.

Stable tags vs. unique tags

- **Stable tags** (for example, app:v1 or latest) can be reused. Pushing a new image with the same tag moves the tag to the new image, which can make deployments less predictable.

- **Unique tags** (for example, v1.0.0-b123, v1.0.1-b124) identify individual builds. They improve traceability and make it easier to deploy or roll back a specific release.
- **Recommendation:** In CI/CD pipelines, use unique tags such as a build ID, Git commit SHA, timestamp, or a combination, rather than relying only on latest.

Lock production images

```
az acr repository update \
  --name myregistry \
  --image inference-api:v1.2.0 \
  --write-enabled false
```

This makes the tag read-only. A locked image cannot be overwritten or deleted through normal registry operations, which protects production releases from accidental change (and survives retention policies).

Clean up untagged images

```
az acr task create \
  --registry myregistry \
  --name cleanup-untagged \
  --cmd "acr purge --filter '.*:.*' --untagged --ago 7d" \
  --schedule "0 0 * * *" \
  --context /dev/null
```

- ACR can accumulate untagged manifests and image content as new builds are pushed.
- The example uses acr purge to remove untagged images older than seven days and runs daily at midnight.
- **Important:** Test purge filters carefully before automating deletion, because shared layers and manifests may be involved.

Practical example: An AI inference API runs in production as inference-api:v1.2.0. Each new build gets a unique tag, is validated, and is deployed, and the approved production image is locked. If a later release fails, you redeploy the previous known-good version.

Module 1 key takeaways

- **Registry hierarchy:** A registry contains repositories, which hold artifacts such as images and Helm charts. Namespaces help organize content.
- **Tags vs. digests:** Tags are mutable references. SHA-256 digests identify exact content and support reproducible deployments.
- **ACR Tasks:** Build images in Azure, automate builds with triggers, and define multi-step workflows.

- **Tagging strategy:** Use unique tags (build ID or Git commit SHA) for production releases. Stable tags can work for base images if updates are managed carefully.
- **Image locking:** Use --write-enabled false to prevent accidental overwrites or deletions.
- **Lifecycle management:** Schedule cleanup of old untagged images to control storage costs.

Module 2: Deploy containers to Azure App Service

2.1 Introduction to Azure App Service with custom containers

Azure App Service is a fully managed platform for hosting web applications in custom Linux containers. You bring your own Docker image and do not manage servers, the operating system, or infrastructure.

- **Custom containers:** Deploy an image from ACR or any supported Docker V2 registry.
- **Autoscaling:** Scale instances based on workload and configured rules.
- **Deployment slots:** Deploy a new version to a staging slot, validate it, then swap it into production with minimal disruption.
- **TLS termination:** Handled by the platform.
- **Configuration:** App settings are injected into the container as environment variables.
- **Diagnostics:** Log Stream, the Kudu console (where supported), and Azure Monitor.
- **Managed infrastructure:** Azure runs the platform, so you can focus on application code and container configuration.

Practical example: You build an AI chatbot in Python and package it as a Docker image. You push it to ACR and configure App Service to deploy it. You then set environment variables for the AI endpoint, enable autoscaling, and monitor logs without managing virtual machines.

2.2 Deploy containers to Azure App Service

App Service can run images from Azure Container Registry, Docker Hub, GitHub Container Registry (GHCR), or other supported registries.

Image sources

- **Azure Container Registry (ACR):** Recommended for Azure production workloads because it integrates with Azure identity and access management.
- **Docker Hub and GHCR:** Useful when images are published in public or private repositories.

- **Self-hosted registries:** Can be used if the registry is accessible and supported by the deployment configuration.

ACR authentication

- **Managed identity (recommended):** Assign a system-assigned or user-assigned identity to App Service and grant it the AcrPull role. No registry passwords are stored in app settings.
- **Admin credentials:** Acceptable for development and testing, but managed identity is preferable for production.

Deployment methods

- **Azure portal:** Guided configuration of the image and registry.
- **Azure CLI:** Automate container configuration and deployment.
- **VS Code:** Use the Docker and Azure App Service extensions.
- **Continuous deployment:** Use a registry webhook so App Service detects image updates and pulls the new image.

Practical example: You push an AI API image to ACR and configure App Service to pull it using its managed identity with the AcrPull role. Users reach the running application through its web endpoint.

2.3 Configure container runtime behavior

After deployment, you can control how the container starts, which port it listens on, how data is stored, and how its health is monitored.

- **Startup command:** Overrides the image's default CMD while keeping its ENTRYPOINT. Use it for runtime arguments, initialization steps (such as migrations), or custom configuration. Make sure it is compatible with the image.
- **Port configuration:** Set WEBSITES_PORT if the container listens on a port other than 80 or 8080. App Service terminates TLS, so the container normally receives plain HTTP internally.
- **Persistent storage:** Container file systems can be temporary, so files may be lost on restart or replacement. Enable persistent storage when data must survive restarts. Persistent storage can be shared across scaled instances.
- **Always On:** Keeps the app loaded and prevents unloading after about 20 minutes of inactivity. Available on Basic tier and above.
- **Health checks:** App Service pings a configured path (such as /health) about once a minute. Unhealthy instances can be removed from load balancing.

Practical example: An AI inference API listens on port 8000 and exposes /health. You configure the container port, enable Always On for production, and use persistent storage if the app must keep files across restarts.

2.4 Configure application settings

Application settings let you configure a containerized app without changing its source code or rebuilding the image. App Service passes them to the container as environment variables at start-up.

- **App settings:** Store API endpoints, environment names, and feature flags. Read them with standard environment-variable methods (for example, os.environ in Python). They are encrypted at rest, but sensitive values are better kept in Azure Key Vault.
- **Connection strings:** Expose database and service connection details as environment variables using type-specific prefixes (for example, SQLAZURECONNSTR_). For non-.NET apps, ordinary app settings are commonly used instead.
- **Slot settings:** Mark a setting as a slot setting so it stays with its deployment slot during a swap. Use this for environment-specific endpoints, database identifiers, and feature flags.
- **Key Vault references:** Use the syntax @Microsoft.KeyVault(SecretUri=...) so App Service retrieves the secret without storing its value in app settings. The app reads it as a normal environment variable. This requires a managed identity with access to the vault.

Practical example: An AI app needs an Azure AI endpoint, a database connection string, and an API key. Put the endpoint in an app setting, keep the connection details in the appropriate configuration, and store the API key in Key Vault, accessed through a managed identity.

2.5 Observe and troubleshoot containerized apps

After deployment, monitor the app and troubleshoot problems such as start-up failures, incorrect configuration, and application errors.

- **Container logs:** Capture stdout, stderr, and platform lifecycle events. Use them to investigate start-up failures, runtime errors, and crashes.
- **Log Stream:** Shows logs in near real time through the portal or CLI, including from all instances of a scaled-out app. CLI example: az webapp log tail --name <app-name> --resource-group <resource-group>
- **Kudu (SCM site):** Diagnostic tools for supported configurations. You can verify environment variables, browse files (for example, /home/LogFiles/), and download diagnostic dumps.
- **Azure Monitor integration:** Send diagnostic logs to Log Analytics for KQL queries and alerting. Log categories include ConsoleLogs (application output and errors), HTTPLogs (incoming requests), and PlatformLogs (hosting platform events).

Practical example: An AI inference container fails to start after deployment. First use Log Stream to read the start-up errors. Then check environment variables and log files with the diagnostic tools. If the problem continues, query retained logs in Log Analytics to find recurring failures.

Module 2 key takeaways

- **Managed identity for ACR:** Lets App Service pull private images with the AcrPull role, with no stored credentials or credential rotation. It is the recommended production method.
- **WEBSITES_PORT:** Must be set when the container listens on a port other than 80 or 8080. Incorrect ports are a very common cause of deployment failures.
- **Startup command:** Overrides CMD, not ENTRYPOINT. Use it for runtime arguments, migrations, and custom configuration.
- **Slot settings:** Mark environment-specific values so they stay with the correct slot during swaps.
- **Container logging:** Enable diagnostic logging explicitly. stdout and stderr are not persisted until you turn it on.

LP-2: Deploy and manage apps on Azure Container Apps

Module 1: Deploy containers to Azure Container Apps

3.1 Introduction to Azure Container Apps

Azure Container Apps is a serverless container platform for running containerized applications without managing Kubernetes infrastructure. It suits APIs, microservices, background processing, and AI-powered applications.

- **Serverless:** Azure manages the infrastructure so you can focus on development and deployment.
- **No Kubernetes management:** Unlike AKS, you do not manage the Kubernetes control plane or cluster operations.
- **Revision-based deployments:** An update can create a new revision. You can test it and gradually shift traffic from the old revision.
- **Built-in ingress:** External ingress for public access and internal ingress for communication inside the environment.
- **Secrets and environment variables:** Configure settings and manage secrets without hardcoding them into the image.
- **Azure integration:** ACR for images, Azure Monitor for observability, VNets for networking, and managed identities for secure access to Azure resources.

Practical example: An AI document-processing API is packaged as a container, pushed to ACR, and deployed to Container Apps. For each new version, you create a new revision, test it, and gradually direct production traffic to it.

3.2 Azure Container Apps environments

A Container Apps environment is a secure boundary around a group of container apps. Apps in the same environment can share networking, communicate privately, and use common monitoring.

What an environment provides

- **Shared networking:** A common network boundary for related apps.
- **Internal DNS:** Apps call each other by app name instead of hardcoded IP addresses.
- **Centralized monitoring:** Integration with Azure Monitor and Log Analytics for collecting and analyzing logs.

For example, an API container app can receive a request and forward the processing work to a worker container app in the same environment.

Design decisions

- **Separate environments by lifecycle:** Use separate development, test, and production environments for stronger isolation and independent configuration.
- **Plan networking:** Group apps that need private communication, and choose internal or external ingress as appropriate.
- **Plan observability early:** Decide where logs and diagnostics are collected before deploying production workloads.

Practical example: An AI document-processing solution has an API that accepts uploads and a worker that processes them. Both run in one environment, communicate privately, and are monitored together through Azure Monitor.

3.3 Deploy a container app using the Azure CLI and YAML

You can deploy quickly with one command, or define the configuration in YAML for repeatable, version-controlled deployments.

Three deployment approaches

- **az containerapp up:** The quickest option. Depending on the parameters, it can create the required resources and environment.
- **az containerapp create:** Gives explicit control over the environment, image, ingress, and resources.
- **--yaml:** Deploys configuration from a YAML file. Useful for source control, peer review, and consistent deployments across environments.

Revisions

- **New revisions:** Changes to revision-scoped settings, such as the image or configuration, can create a new revision.
- **Validation:** Test the new revision before sending production traffic to it.
- **Traffic management:** In multiple-revision mode, split traffic between revisions for gradual rollouts.
- **Updates:** Use az containerapp update to change an image or supported settings.

Practical example: An AI inference API runs as revision 1. You deploy a new image with an updated model integration, which creates a new revision. After validating it, you gradually route traffic to it. If errors occur, you redirect traffic to the previous healthy revision.

3.4 Configure runtime settings with environment variables and secrets

Environment variables and secrets let you reuse the same container image across development, staging, and production without rebuilding it.

- **Environment variables:** For non-sensitive configuration such as log levels, feature flags, and service endpoint URLs. They can be set at deployment or updated later, and the app reads them at runtime.
- **Secrets:** For sensitive data such as API keys, database credentials, and signing keys. They are managed separately from the image and can be updated or rotated without rebuilding it. Depending on how the app consumes the secret, you may need to restart the app or create a new revision for the change to take effect.
- **Secret references:** Reference a secret by name instead of placing its value in an environment variable definition. The platform resolves the reference and exposes the value to the app as an environment variable. Azure Key Vault can provide centralized secret management and access control.
- **Avoid** putting secret values in YAML files, source control, or shell command history.

Practical example: An AI app needs a log level, an Azure AI endpoint, and an API key. Set the log level and endpoint as environment variables, store the API key as a secret, and reference that secret from the app's environment variable configuration.

3.5 Configure image pull authentication for private registries

To pull private images, Container Apps must authenticate with the registry, using either credentials or a managed identity.

- **Username and password:** Works with private registries such as ACR, Docker Hub, and GHCR. It is simple and suits development, testing, and prototyping.

Credentials must be stored securely and rotated, which adds management effort.

- **Managed identity (recommended for ACR):** The app authenticates without stored registry passwords. Assign a system-assigned or user-assigned identity, grant it the minimum permissions needed to pull images (typically AcrPull), and configure the registry to use it. For example, the Azure CLI supports --identity system for a system-assigned identity.

Troubleshooting tip: Registry authentication is configured separately from environment variables and secrets. If a container fails to start because its image cannot be downloaded, check the registry configuration, identity assignment, and permissions before looking at the application code.

Practical example: An AI inference service pulls its image from a private ACR repository. You configure a managed identity for the container app and grant it pull permissions, so Azure retrieves the image without registry credentials in the app.

3.6 Verify deployments with logs and status

After deploying, confirm that the app started successfully, the correct revision is active, and the expected number of replicas is running.

1. Check logs to identify errors

- Console logs show application output and errors, such as missing environment variables, failed connections, or start-up exceptions.
- System logs show platform-level problems, such as image-pull failures and scaling issues.
- Real-time log streaming helps when debugging an active deployment.

2. Check revisions to verify the version

- Confirm that the expected revision was created after the update, and that it is active and healthy.
- Inspect inactive revisions when investigating failed deployments or earlier versions.

3. Check replicas to verify scaling

- Replicas are the running instances of a revision. Check that the expected number is running under the current load.
- Investigate repeated restarts, scale-to-zero behavior, and cold starts when requests are delayed.

Practical example: You deploy a new version of an AI inference API and requests fail. Inspect the console and system logs for start-up or image-pull errors, verify that the new revision exists and is healthy, then inspect its replicas for crashes or scaling issues.

Module 1 key takeaways

- **Environments:** A shared boundary for networking, logging, and communication between related apps.
- **Deployment paths:** Use az containerapp up for quick deployments and YAML for repeatable, source-controlled production deployments.
- **Revisions:** Versioned snapshots of changes that enable controlled rollouts and traffic management.
- **Secrets management:** Store sensitive values separately and reference them through environment variables instead of hardcoding credentials.
- **Registry authentication:** Prefer managed identity with ACR pull permissions for secure production deployments.
- **Deployment verification:** Check logs first for errors, then revisions to confirm the deployed version, then replicas to validate runtime and scaling.

Practical scenario: An AI document-processing solution is deployed with YAML, pulls its image from ACR using a managed identity, keeps API credentials as secrets, and releases updates through revisions. If a deployment fails, you inspect logs, revision status, and replicas to find the problem.

Module 2: Manage containers in Azure Container Apps

4.1 Introduction to managing containers (day-two operations)

Deploying an app is only the first step. You must also manage it after deployment by monitoring performance, handling failures, controlling resource use, and keeping the app available. These activities are called day-two operations.

- **Revision control:** Updates create revisions. Validate a new revision and manage traffic between versions.
- **Lifecycle actions:** Start, stop, restart, or deactivate apps and revisions for maintenance and incident response, depending on the operation supported.
- **Log diagnostics:** Use Azure Monitor and Log Analytics to investigate errors, start-up failures, and runtime issues.
- **Health probes:** Liveness probes identify containers that need restarting. Readiness probes determine whether a replica is ready to receive traffic.
- **Resource sizing and autoscaling:** Configure CPU and memory per container and scaling rules to handle changing workloads while controlling cost.

Practical example: An AI inference app becomes slow at peak usage. You inspect logs, check replica health, and review CPU and memory use. You then adjust resource allocations or scaling rules, and deploy an updated revision if needed.

4.2 Update images and manage revisions safely

When you update an image, deploy the new version carefully, verify it works, and only then send production traffic to it. This lowers the risk of downtime.

Image identity

- **Tags:** Human-readable references such as v1.2.0. They are convenient but can be moved to another image.
- **Digests:** SHA-256 identifiers of the exact image content. Using digests in production improves traceability and consistency.
- **Build traceability:** Link each deployed image to its build artifact or Git commit so issues can be traced to a specific release.

Revision workflow

1. **Deploy a new image** by tag or digest. This normally creates a new revision.
2. **Verify health** by checking logs, revision status, and health probes before shifting traffic.
3. **Shift traffic** gradually in multiple-revision mode, or switch it after validation.
4. **Handle failures** by routing traffic back to a healthy revision. Deactivate the problem revision when appropriate, while keeping the diagnostic evidence.

Revision retention: Keep useful inactive revisions for troubleshooting and rollback, but set a retention policy to avoid accumulating unnecessary revisions.

Practical example: An AI inference API runs revision 1 (image digest A). You deploy revision 2 (digest B), verify its health, and gradually direct traffic to it. If errors increase, you route traffic back to revision 1 while you investigate.

4.3 Manage the container app lifecycle

Lifecycle operations help you recover from failures, handle faulty releases, and keep the app available. Choose the action that matches the scope and nature of the problem.

Choose the right action

- **Stop the application:** Halts all processing during an incident or maintenance. This is a broad action and can interrupt all traffic.
- **Restart:** Use when a running container is stuck or has a transient problem. It clears temporary runtime state but does not fix persistent configuration or code problems.
- **Deactivate a revision:** Use when a specific release is faulty. It isolates the problem revision while keeping other revisions for investigation and rollback.

Common failure patterns

- **Image-pull failures:** Verify registry credentials, managed identity configuration, and image permissions.
- **Port mismatches:** Confirm that the app listens on the expected port and that ingress targets that port.
- **Missing environment variables or secrets:** Check configuration names, secret references, and permissions.
- **Probe failures:** Verify health-check paths, response codes, and timeout settings.
- **Resource pressure:** Look for CPU throttling, memory limits, and out-of-memory (OOM) restarts. Increase resources or adjust scaling when justified.

Practical example: A new AI API revision repeatedly fails its health checks. Inspect the logs and probe configuration. If only the new revision is affected, deactivate it or route traffic back to the last healthy revision rather than stopping the whole app.

4.4 Monitor logs and troubleshoot issues

When a container app fails or behaves unexpectedly, follow a systematic process instead of making random configuration changes. Logs show the cause, and revision comparisons show what changed.

1. **Identify the failing revision.** Check its status and whether the problem started after a recent deployment.
2. **Stream logs.** Console logs show application output, start-up messages, exceptions, and runtime errors. System logs show platform events such as image-pull failures, scaling activity, and health-probe problems. Stream logs while reproducing the problem.
3. **Compare revisions.** Compare the failing revision with a working one: container image, environment variables, secrets, ingress, health probes, and resource allocation.
4. **Fix and verify.** Apply a targeted fix, deploy or activate the corrected revision, and verify its health before sending production traffic to it.

Structured logging best practices

- Include a request ID to trace individual requests across services.
- Record the revision ID and AI model version to identify which deployment handled a request.
- Log latency for dependencies such as databases and AI endpoints.
- Never log sensitive information such as API keys, authentication tokens, private prompts, or confidential document contents.

Practical example: An AI app returns errors after a deployment. You identify the failing revision, inspect its console and system logs, and compare its configuration with the

previous healthy revision. The new revision references an incorrect AI endpoint. You correct it and verify the next revision before shifting traffic.

4.5 Configure health probes and troubleshoot failures

Health probes tell Container Apps whether a container is ready to serve requests and whether it is still working. They matter especially for AI apps, because loading models and initializing dependencies can take longer than normal start-up.

Readiness vs. liveness

- **Readiness probe:** Checks whether the app is ready to receive traffic. For example, an inference API should not get requests until its model has loaded.
- **Liveness probe:** Checks whether the app is still functioning. If it repeatedly fails, the platform can restart the container.
- **Key difference:** Readiness controls whether an instance receives traffic. Liveness detects containers that need restarting.

Tuning probes for AI services

- **initialDelaySeconds:** Delay before probing begins, where supported.
- **failureThreshold:** How many consecutive failures count as a failed probe.
- **Startup probes:** Allow slow initialization without triggering premature liveness failures.
- **Independent health endpoints:** Make probe endpoints respond quickly and avoid depending on external services such as databases or AI APIs.

If probes fail right after deployment

- Check that the configured port matches the port the container listens on.
- Check that the HTTP path exists and returns an acceptable status code.
- Make sure the timeout gives the endpoint enough time to respond.
- Review console and system logs for start-up errors or resource constraints.

Practical example: An AI inference container takes 45 seconds to load its model. If the liveness probe starts too early, Azure may keep restarting the container before it finishes initializing. Configure suitable startup-probe settings and use readiness checks so traffic does not reach the app prematurely.

4.6 Optimize container resources and scaling

You configure CPU and memory for each replica and define scaling rules for changing workloads. The aim is to balance performance, throughput, and cost.

Per-replica resource sizing

- **CPU throttling:** If the app is often constrained by its CPU limit, consider giving each replica more CPU.

- **Out-of-memory (OOM) restarts:** A container that exceeds its memory limit may be terminated. Investigate memory use and increase the allocation if needed.
- **Cost:** Total consumption depends on resources per replica, the number of replicas, and how long they run.

Choose the right scaling strategy

- **Synchronous APIs:** They respond directly to users. Keep a suitable minimum number of replicas when low latency matters, because scaling from zero can cause cold-start delays.
- **Background workers:** They process queued jobs or events. Event-driven scaling and scale-to-zero can cut costs when there is no work.

Optimize performance

- **Measure first:** Use logs and metrics to find CPU pressure, memory use, latency, and request volume.
- **Change one variable at a time:** Adjust CPU, memory, minimum replicas, or thresholds individually so you can measure each effect.
- **Reassess after model updates:** A new AI model version may need more memory or processing capacity.

Practical example: An AI inference API has high latency at peak traffic. You check CPU use and throttling, increase CPU per replica if needed, and configure scaling rules for demand. A background document-processing worker can instead use queue-based scaling and scale to zero when the queue is empty.

Module 2 key takeaways

- **Image digests:** Use immutable SHA-256 digests to identify the exact image in production. Tags are convenient but can change.
- **Revision workflow:** Deploy an updated image, verify its health, then shift traffic. Keep useful inactive revisions for troubleshooting and rollback.
- **Lifecycle management:** Deactivating a revision isolates a faulty release. Stopping or restarting the app has a broader impact.
- **Logs first:** Check console logs for application errors and system logs for platform issues such as image-pull failures and scaling problems.
- **Health probes:** Readiness probes control whether an instance receives traffic. Liveness probes detect failures that need a restart.
- **Resource optimization:** Fix CPU or memory bottlenecks before increasing replica counts. Change one setting at a time and measure the results.

Module 3: Scale containers in Azure Container Apps

5.1 Introduction to scaling containers

Container Apps supports automatic horizontal scaling: it increases or decreases the number of running replicas based on demand. This keeps performance steady in busy periods and reduces cost when the app is idle.

- **Automatic horizontal scaling:** Adds or removes replicas according to scaling rules.
- **KEDA:** Container Apps uses Kubernetes Event-Driven Autoscaling (KEDA) for event-based scaling, without you managing Kubernetes.
- **Scale to zero:** For supported workloads and configurations, the app can drop to zero replicas when there is no demand.
- **Built-in triggers:** HTTP requests, TCP connections, CPU usage, or memory usage, depending on the rule.
- **Event-driven scaling:** Background workers can scale on queue length, stream activity, or other supported event sources.
- **Traffic splitting:** Distribute requests across revisions to test new versions gradually before a full rollout.

Practical example: An AI chatbot scales out to more replicas during business hours when requests increase, and scales down when it is quiet. A background document-processing worker scales based on the number of messages waiting in a queue.

5.2 Configure scale rules

Scale rules determine when replicas are added or removed. You set minimum and maximum replica counts, define triggers, and control how quickly scaling decisions take effect.

Components of a scale definition

- **Minimum and maximum replicas:** The lower and upper limits. The minimum can be zero if the rules support scale-to-zero.
- **Scaling rules:** The conditions that trigger scaling, such as HTTP requests, TCP connections, CPU, memory, or events.
- **Scaling behavior:** Polling intervals, cooldown periods, and stabilization that avoid unnecessary rapid changes.

Built-in rule types

- **HTTP:** Scales on concurrent HTTP requests. Useful for APIs and web apps.
- **TCP:** Scales on concurrent TCP connections. Useful for workloads with persistent connections.
- **CPU:** Scales on CPU utilization. It cannot scale to zero on its own.

- **Memory:** Scales on memory utilization. It also cannot scale to zero on its own.

How scaling decisions work

1. The platform evaluates the configured triggers.
2. It compares observed values with the configured thresholds.
3. It decides whether to scale out or in, within the replica limits.
4. It adjusts the replica count according to the scaling behavior.

Important: When several scaling rules are configured, the platform uses the rule that requires the highest replica count.

Practical example: An AI inference API has a minimum of 1 and a maximum of 10 replicas. At peak, HTTP concurrency exceeds the threshold and the app scales out. When demand falls, it scales back in while respecting the minimum.

5.3 Implement event-driven scaling with KEDA

KEDA (Kubernetes Event-Driven Autoscaling) lets Container Apps scale replicas based on events and metrics from external services. It is especially useful for background workers that process messages, files, or events.

Azure-native event scalers

- **Azure Service Bus:** Scales workers on the number of messages waiting in a queue or subscription.
- **Azure Event Hubs:** Scales on consumer lag, which shows how far processing is behind incoming events.
- **Azure Storage Queue:** Scales on the approximate number of messages waiting to be processed.
- **Azure Blob Storage:** Scales on blob-related metrics, such as blob count, using a supported scaler or event integration.

How it works

1. KEDA monitors the event source and retrieves its metrics.
2. It compares the observed metric with a configured threshold.
3. Container Apps adjusts the replica count according to the rule.
4. When no work is pending, supported configurations can scale to zero.

For example, a queue with many pending messages starts more worker replicas to process them in parallel. When the queue empties, workers scale down and cost falls.

Authentication

- **Managed identity:** Preferred for production because no connection credentials are stored in app configuration. Grant it the necessary permissions on the event source.

- **Secrets-based connection strings:** An alternative when managed identity is not suitable. Store them as secrets, never hardcoded.

Practical example: In an AI document-processing app, uploaded documents create messages in Azure Service Bus. KEDA watches the queue and scales worker replicas to match the backlog. Each worker processes documents using an AI service, and the worker count falls when the queue is empty.

5.4 Apply KEDA scalers for custom workloads

KEDA-based scaling also covers workloads beyond HTTP and Azure-native services. Different scalers monitor external systems and adjust replicas to match demand.

Scalers for custom workloads

- **Kafka:** Scales on consumer group lag, which shows how far consumers are behind incoming messages.
- **Redis:** Scales on list length or pending stream entries, depending on the scaler configuration.
- **Cron:** Schedules replicas in advance for predictable peaks, such as daily batch processing.
- **Prometheus:** Scales on custom metrics queried through PromQL, such as pending jobs or other application-specific indicators.

Combining multiple scalers

- **Cron scaler:** Sets baseline capacity before a predictable busy period.
- **HTTP or event scaler:** Responds to real-time demand.
- **Highest replica count wins:** When rules recommend different counts, the platform scales to satisfy the one requiring the most replicas, within the configured limits.

Example: An AI document-processing service receives a large batch of jobs every morning. A Cron rule prepares capacity before the batch starts, and a queue-based scaler adds replicas if the backlog grows unexpectedly.

Migrating from Kubernetes KEDA

- Map Kubernetes ScaledObject configurations to the matching Container Apps scale rules.
- Review scaler metadata, thresholds, and authentication requirements.
- Replace Kubernetes TriggerAuthentication configurations with supported Container Apps methods, such as managed identity or secrets.
- Not every Kubernetes KEDA configuration transfers directly, so confirm that the scaler and authentication approach are supported.

5.5 Select compute resources for performance and cost

You allocate CPU and memory to each replica. The right choice affects performance, reliability, and cost, especially for AI workloads that need significant processing power and memory.

Resource allocation

- **CPU:** Allocated in cores or fractions of a core.
- **Memory:** Allocated in GiB. Available CPU and memory combinations depend on the workload profile and configuration.
- **Memory limit exceeded:** The container can be terminated and restarted.
- **CPU limit exceeded:** The container is throttled (its processing is restricted) rather than restarted.

Consumption vs. Dedicated plans

- **Consumption:** Shared infrastructure with usage-based billing. Supported apps can scale to zero, which reduces compute cost when idle.
- **Dedicated:** Reserved compute capacity for more predictable performance, larger resource allocations, and supported GPU workloads.
- Choose based on workload requirements, expected traffic, latency needs, and budget.

Optimize cost and performance

- **Scale to zero:** Useful for intermittent workloads that do not need replicas running all the time.
- **Right-size resources:** Start with a sensible configuration and adjust it based on measured CPU, memory, and latency.
- **Calculate total capacity:** Multiply resources per replica by the maximum replica count to estimate potential compute needs.
- **Balance replica size:** Smaller replicas give finer scaling steps, while larger replicas may suit memory-intensive AI inference.

Practical example: An AI inference container often restarts because of out-of-memory errors, so you increase its memory allocation. If it instead suffers high CPU throttling, increase CPU per replica or scale out to improve throughput.

5.6 Choose and apply revision modes

Container Apps has two revision modes. The mode you choose decides how versions are activated and how traffic is routed during deployments.

Single revision mode

- Only one revision serves traffic at a time.

- On update, the platform waits for the new revision to become ready before completing the transition, which simplifies management and helps minimize downtime.
- Traffic splitting between revisions is not supported.
- **Best for:** Straightforward deployments without manual traffic management.

Multiple revision mode

- Several revisions can be active at once, and you assign traffic percentages to each.
- **Canary deployment:** Send a small share of traffic to the new version before increasing it.
- **Blue-green deployment:** Keep the current version serving production while you validate the new one, then switch traffic.
- Old revisions may need to be deactivated when no longer required.
- **Example:** Route 80% of requests to revision v1 and 20% to v2. If v2 performs well, gradually increase its share. If errors increase, route traffic back to v1.

Revision labels

- Labels give named endpoints for specific revisions. For example, a "blue" label can identify the production revision and a "green" label the staging revision.
- They let you test a particular revision without sending normal production traffic to it.
- Moving a label to another revision supports controlled blue-green switching.

Practical example: An AI inference API uses single revision mode for simple updates. For a high-risk model upgrade, it uses multiple revision mode to test the new model with a small percentage of traffic before a full release.

LP-3: Deploy and monitor applications on Azure Kubernetes Service

Module 1: Deploy applications to Azure Kubernetes Service

6.1 Introduction to deploying applications to AKS

Azure Kubernetes Service (AKS) is a managed Kubernetes platform for deploying, scaling, and managing containerized applications. Azure manages the Kubernetes control plane. You manage the application deployments and configure the worker node infrastructure to your requirements.

- **Managed Kubernetes:** Azure runs the control plane, which reduces operational overhead.

- **Containerized workloads:** AKS runs APIs, background workers, microservices, and AI inference services at scale.
- **Deployments:** Maintain the desired number of Pod replicas and replace failed Pods.
- **Services:** Give stable network access to Pods, even when Pods are replaced or their IP addresses change.
- **Load balancers:** Azure Load Balancer can expose applications through public endpoints when needed.
- **kubectl:** The primary command-line tool to deploy, inspect, view logs, and troubleshoot workloads.

Practical example: You package an AI inference API as a Docker image, deploy it to AKS with a Kubernetes Deployment, expose it through a Service, and configure scaling to handle higher traffic.

6.2 Create Kubernetes Deployment manifests

A Deployment manifest is a YAML file describing the desired state of an application. Kubernetes continuously works to maintain that state, including the replica count, container image, and Pod configuration.

Define the desired state in YAML

- **Image and replicas:** Specify the container image and the number of Pod replicas.
- **Labels and selectors:** The labels in spec.template.metadata.labels must match the Deployment's spec.selector.matchLabels. Kubernetes uses the selector to identify the Pods the Deployment manages.
- **Source of truth:** Keep manifests in source control so changes can be reviewed and deployed consistently.

Configure runtime resources

- **Requests:** The CPU and memory a container needs. Kubernetes uses them to decide where to schedule Pods.
- **Limits:** Maximum CPU and memory. CPU is throttled at its limit, while exceeding the memory limit can cause the container to be terminated. Limits also prevent runaway usage and noisy neighbors.
- **Replica count:** More replicas improve availability and throughput but consume more resources.

Manage configuration securely

- **Environment variables:** For non-sensitive values such as log levels and endpoints.

- **Kubernetes Secrets:** Reference Secrets for API keys, passwords, and other credentials.
- **Security:** Do not embed sensitive values in manifests or commit them to source control. Apply access controls to Secrets.

Example manifest

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-api  # must match
  template:
    metadata:
      labels:
        app: ai-api  # must match
    spec:
      containers:
      - name: ai-api
        image: myacr.azurecr.io/ai-api:v1
        # ports, resources, env, secrets
```

- **apiVersion: apps/v1** identifies the Kubernetes API version, and **kind: Deployment** the resource type.
- **replicas: 3** tells Kubernetes to maintain three Pod replicas.
- **selector.matchLabels** identifies the Pods the Deployment manages.
- **template** defines the Pod configuration, including its labels and container image.

Practical example: For an AI inference API you deploy three replicas so requests are spread across several Pods. If a Pod fails, the Deployment controller works to restore the desired replica count.

6.3 Expose applications in AKS

Pods need a way to talk to other applications and receive requests from users. A Kubernetes Service provides a stable network endpoint, even when Pods are replaced or their IP addresses change.

- **ClusterIP:** The default type. Provides an internal-only endpoint inside the cluster. Best for communication between backend services. For example, a

frontend calls an inference API through a ClusterIP Service without exposing it to the internet.

- **NodePort:** Opens a port on every node so external clients can reach the app through a node IP and that port. Useful for development, testing, and simple validation, but usually not the preferred production entry point.
- **LoadBalancer:** Requests an external load balancer (typically an Azure Load Balancer in AKS) and provisions an Azure-managed public entry point. Best for production APIs serving external clients.

Important Service settings

- **selector:** Must match the labels on the Pods that should receive traffic.
- **port:** The client-facing port exposed by the Service.
- **targetPort:** The container-facing port where the application receives traffic.
- For example, a Service can expose port 80 and forward requests to port 8080 in the container.

Practical example: An AI inference API runs on three Pods, each listening on port 8080. You create a LoadBalancer Service exposing port 80 with targetPort 8080. The Azure-managed load balancer provides an external endpoint and distributes traffic to the eligible Pods.

6.4 Deploy applications to AKS

Deploying to AKS means applying Kubernetes YAML manifests, verifying that Pods run correctly, and confirming the Service can route traffic to them. Kubernetes keeps working to maintain the desired state in the manifests.

Deployment flow

1. **Apply manifests:** Use kubectl apply -f to create or update resources, then let the cluster reconcile state.
2. **Verify Pods:** Wait for Pods to reach the Running state, with readiness checks passing and replicas Ready.
3. **Verify the Service:** Confirm it has healthy endpoints and, for a LoadBalancer Service, an external address.

Common failure signals

- **ImagePullBackOff:** Kubernetes cannot pull the image. Verify the image name, tag, registry path, access, and authentication.
- **CrashLoopBackOff:** The container starts, then exits or fails health checks, repeatedly. Check application logs, startup commands, environment variables, and probes.

- **Pending:** The Pod cannot be scheduled. Resource requests may exceed available node capacity. Check node capacity, CPU and memory requests, and scheduling constraints.
- **No Service endpoints:** The Service selector may not match the Pod labels, or the Pods may not be ready.

Verification commands

```
kubectl get deployments,pods,services
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get endpoints <service-name>
```

Use describe to investigate scheduling and image-pull events, and logs to find application-level errors. A good order is Pod and Deployment status first, then Service exposure and endpoints, then logs, before changing any configuration.

Practical example: An AI inference API is deployed to AKS but is not reachable. Check the Pod status first. If it shows ImagePullBackOff, verify ACR access. If the Pods are healthy but the Service has no endpoints, check the Service selector against the Pod labels.

Module 1 key takeaways

- **Deployment manifest:** A YAML file defining the desired state: images, replica count, resources, and configuration.
- **Requests and limits:** Requests drive scheduling. Limits control resource consumption and stability. Together they affect total cluster cost.
- **Service types:** Choose the exposure scope first (ClusterIP for internal, NodePort for node-port access, LoadBalancer for external), then map ports correctly.
- **Label matching:** The most important link between a Service and the Pods it should reach.
- **Verification loop:** Check Deployment and Pod status, then Service exposure and endpoints, then logs and events, before editing manifests.
- **Troubleshooting signals:** Pod state names are clues. ImagePullBackOff points to registry or image problems, CrashLoopBackOff to repeated container failures, and Pending to scheduling or resource constraints.

Module 2: Configure applications on Azure Kubernetes Service

7.1 Introduction to configuring applications on AKS

Keep configuration separate from application code and container images. This lets you use the same image in development, testing, and production while changing settings for each environment.

- **ConfigMaps:** Non-sensitive configuration such as API endpoints, feature flags, and log levels.
- **Kubernetes Secrets:** Sensitive data such as API keys, passwords, and connection strings. Apply access controls and encryption protections.
- **PersistentVolumeClaims (PVCs):** Request durable storage that can survive Pod restarts and replacement, depending on the storage configuration.
- **Azure integrations:** Azure Key Vault manages secrets, Azure App Configuration centralizes settings and feature flags, and Azure managed disks provide persistent block storage.
- **kubectl verification:** Confirm that configuration and storage resources are defined correctly and available to Pods.

Practical example: An AI inference API needs an AI endpoint, an API key, and persistent storage. Store the endpoint in a ConfigMap, protect the API key in a Secret or Azure Key Vault, and use a PVC when data must outlive the Pod.

7.2 Define ConfigMaps for application settings

A ConfigMap stores non-sensitive configuration as key-value pairs (up to 1 MiB). You can change settings without rebuilding the container image, so one image works across environments. ConfigMaps are for non-sensitive data only, never passwords or API keys.

How applications consume ConfigMaps

- **Environment variables:** Use configMapKeyRef to inject individual values into a container.
- **Volume mounts:** Mount a ConfigMap as files when the app expects file-based configuration. Mounted values update eventually after a change, but the app may need to reload the files.
- **Immutable ConfigMaps:** Set immutable: true to prevent changes and reduce API server load. To change the configuration, create a replacement ConfigMap and update the workload reference (delete and recreate).

Example ConfigMap

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-settings
data:
  FEATURE_X_ENABLED: "true"
  SERVICE_ENDPOINT: "https://api.example.com"
  app.config: |
    log_level=info
    timeout_seconds=30
```

- **FEATURE_X_ENABLED:** Turns a feature on or off.
- **SERVICE_ENDPOINT:** The service URL.
- **app.config:** File-style configuration containing a log level and a timeout.

A Pod can use these values as environment variables or mount them as configuration files.

Practical example: An AI inference API uses the same image in development and production. You supply different service endpoints and log levels through separate ConfigMaps, with no image rebuild.

7.3 Implement Secrets for sensitive data

Kubernetes Secrets store API keys, passwords, database connection strings, and other credentials, so applications can use sensitive values without hardcoding them in images or code.

Kubernetes Secrets

- **Environment variables:** Use secretKeyRef to inject a specific Secret value.
- **Volume mounts:** Mount Secrets as files when the app expects file-based credentials.
- **Access control:** Use Kubernetes RBAC to restrict who and what can read Secrets. Secret values are base64-encoded, not automatically encrypted just because they are Secrets.

Azure Key Vault integration

- **Secrets Store CSI Driver:** Mounts secrets from Azure Key Vault directly into Pods as files.
- **Automatic rotation:** When configured and supported, mounted values can be refreshed periodically.
- **Audit logging:** Key Vault logging tracks secret access and related operations.
- **Managed identity:** Lets workloads authenticate to Key Vault without separate stored credentials.

Security best practices

- Never commit passwords, API keys, or literal secret values to source control.
- Apply least-privilege access through RBAC and Azure permissions.
- Enable appropriate encryption and audit controls.
- Rotate credentials regularly and make sure apps pick up updated values. Environment variables generally need a container restart or rollout to receive changed Secret values.

Practical example: An AI inference app needs an API key for an AI endpoint. Store the key in Azure Key Vault and mount it into the Pod with the Secrets Store CSI Driver using a

suitable workload identity. The key is never embedded in the Docker image or Git repository.

7.4 Attach persistent storage to an app

Containers are generally temporary, so data stored inside a container can be lost when the Pod is replaced. Persistent storage lets an application keep data independently of a Pod's lifecycle.

Storage model

- **PersistentVolumeClaim (PVC):** A request for storage that specifies the capacity and access mode.
- **StorageClass:** Defines how storage is provisioned. It can dynamically create Azure Disk or Azure Files storage, depending on the driver and class.
- **Azure Disk:** Block storage, commonly used when a workload needs a disk attached to a node.
- **Azure Files:** Shared file storage that multiple Pods can access when the access mode and protocol support it.

Access modes

- **ReadWriteOnce (RWO):** Mounted read-write by a single node at a time (for example, Azure Disk). Several Pods on that node may be able to use it, depending on the setup.
- **ReadWriteMany (RWX):** Mounted read-write by multiple nodes (for example, Azure Files). Used when Pods need shared access.
- **Important:** RWO does not necessarily mean "one Pod only". It describes node-level attachment.

Verify persistent storage

1. Check that the PVC reaches the Bound state after applying it (kubectl get pvc).
2. Write a test file to the mounted path.
3. Delete or replace the Pod.
4. Start a new Pod using the same PVC and verify the file is still there.

Practical example: An AI document-processing app must keep generated files when a Pod restarts. You mount an Azure Files volume through a PVC so replacement Pods can access the same data.

Module 2 key takeaways

- **ConfigMaps:** Externalize non-sensitive settings and update them without rebuilding images.
- **Consumption patterns:** Use environment variables for process-level settings and volume mounts for file-based configuration.

- **Secrets:** Inject values with secretKeyRef and restrict access with RBAC.
- **Key Vault integration:** Use the Secrets Store CSI Driver for rotation, audit trails, and production-grade secret management.
- **Persistent storage:** PVCs with StorageClasses provision Azure Disk or Azure Files automatically.
- **Access modes:** ReadWriteOnce allows read-write mounting by one node at a time (for example, single-Pod databases). ReadWriteMany supports multiple nodes when the storage provider supports it (shared file access).

Practical scenario: An AI inference app needs an endpoint, an API key, and storage for generated files. Use a ConfigMap for the endpoint, Azure Key Vault for the API key, and a PVC backed by Azure Files when files must be shared across Pods.

Module 3: Monitor and troubleshoot applications on Azure Kubernetes Service

8.1 Introduction to monitoring and troubleshooting on AKS

Monitoring and troubleshooting keep applications reliable. You use the Azure portal and kubectl to find problems, understand root causes, and apply fixes.

Observability on AKS

- **Azure portal:** Dashboards, Live Logs, metrics, and guided diagnostics.
- **kubectl:** Direct access to Pods, logs, metrics, resource status, and Kubernetes events.
- **Key indicators:** Watch latency spikes, error rates, CPU usage, and memory use to catch issues before users are affected.

Structured troubleshooting

1. **Inspect Pod status and events** to spot CrashLoopBackOff, ImagePullBackOff, or Pending.
2. **Check logs** for application errors and container start-up messages.
3. **Verify networking:** Service selectors match Pod labels, ports are correct, and endpoints are available.
4. **Fix and redeploy:** Update the manifest, apply it, and confirm the app is healthy.

Useful commands

```
kubectl get pods
kubectl describe pod <pod-name>
kubectl logs <pod-name>
kubectl get services
kubectl get events --sort-by=.metadata.creationTimestamp
```

Practical example: An AI inference API on AKS returns errors. Check the Pod status, inspect its logs for application failures, and verify that the Service routes traffic to the correct Pods. After fixing the issue, redeploy and confirm the app responds successfully.

8.2 Monitor application logs and metrics

Monitoring shows whether an application is healthy, responsive, and using resources efficiently. In AKS you use Azure Monitor, Container Insights, and kubectl to find performance problems and investigate failures.

Key monitoring signals

- **Response latency and error rates:** Reveal slow requests and application failures.
- **Pod restart count:** Frequent restarts can point to crashes, failed health probes, or memory issues.
- **CPU and memory:** Compare actual use with configured requests and limits to find bottlenecks.
- **Exit codes and health probe results:** Explain why containers terminate or fail readiness and liveness checks.

Monitoring tools

- **Azure Monitor and Container Insights:** Cluster-wide views of resource use, performance, and container logs (the portal's Monitoring tab).
- **Live Logs:** Container output in near real time.
- **kubectl logs:** Application logs for a specific Pod.
- **kubectl top pods:** Current CPU and memory use, provided the cluster's Metrics Server is available.

Command examples

```
kubectl get pods -n ai-workloads
kubectl logs <pod-name> -n ai-workloads
kubectl logs -f <pod-name> -n ai-workloads
kubectl top pods -n ai-workloads
```

- **get pods:** Lists Pods in the ai-workloads namespace.
- **logs:** Shows the container's application logs.
- **logs -f:** Streams logs continuously while troubleshooting.
- **top pods:** Reports current CPU and memory consumption.

Practical example: An AI inference API becomes slow. Check Pod restarts, stream its logs, and run kubectl top pods to see whether CPU or memory use is unusually high. Use Azure Monitor for a broader view of performance over time.

8.3 Troubleshoot Pods and Services

When an AKS application fails, investigate Pod status, review Kubernetes events and logs, and verify network connectivity. A structured process helps find the root cause instead of changing configuration at random.

Recognize common failure states

- **ImagePullBackOff:** The image cannot be downloaded. Check the image name, tag, registry path, permissions, and authentication.
- **CrashLoopBackOff:** The container repeatedly starts and exits or fails health checks. Check application errors, start-up commands, missing environment variables, and health probes.
- **Pending:** The Pod cannot be scheduled. Resource requests may exceed available node capacity. Check node capacity, requests, and scheduling constraints.

Five investigation steps

1. **Check Pod status:** kubectl get pods -n ai-workloads. Identify Pods that are not running or ready.
2. **Describe the Pod:** kubectl describe pod <pod-name> -n ai-workloads. Review the Events section for failed image pulls, scheduling problems, or probe failures.
3. **Review logs:** kubectl logs <pod-name> -n ai-workloads. For a container that has restarted, add --previous to see the earlier instance's logs.
4. **Debug the container:** Verify environment variables, mounted Secrets, configuration files, start-up behavior, and health probes. Use the portal's Diagnose and solve problems for guided fixes.
5. **Validate the Service:** kubectl get services,endpoints -n ai-workloads. Confirm the selector matches the Pod labels, the target port is correct, and ready Pods appear as endpoints.

Practical example: An AI inference API is deployed but requests fail. The Pods show Running, yet the Service has no endpoints. The Service selector uses app: ai-api while the Pods are labeled app: inference-api. Correcting the label mismatch lets the Service route traffic to the Pods.

8.4 Verify Service connectivity and endpoints

If an application is running but users cannot reach it, verify the Service configuration, Pod labels, ports, endpoints, and external exposure.

Service exposure options

- **ClusterIP:** Internal connectivity between applications in the cluster.

- **LoadBalancer:** Exposes a Service through an Azure-managed load balancer, typically for external access.
- **Ingress:** Routes HTTP/HTTPS traffic to Services using hostnames and URL paths.
- **Selectors and ports:** Service selectors must match Pod labels, and the Service targetPort must match the port the application listens on.

Traffic flow (from the Pods up to the client)

1. **Pods:** Run the application containers.
2. **EndpointSlices:** Track the Pod IP addresses and ports associated with the Service.
3. **Service:** Provides a stable endpoint and routes traffic to eligible Pods.
4. **LoadBalancer or Ingress:** Exposes the app when external access is needed.
5. **Client:** Sends requests to the application endpoint.

Verify connectivity

```
kubectl get services -n ai-workloads
kubectl get pods --show-labels -n ai-workloads
kubectl get endpointslices -n ai-workloads
kubectl describe service <service-name> -n ai-workloads
```

To test the Service from your local machine before configuring external access, use port forwarding, then browse to http://localhost:8080:

```
kubectl port-forward service/<service-name> 8080:80 -n ai-workloads
```

Practical example: An AI inference API has healthy Pods, but requests fail. The Service selector does not match the Pod labels. After you correct the selector, the EndpointSlices list the eligible Pods and the Service routes requests correctly.

Module 3 key takeaways

- **Monitor key signals:** Track response latency, error rates, Pod restarts, and CPU and memory use to catch performance problems early.
- **Use the portal and kubectl together:** Azure Monitor and Container Insights give dashboards and broad visibility. kubectl supports detailed investigation of individual resources.
- **Understand Pod failure states:** ImagePullBackOff means an image-pull problem, CrashLoopBackOff means repeated container failures, and Pending usually means scheduling constraints or insufficient resources.
- **Verify Service connectivity:** Selectors must match Pod labels, ports and targetPort values must be correct, and EndpointSlices must list the expected ready Pods.
- **Test before exposing:** Use kubectl port-forward to validate connectivity before configuring public access through a LoadBalancer or Ingress.

LP-4: Develop AI solutions with Azure Cosmos DB for NoSQL

Module 1: Build queries for Azure Cosmos DB for NoSQL

9.1 Introduction to Azure Cosmos DB for NoSQL

Azure Cosmos DB for NoSQL is a globally distributed database service for applications that need fast data access, flexible JSON document storage, and scaling across regions. It suits modern apps, including AI solutions that store user data, chat history, metadata, and application state.

- **Global distribution:** Replicate data across Azure regions for availability and low-latency access worldwide.
- **Flexible JSON storage:** Store documents without a fixed relational schema.
- **Automatic indexing:** By default, all document properties are indexed to support efficient queries.
- **Request Units (RUs):** Measure the compute resources consumed by database operations. Throughput can be set manually or with autoscale.
- **Consistency levels:** Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual.
- **SDKs:** Python, .NET, Java, and JavaScript.

Practical example: An AI chatbot stores conversation history in Cosmos DB. Each conversation is a JSON document with the user ID, messages, timestamps, and model information. Cosmos DB indexes properties automatically, scales throughput with demand, and replicates data across regions when required.

9.2 Explore Azure Cosmos DB for NoSQL

Cosmos DB stores data as JSON items and organizes resources into accounts, databases, and containers. Understanding this hierarchy, choosing the right partition key, and configuring throughput are essential for scalable, cost-effective applications.

Resource hierarchy

- **Account:** The top-level Azure resource. Provides the endpoint, global distribution, and consistency configuration.
- **Database:** A logical namespace that organizes containers.
- **Container:** Stores JSON items and defines the partition key and indexing policy.
- **Item:** An individual JSON document, such as a user profile, chat message, or transaction.

Choose the right partition key

The partition key determines how data is spread across logical partitions. A good key supports efficient queries and distributes storage and workload.

- Choose a field with high cardinality (many distinct values), such as userId, tenantId, or categoryId, depending on the access pattern.
- Avoid keys that concentrate most requests on a few values, which creates hot partitions.
- Example: For a multi-user AI chatbot, userId is a suitable key when most queries fetch a specific user's conversation history.

Throughput and Request Units

- **Manual throughput:** A fixed number of Request Units per second (RU/s).
- **Autoscale throughput:** Adjusts automatically within a configured maximum to meet demand.
- **Request Units:** Measure the resources used by reads, writes, queries, and indexing-related work. A point read of a small item may cost around 1 RU, while writes and queries usually cost more. Actual cost depends on item size, indexing, and query complexity.

Practical example: An AI chatbot stores conversation items in Cosmos DB. You pick a partition key that matches how conversations are retrieved, monitor RU consumption, and choose manual or autoscale throughput based on workload patterns.

9.3 Implement the Azure Cosmos DB for NoSQL SDK

The SDK lets an application connect to a Cosmos DB account, create and manage resources, and run database operations. Reusing one client and choosing the right read operation improves performance and cuts cost.

Use the singleton client pattern

- **Reuse one CosmosClient:** Create it at application start-up and reuse it for the application's lifetime.
- **Connection management:** The client handles connections, connection pools, and request routing. A new client per request adds unnecessary overhead.
- **Lightweight handles:** get_database_client() and get_container_client() return handles to existing resources.
- **Idempotent setup:** create_database_if_not_exists() and create_container_if_not_exists() initialize resources safely without duplicates.

Point read vs. query

- **Point read, read_item():** Use when you know the item's unique id and its partition key. This is usually the most efficient way to fetch a specific item.

- **Query, query_items():** Use to filter or search items by their properties. Cost and latency depend on indexing, result size, and partition scope.

Example: In an AI chatbot, fetch a known conversation by item ID and partition key with read_item(). To find all of a user's conversations that meet certain conditions, use query_items().

9.4 Query Azure Cosmos DB for NoSQL: syntax, parameters, and partitions

Cosmos DB for NoSQL uses a SQL-like query language to retrieve and filter JSON documents. You can select specific properties, apply conditions, use string functions, and pass parameters safely from application code.

SQL syntax and filtering

- **SELECT and FROM:** Choose which properties to return and which container to query.
- **WHERE:** Filter by conditions such as category, price, or status. Combine conditions with AND or OR.
- **String functions:** CONTAINS() checks whether a string contains given text.
- Example queries: products in the electronics category priced below 500, and products whose name contains "Speaker".

Parameterized queries

Use placeholders such as @category and @maxPrice instead of inserting values directly into the query string.

- **Security:** Helps prevent query injection from untrusted input.
- **Reusability:** The same query runs with different parameter values.
- **Maintainability:** Keeps query logic separate from user-supplied data.
- In Python, query_items() runs the query with the supplied parameters.

The partition_key argument

The partition_key argument limits a query to one logical partition when the query is designed for it. For example, partition_key="electronics" is appropriate only if the container's partition key is /categoryId and "electronics" is a value of that key.

Single-partition vs. cross-partition queries

- **Single-partition (preferred):** Pass the partition key value with partition_key in query_items(). Cosmos DB routes the query to that partition, which generally uses fewer RUs and has lower latency. Include the partition key in the query design when it fits your access patterns.

- **Cross-partition:** Used when the query cannot be limited to one partition, for example when searching across users or categories. Cosmos DB may query multiple partitions, which raises RU consumption and possibly latency.
- The enable_cross_partition_query=True setting shown in the course is SDK-version dependent, so check the SDK version and behavior you use.

Practical example: An online store keeps products with /categoryId as the partition key. A query for categoryId = "electronics" can target that partition, and the store passes the category and maximum price (for example, electronics under ₹500) as parameters instead of concatenating user input. A query for products below a given price across all categories may need to span multiple partitions.

Module 1 key takeaways

- **Partition key:** One of the most important design decisions, because it affects data distribution and query performance. It generally cannot be changed in place for an existing container. Changing it may require migrating data to a new container.
- **Singleton client:** Reuse one CosmosClient to benefit from connection management, pooling, and caching.
- **Point reads:** Use read_item() when you know both the item ID and partition key. It is usually more efficient than a query.
- **Parameterized queries:** Use parameters instead of concatenating user input. This improves security and can support query-plan reuse.
- **Single-partition queries:** Prefer them when your access pattern allows. Cross-partition queries may consume more RUs.
- **RU monitoring:** Track RU consumption to find expensive queries and tune indexing, partitioning, and query patterns.

Practical scenario: An AI chatbot stores conversation documents in Cosmos DB. You choose a partition key aligned with conversation retrieval, reuse one SDK client, use point reads for known conversations, and monitor RU consumption to optimize frequent queries.

Module 2: Implement vector search on Azure Cosmos DB for NoSQL

10.1 Introduction to vector search

Vector search finds information by meaning and similarity instead of relying only on exact keyword matches. Cosmos DB for NoSQL supports vector search alongside business data, which suits AI apps such as semantic search and retrieval-augmented generation (RAG).

Integrated vector search

- Store vector embeddings next to regular JSON properties in the same container.
- Embeddings are numerical representations of text, images, or other content, generated by an embedding model.
- You can search for semantically similar content without a separate vector database.

Distance functions

- **Cosine:** Measures the angle between vectors. Commonly used for text embeddings.
- **Dot product:** Multiplies corresponding vector values. Useful when it matches the embedding model's similarity approach.
- **Euclidean:** Measures the straight-line distance between vectors.
- Choose the function that matches the embedding model and your search needs.

Vector index types

- **Flat:** Exact nearest-neighbor search. Suits smaller datasets or when exact results matter.
- **quantizedFlat:** Uses vector quantization to reduce the data processed during search, possibly with some accuracy trade-off.
- **DiskANN:** Approximate nearest-neighbor indexing designed for efficient search over larger datasets.

Hybrid search: Combines vector similarity with keyword searches or metadata filters, so results are both semantically relevant and aligned with business requirements.

Practical example: An AI support assistant stores support articles with their text embeddings. When a user asks "How can I reset my password?", vector search finds password-recovery articles even if their wording differs. Metadata filters then limit results to a particular product or language.

10.2 Store and retrieve embeddings

Cosmos DB can store embeddings alongside regular JSON data, so AI applications retrieve semantically similar documents while keeping the content and metadata in one database.

Configure the vector policy

The vector policy is defined when you create the container. It specifies how vector data is stored and searched.

- **path:** The JSON property holding the embedding, such as /embedding.
- **dataType:** The numeric format of vector values, such as float32, float16, or supported integer types.

- **dimensions:** The number of values in each embedding. It must match the embedding model's output.
- **distanceFunction:** How similarity is measured: cosine, dot product, or Euclidean.

Vector index types

- **flat:** Exact vector search for smaller datasets, subject to dimensionality limits.
- **quantizedFlat:** Compressed vector representations that reduce storage and search cost, with possible accuracy trade-offs.
- **diskANN:** Approximate nearest-neighbor search for larger vector datasets.
- The right index depends on dataset size, vector dimensions, accuracy needs, and performance requirements.

Design decisions

- **Exclude embeddings from ordinary range indexing:** The vector index handles vector arrays better, and excluding them reduces indexing overhead.
- **Plan before creating the container:** Vector policy and index choices have constraints, so validate dimensions, data type, and index type before deployment.
- **Choose data types carefully:** float16 uses less storage than float32, but check the effect on search quality.

Practical example: An AI support assistant stores troubleshooting articles. Each document has a title, article text, category, and an embedding from an embedding model. For a question, the app generates a query embedding and searches for similar article embeddings, optionally filtering by category.

10.3 Execute vector similarity queries for semantic search

Vector similarity search finds documents that are semantically similar to a user's question, even without the exact same words. Cosmos DB compares the query embedding with stored embeddings and returns the closest matches.

The query pattern

- **VectorDistance():** Compares a stored document embedding with the query vector.
- **ORDER BY:** Sorts by vector distance so the closest matches come first, according to the distance function and query syntax.
- **TOP 10:** Limits the output to the ten best matches.
- **@queryVector:** A parameter carrying the embedding generated from the user's question.

How semantic search works

1. The user asks a question, such as "WiFi not working."
2. An embedding model converts the question into a numerical vector.
3. Cosmos DB compares it with the document embeddings in the container.
4. The query returns the most relevant documents, which the app can show or pass to an AI model to produce an answer.

Best practices

- **Use the same embedding model** for document and query embeddings, with compatible dimensions, so comparisons are meaningful.
- **Always use TOP N:** Limit results (for example, TOP 10) to avoid unnecessary processing, higher RU use, and added latency.
- **Parameterize the vector:** Pass the query embedding through @queryVector instead of building the query string dynamically.
- **Interpret scores correctly:** The meaning of a score depends on the distance function and embedding model. With cosine similarity, higher generally means more similar. Do not assume a universal threshold for a good match.

Practical example: A support portal receives "WiFi not working." The embedding API converts it into a vector and Cosmos DB searches the stored troubleshooting articles. The app retrieves the top 10 relevant articles and can pass their content to an AI model to generate a grounded answer.

10.4 Combine vector similarity with metadata filtering and hybrid search

Vector search finds semantically similar documents, but applications often also need restrictions such as category, date, product type, or tenant. Metadata filtering narrows the results. Hybrid search combines semantic similarity with keyword search to improve relevance.

Filtered vector search

- **Metadata filters:** The WHERE clause restricts results using fields such as category and creation date.
- **Pre-filtering:** Suitable filters shrink the search space and keep results relevant to the current context.
- **Partition key:** If the filter includes the container's partition key, the query may be routed to a single logical partition. Pass partition_key in the SDK when appropriate.
- **Use case:** Find troubleshooting articles in a specific category or published after a certain date.

Hybrid search with Reciprocal Rank Fusion (RRF)

- **Vector similarity:** Finds content with similar meaning to the query.
- **Full-text search:** Finds content matching words, phrases, or technical terms.

- **RRF ranking:** Merges the rankings from both methods into one list, so results benefit from semantic relevance and keyword matching.
- Example: for "WiFi connection error E102", semantic matching helps with WiFi troubleshooting, while keyword matching prioritizes documents containing the exact error code E102.

When to use each approach

- **Filtered vector search:** When semantic similarity must be combined with specific metadata conditions.
- **Hybrid search:** When both meaning and exact keywords matter, such as error codes, product names, or technical identifiers.

Practical example: An AI support assistant filters troubleshooting documents by product category and date, then combines vector similarity with full-text ranking to find the most useful articles for the question.

10.5 Use the Change Feed to trigger embedding refresh

The Cosmos DB Change Feed captures changes to items in a container. An Azure Functions trigger can process those changes and regenerate embeddings when document content changes. This keeps embeddings in sync with the latest data without a manual refresh job.

How the workflow works

1. **Document updated:** A document is created or modified.
2. **Change Feed:** Cosmos DB records the change in its ordered change stream.
3. **Azure Functions:** A Cosmos DB trigger processes the change and generates a new embedding with an embedding model API.
4. **Update Cosmos DB:** The function saves the new embedding with the document.
5. **Lease container:** The trigger uses a lease container to track progress and coordinate work across function instances.

Selective embedding refresh

- **Detect meaningful changes:** Compare or hash the content fields used to generate embeddings.
- **Skip metadata-only updates:** Changes to fields such as timestamps or status may not need a new embedding.
- **Batch API calls:** For high volume, batching can reduce overhead when the embedding API supports it.

Error handling and reliability

- **Idempotent processing:** Processing the same change more than once must not give incorrect results or duplicate side effects.

- **Handle deletions:** Define how to remove or invalidate embeddings when documents are deleted. The standard latest-version change feed does not report deletes the way it reports inserts and updates, so use a suitable deletion strategy.
- **Persistent failures:** Use retry policies and a dead-letter mechanism to keep failures for investigation and reprocessing.

Practical example: An AI support assistant stores troubleshooting documents. When an article's content changes, an Azure Function generates a new embedding and updates the document. If only the article's display status changes, the function skips the embedding API call.

Module 2 key takeaways

- **Vector policy:** Defines the embedding path, data type, dimensions, and distance function when you configure the container. Dimensions must match the embedding model's output.
- **DiskANN index:** Supports approximate nearest-neighbor search and suits large-scale vector workloads.
- **Similarity queries:** Use VectorDistance(), ORDER BY, and TOP N to get the closest matches while limiting the result count.
- **Metadata filtering:** Restricts results by properties such as category, date, or access permissions, to keep results relevant to the required context.
- **Hybrid search with RRF:** Combines vector and keyword rankings. Useful for queries mixing natural language and exact terms such as error codes.
- **Change Feed with Azure Functions:** Detects document changes and triggers embedding refresh so vector data stays in sync with the source content.

Module 3: Optimize query performance for Azure Cosmos DB for NoSQL

11.1 Introduction to query performance optimization

Cosmos DB automatically indexes document properties, which makes it easy to start querying. As an application grows, tuning indexing policies, query patterns, and consistency settings improves performance and controls RU consumption.

- **Optimize indexing policies:** Automatic indexing simplifies development. With custom indexing, include only the properties your queries need to reduce indexing overhead and storage cost. Exclude embedding arrays from ordinary range indexing when appropriate, and configure a dedicated vector index for similarity search.

- **Understand AI query patterns:** AI apps commonly filter by metadata, retrieve by relevance, and search by vector embeddings. Design the indexing policy and partition key around these frequent access patterns.
- **Choose the right consistency level:** There are five levels: Strong, Bounded Staleness, Session, Consistent Prefix, and Eventual.
    - **Session:** Often suits user-facing AI apps because it supports read-your-writes within a session.
    - **Strong:** The strongest read guarantees, but can increase latency and affect availability in some scenarios.
    - **Eventual:** Replicas converge over time. Suits apps that tolerate temporarily stale reads.

Practical example: An AI support assistant stores document text, metadata, and embeddings. You index frequently queried metadata properties, use a vector index for semantic search, and choose session consistency so users see their own updates without needing strong consistency globally.

11.2 Understand indexes

Indexes let Cosmos DB retrieve data efficiently without scanning every document. The right index types and paths improve query performance and reduce RU consumption.

Index types

- **Range index:** Supports equality comparisons, range filters, and single-property sorting with ORDER BY.
- **Composite index:** Supports queries that sort by several properties, or combine filtering and sorting in ways that need a composite index.
- **Vector index:** Supports similarity search with VectorDistance(). Options are flat, quantizedFlat, and diskANN.
- **Spatial index:** Supports geographical queries, such as finding locations within a distance or region, using functions such as ST_DISTANCE and ST_WITHIN.

Indexing path syntax

Indexing paths define which JSON properties are indexed.

- **/*:** Indexes properties throughout the document, recursively.
- **/property/?:** Targets a scalar property, such as /category/?.
- **/array/[]:** Targets elements within an array, so applicable array queries can use the index.
- **Specific paths:** More specific rules can override broader ones, letting you customize indexing.

Practical example: An AI support application stores documents with category, createdDate, location, and embedding properties. Use range indexes for metadata

filters, composite indexes for supported multi-property sorting, a spatial index for location searches, and a vector index for semantic similarity.

11.3 Configure range and composite indexes

Range indexes support filtering and single-property sorting. Composite indexes speed up queries that combine several properties in filtering and sorting.

Range indexes for filters

- Support comparisons such as =, >, <, >=, and <=.
- Example: filter documents where documentType is pdf and uploadDate is after January 1, 2024.
- Help Cosmos DB locate matching documents without scanning every item.

Composite indexes for sorting

- Contain several properties in a defined order, for example relevanceScore DESC followed by uploadDate DESC.
- The definition must match the property order in the query's ORDER BY clause.
- The same composite index can be used in reverse when all sort directions are reversed. Mixed sort directions may need a separate index.

Optimize queries with filters and sorting

To get PDF documents sorted by upload date, a composite index can include the filter property documentType and the sort property uploadDate.

Important considerations

- Put equality-filter properties before range-filter properties when designing composite indexes.
- Include the required properties in the correct order and direction.
- Composite indexes add indexing and write overhead, so create them only for query patterns that benefit.

Practical example: An AI document-search app stores PDFs with a relevance score and upload date. A composite index on relevanceScore and uploadDate, both descending, helps retrieve the highest-ranked and most recent documents efficiently.

11.4 Tune vector indexes for embedding workloads

Different vector index types fit different dataset sizes and accuracy needs. The goal is to balance search speed, accuracy, storage, and RU consumption.

Choose the right vector index

- **flat:** Exact vector search. Suits smaller datasets, but supported dimensionality is limited.

- **quantizedFlat:** Compressed vector representations reduce the cost of storing and searching larger collections.
- **diskANN:** Approximate nearest-neighbor search for large-scale workloads where search performance matters.

Design decisions

- **Vector count and index thresholds:** Some index types have minimum data requirements to perform as intended. Below those thresholds, searches may use brute force. Check the current service requirements.
- **Exclude embeddings from range indexes:** Embedding arrays are better handled by the vector index, and excluding them reduces overhead.
- **Plan the vector policy carefully:** Vector indexing settings have creation-time constraints. Validate the embedding path, dimensions, data type, and index type first.
- **Choose the data type carefully:** float16 uses less storage than float32. Check whether the precision is enough for your search quality needs.
- **Use metadata filters:** Index properties such as category or tenant ID when those filters are common, and measure the effect on query performance and RU use.

Practical example: An AI support assistant stores 100,000 document embeddings. You use diskANN for large-scale approximate search, exclude embedding arrays from standard range indexing, and index metadata fields such as category for filtering. You then measure latency, recall, and RU consumption to fine-tune the setup.

11.5 Reduce RU costs with strategic indexing

Default automatic indexing makes querying easy but can increase storage and write cost as data grows. Strategic indexing means finding which properties your app actually queries and indexing for those access patterns.

Analyze queries before indexing

- **Filter properties:** Fields used in WHERE clauses, such as category, status, or userId.
- **Sort properties:** Fields used in ORDER BY clauses.
- **Property combinations:** Frequent queries that filter and sort on several fields may benefit from composite indexes.
- **Query frequency:** Prioritize frequent queries that consume significant RUs.

Apply selective indexing

- **Exclude unused properties:** You can exclude everything with "excludedPaths": [{"path": "/*"}] and then explicitly include the paths you need.
- **Index only what you query:** Avoid indexing large text fields or raw metadata that are not used for filtering or sorting.

- **Include required paths:** Keep the properties needed by important queries indexed, including the partition key when it is queried.
- **Use composite indexes selectively:** Add them for common multi-property patterns rather than every possible combination.
- **Important:** Test selective indexing carefully. A property excluded from indexing may not support the same query patterns efficiently.

Monitor query metrics

- **Index utilization:** Shows how effectively a query uses indexes.
- **Retrieved vs. output document count:** A large gap suggests many documents are examined but few are returned.
- **RU charge:** The RUs a query consumes. In SDK responses it is commonly available through request diagnostics or the x-ms-request-charge header.

Match indexing to the workload

- **Write-heavy apps:** Fewer indexes reduce indexing work and write cost, but queries may become more expensive.
- **Read-heavy apps:** Carefully chosen indexes improve query efficiency and reduce the cost of frequent reads.
- **Composite indexes:** Can improve supported query patterns, but each one adds storage and write overhead.

Practical example: An AI document-search app often filters by category and sorts by createdDate. You index those fields, add a composite index if query metrics show a benefit, and exclude large document content from ordinary range indexing when it is not queried directly.

11.6 Choose consistency levels for optimal performance

Cosmos DB offers five consistency levels that control how users see data updates across replicas. Choosing one means balancing data freshness, read performance, availability, and application requirements.

The five consistency levels

- **Strong:** Reads always return the most recent committed write. Use when strict consistency is essential. It can increase latency and reduce availability in some scenarios.
- **Bounded staleness:** Reads may lag behind writes, but the lag is limited by a configured time or number of versions.
- **Session:** Gives read-your-writes and consistent-prefix guarantees within a session. A common choice for user-facing apps.
- **Consistent prefix:** Updates are seen in order, but some recent updates may not be visible yet.

- **Eventual:** Replicas converge when no new writes occur. Suits cases where temporarily stale data is acceptable.

Choose the right level for AI applications

- **Session:** For chat apps and user profiles where users should see their own recent updates.
- **Eventual:** For analytics, background processing, and workloads that tolerate stale reads.
- **Strong:** For business operations that need the latest committed data visible immediately.
- **Vector search:** Eventual consistency may be acceptable for many search workloads if the app tolerates indexing and replication delays.

Session tokens and SDK clients

- **Session tokens:** Preserve read-your-writes behavior across client instances when needed. The SDKs generally manage them automatically within a client session.
- **Client configuration:** Create separate CosmosClient instances when the app needs different account-level consistency configurations.
- **RU consumption:** Strong and bounded staleness reads can consume more RUs than weaker levels. Validate the exact cost with metrics.

Practical example: An AI chatbot saves a new conversation message and immediately retrieves it for display. Session consistency ensures the user sees the message they just submitted, without requiring strong consistency across the whole application.

Module 3 key takeaways

- **Index types:** Range indexes support filtering and sorting, composite indexes support multi-property query patterns, vector indexes support similarity search, and spatial indexes support geographical queries.
- **Composite indexes:** The property order and sort directions must match the query pattern. Use them for frequent multi-property sorting queries.
- **Vector index selection:** flat for exact search on supported smaller workloads, quantizedFlat for compressed vector indexing, and diskANN for large-scale approximate nearest-neighbor search.
- **Selective indexing:** Exclude properties that need no ordinary indexing, including embedding arrays handled by a vector index, to reduce storage and write overhead.
- **Query metrics:** Monitor RU charge, index utilization, and documents retrieved vs. returned to find inefficient queries and indexing opportunities.
- **Session consistency:** Gives read-your-writes behavior within a session, which suits user-facing apps such as AI chatbots.

LP-5: Develop AI solutions with Azure Database for PostgreSQL

Module 1: Build and query with Azure Database for PostgreSQL

12.1 Introduction to Azure Database for PostgreSQL

Azure Database for PostgreSQL is a fully managed relational database service on Azure. Developers can use PostgreSQL and its familiar SQL capabilities without managing the underlying database infrastructure.

- **Managed PostgreSQL:** Azure handles routine maintenance, patching, backups, and infrastructure operations. High availability can be configured to match workload requirements.
- **Community compatibility:** Supports PostgreSQL and its ecosystem of tools, drivers, and extensions.
- **Independent scaling:** Compute and storage can be adjusted separately, although storage scaling options and limits depend on the service configuration.
- **PgBouncer connection pooling:** Reuses database connections and reduces connection overhead, particularly for applications that open many short-lived connections.
- **JSONB support:** Stores and queries flexible JSON data alongside relational tables, so structured and semi-structured data can live in the same database.
- **pgvector extension:** Adds vector storage and similarity search, so AI applications can store embeddings next to their business data.

Practical example: You build an AI-powered customer support application. Customer details and support tickets are stored in relational tables, JSONB holds flexible metadata, and pgvector stores document embeddings for semantic search. PgBouncer helps manage database connections when many users access the application at the same time.

12.2 Explore Azure Database for PostgreSQL

Azure provides the PostgreSQL database engine and manages much of the surrounding infrastructure, including backups, maintenance, and configured high-availability capabilities. Your main decisions are which compute tier to use and which managed capabilities to enable.

Choose the right compute tier

- **Burstable:** For development, testing, and workloads with intermittent CPU usage. Usually the most cost-conscious option for lighter workloads.

- **General Purpose:** For production applications that need consistent, steady compute and memory.
- **Memory Optimized:** For workloads that benefit from a large memory allocation, such as large in-memory datasets, memory-intensive analytics, and caching.

The compute tier affects performance, available resources, and cost. Choose it based on the application's actual workload instead of selecting the largest tier by default.

Understand the managed capabilities

- **Automated backups:** Azure takes backups and supports point-in-time restore within the configured retention period. The default retention is 7 days and it can be extended to a maximum of 35 days.
- **PgBouncer:** Built-in connection pooling on the General Purpose and Memory Optimized tiers. Availability depends on the selected service configuration, and it may need to be enabled.
- **Extensions:** Add capabilities without a separate database engine. Examples include pgvector (vector search), pg_trgm (similarity matching), and hstore (key-value data).

Practical example: An AI application stores customer records and vector embeddings in PostgreSQL.

- Start with Burstable for development and testing, where workloads are light and intermittent.
- Choose General Purpose for a production API that needs predictable compute and concurrency.
- Choose Memory Optimized when performance depends on keeping a large working set in memory.
- Enable and configure PgBouncer if the application creates many database connections.
- Set backup retention to meet recovery requirements.

12.3 Connect to Azure Database for PostgreSQL

Applications that connect to the database must authenticate their identity, encrypt network traffic, and satisfy the database's network access rules. The two main authentication options are Microsoft Entra ID and PostgreSQL native authentication.

Authentication options

- **Microsoft Entra ID:** Token-based authentication with no stored database password. The application obtains an access token and presents it when connecting.

- **PostgreSQL native authentication:** Uses a database username and password. Store the credentials securely, preferably in Azure Key Vault, rather than hardcoding them in application code.

How Entra token authentication works

1. The application requests a token from Microsoft Entra ID.
2. Entra ID returns the token.
3. The application connects to PostgreSQL over TLS.
4. PostgreSQL validates the token.

TLS encryption

- **TLS 1.2 or later:** Enforced by default. It encrypts data travelling between the application and the database.
- **Certificate validation:** In production, validate both the server certificate and the server hostname.
- **sslmode=verify-full:** In clients that support it, this setting verifies the certificate chain and confirms that the hostname matches the certificate. It protects against connecting to an impersonated database endpoint.

Network access

- **Public access:** Use firewall rules to allow only trusted client IP addresses or networks.
- **Private access:** Use supported virtual network (VNet) integration and private networking options to limit access to authorized network paths.

Practical example: An AI application running in Azure needs customer data from PostgreSQL. It obtains a Microsoft Entra access token, connects over TLS with certificate verification enabled (sslmode=verify-full), and reaches the database through an authorized network path.

12.4 Create and manage schemas

A database schema defines how data is organized into tables, columns, relationships, and constraints. Good schema design protects data integrity, simplifies queries, and supports reliable application development.

Table design decisions

- **BIGSERIAL:** Generates increasing integer IDs automatically. Useful for internal primary keys.
- **UUID:** A globally unique identifier. Useful when records must be identified across systems or for external IDs.
- **JSONB:** Stores flexible, semi-structured data in a queryable format. Useful for metadata whose structure varies between records.

- **TIMESTAMPTZ:** Time-zone-aware timestamps. Use it for all temporal data, such as when conversations, messages, or transactions occur.

Enforce data integrity with constraints

- **NOT NULL:** A required column cannot be left empty.
- **CHECK:** A value must satisfy a defined condition.
- **UNIQUE:** Prevents duplicate values in a column or combination of columns.
- **Foreign keys:** Maintain relationships between tables and prevent references to records that do not exist.
- **CASCADE and RESTRICT:** Control what happens to related rows when a referenced record is deleted or updated.
- **Composite primary keys:** Commonly used for junction tables.

Example: tables for an AI chatbot

```
CREATE TABLE conversations (
  id BIGSERIAL PRIMARY KEY,
  session_id UUID NOT NULL,
  user_id VARCHAR(255) NOT NULL,
  started_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP,
  metadata JSONB DEFAULT '{}'
);

CREATE TABLE messages (
  id BIGSERIAL PRIMARY KEY,
  conversation_id BIGINT NOT NULL REFERENCES conversations(id),
  role VARCHAR(50) NOT NULL,
  content TEXT NOT NULL,
  created_at TIMESTAMPTZ DEFAULT CURRENT_TIMESTAMP
);
```

- **conversations:** Stores the conversation ID, session ID, user ID, start time, and flexible metadata.
- **messages:** Stores each message with its role, content, and creation timestamp.
- **conversation_id:** A foreign key that links each message to its parent conversation.
- **DEFAULT CURRENT_TIMESTAMP:** Assigns a timestamp automatically when a row is inserted without one.

This design separates conversation-level information from individual messages, which makes it easy to retrieve all messages that belong to a conversation.

Practical example: When a user starts a chatbot session, the application creates a row in conversations. Every later question or answer is stored in messages with the matching conversation_id.

12.5 Query data

PostgreSQL provides powerful SQL features for relational data and flexible JSONB documents, plus retrieval patterns that let applications handle large datasets without unnecessary database work.

PostgreSQL-specific features

- **JSONB operators:** Extract and filter values from JSON data.
  - -> returns a JSON object or value.
  - ->> returns a value as text.
  - @> checks whether the JSON contains a specified structure.
  - ? checks whether a top-level key exists.
- **RETURNING:** Returns the affected rows directly after an INSERT, UPDATE, or DELETE, which avoids an additional query.
- **ON CONFLICT:** Supports upserts: insert a new row, or update the existing row when a unique constraint conflict occurs.

Example: upsert with ON CONFLICT

```
INSERT INTO user_preferences (user_id, key, value)
VALUES ('u1', 'theme', 'dark')
ON CONFLICT (user_id, key)
DO UPDATE SET
  value = EXCLUDED.value,
  updated_at = CURRENT_TIMESTAMP;
```

- If the (user_id, key) combination does not exist, PostgreSQL inserts a new row.
- If it already exists, ON CONFLICT updates the existing preference.
- EXCLUDED.value refers to the value proposed by the attempted insert.
- updated_at = CURRENT_TIMESTAMP records when the update happened.
- **Important:** The (user_id, key) columns must have a matching unique constraint or unique index.

Performance patterns

- **Keyset pagination:** Retrieves the next page by using the last row's timestamp and ID instead of skipping many rows with OFFSET. Useful for large message histories.
- **CTEs (Common Table Expressions):** Use WITH clauses to break complex queries into readable steps.

- **Recursive CTEs:** Query hierarchical data such as organization structures, nested categories, or parent-child relationships.

Example: keyset pagination

```
SELECT id, content, created_at
FROM messages
WHERE (created_at, id) < (:last_ts, :last_id)
ORDER BY created_at DESC, id DESC
LIMIT 20;
```

The (created_at, id) pair acts as a cursor that identifies the last row returned on the previous page. PostgreSQL returns the 20 rows after that cursor in descending order, and the ID acts as a tie-breaker when several messages share the same timestamp.

Practical example: An AI chatbot displays conversation messages in pages. Keyset pagination lets the application fetch the next 20 messages without repeatedly skipping earlier rows as the conversation grows.

12.6 Integrate SDKs and applications

Applications connect to PostgreSQL through database drivers such as Psycopg 3 for Python. To improve performance and reliability, reuse connections, parameterize queries, and use efficient methods for inserting large amounts of data.

Use Psycopg 3 for Python

- **Parameterized queries:** Use %s placeholders to pass values safely instead of concatenating user input into SQL strings. This helps prevent SQL injection.
- **Context managers:** Use Python's with statement so connections and cursors are cleaned up properly.
- **Error handling:** Catch database exceptions and handle connection failures gracefully.

```
cursor.execute(
  "SELECT * FROM conversations WHERE user_id = %s",
  (user_id,)
)
```

The parameter value is passed separately from the SQL statement.

Manage database connections

- **ConnectionPool:** Reuses database connections instead of opening a new one for every request, which reduces setup overhead.
- **PgBouncer:** Pools connections between applications and PostgreSQL and helps manage large numbers of client connections.

- **Timeouts and retries:** Set connection and query timeouts. Use exponential backoff for transient failures so that the database is not overwhelmed by repeated retries.

The usual architecture has two possible pooling layers: an application-side connection pool and PgBouncer. Configure them deliberately, because excessive pooling or incompatible settings can waste resources.

Improve batch performance

- **executemany():** Convenient for inserting or updating hundreds of rows. Actual performance depends on the driver and query configuration.
- **COPY:** PostgreSQL's bulk-loading command. Generally better for importing 10,000 or more rows.

Practical example: An AI application must import 20,000 customer records. Instead of issuing 20,000 separate inserts, it uses COPY for bulk loading. During normal operation, a connection pool handles concurrent API requests efficiently.

Module 1 key takeaways

- **Compute tier selection:** Burstable for development and intermittent workloads, General Purpose for balanced production workloads, and Memory Optimized for memory-intensive workloads such as analytics.
- **Microsoft Entra authentication:** Token-based access reduces dependence on stored passwords. Use sslmode=verify-full in production to validate the server certificate and hostname.
- **Schema design:** Use BIGSERIAL for auto-incrementing IDs, JSONB for flexible metadata, and database constraints to maintain data integrity.
- **Keyset pagination:** Retrieve subsequent pages with the last-seen values instead of large OFFSET values. This avoids the OFFSET performance penalty on large tables.
- **Upsert with ON CONFLICT:** Insert a new row or update an existing one on a unique-constraint conflict. This enables idempotent insert-or-update operations that are useful for managing AI application state.
- **Connection pooling:** Use the Psycopg ConnectionPool and the built-in PgBouncer to reduce connection overhead in high-throughput applications.

Module 2: Implement vector search with Azure Database for PostgreSQL

13.1 Introduction to vector search with PostgreSQL

Azure Database for PostgreSQL supports vector search through the pgvector extension. You can store numerical embeddings, compare their similarity, and retrieve relevant information directly from PostgreSQL without maintaining a separate vector database.

Understand pgvector

- **Vector data type:** Stores embeddings as arrays of numbers that represent text, images, or other content.
- **Similarity operators:** Compare embeddings to find items that are semantically similar.
- **Relational integration:** Store vectors alongside regular PostgreSQL data such as document titles, categories, customer IDs, and timestamps.

Where vector search is used

- **Semantic search:** Find documents by meaning rather than exact keyword matches.
- **Recommendations:** Identify products or content similar to items a user likes.
- **Retrieval-Augmented Generation (RAG):** Retrieve relevant documents from PostgreSQL and give them to an AI model as context for generating an answer.

Vector index types

- **IVFFlat:** Organizes vectors into lists (clusters) to speed up approximate nearest-neighbor (ANN) searches. It needs existing data to build the index and some tuning to balance speed and recall.
- **HNSW:** Builds a graph-based index for fast approximate nearest-neighbor search. It often gives strong search performance but can need more memory and take longer to build.

Both index types trade some accuracy for speed compared with exact nearest-neighbor search. Choose based on dataset size, latency requirements, memory, and acceptable recall.

Practical example: An AI support assistant stores troubleshooting articles in PostgreSQL. Each article has its text, a category, and an embedding from an embedding model. When a user asks "My Wi-Fi keeps disconnecting", the application converts the question into a vector and searches for similar article embeddings with pgvector. The most relevant articles are then passed to the AI model to generate a grounded answer.

13.2 Store and query embeddings with pgvector

pgvector lets PostgreSQL store embeddings next to document content and metadata in the same table, which makes it easier for AI applications to retrieve relevant information with a single query.

Design the schema

```
CREATE TABLE documents (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  content TEXT,
  category TEXT,
  embedding vector(1536)
);
```

- **id:** Unique identifier for each document.
- **title and content:** The document's title and text.
- **category:** Metadata that can be used to filter search results.
- **embedding vector(1536):** Stores a vector of 1,536 numbers. The dimension must match the output of the embedding model.

Distance operators

- **<-> (L2 / Euclidean distance):** Measures the straight-line distance between two vectors.
- **<=> (cosine distance):** Measures the difference in direction between vectors. The most common choice for text embeddings.
- **<#> (negative inner product):** Supports inner-product-based similarity searches. Because the result is negated, smaller values indicate a larger inner product.

For distance-based searches, a smaller distance generally means the vectors are more similar.

Example: find the five closest documents

```
SELECT title, content
FROM documents
ORDER BY embedding <=> '[query_embedding_values]'
LIMIT 5;
```

The <=> operator sorts documents by cosine distance and LIMIT 5 returns the five closest matches.

Practical example: When a user asks a question, the application generates a query embedding and retrieves the closest matching troubleshooting articles from the documents table.

13.3 Perform fast vector similarity search

When a table holds thousands or millions of embeddings, comparing the query vector with every stored vector becomes expensive. pgvector supports approximate nearest-neighbor (ANN) indexes that speed up similarity searches while accepting a small trade-off in accuracy.

IVFFlat index

- **How it works:** Groups vectors into lists (clusters) and searches selected lists instead of examining every vector.
- **Needs existing data:** It works best when the table already holds representative vector data, because the index needs data to build its clusters.
- **Advantages:** Generally faster to build and uses less memory than HNSW.
- **Best for:** Workloads where low index-building cost and low memory use matter.

```
CREATE INDEX documents_embedding_ivfflat
ON documents USING ivfflat (embedding vector_cosine_ops)
WITH (lists = 100);
```

lists = 100 sets the number of clusters. The right value depends on dataset size and workload.

HNSW index

- **How it works:** Builds a multi-layer navigable graph that leads efficiently toward nearby vectors.
- **Incremental indexing:** Can be created on an empty table and maintained as data is inserted.
- **Advantages:** Often gives higher recall and better query performance, but usually needs more memory and takes longer to build.
- **Best for:** Production similarity-search workloads where query performance and search quality are priorities.

```
CREATE INDEX documents_embedding_hnsw
ON documents USING hnsw (embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 64);
```

- **m:** The maximum number of connections per graph element. Higher values can improve search quality but increase memory use and build cost.
- **ef_construction:** The search effort during index creation. Higher values can improve index quality at the cost of a slower build.

Which index should you choose?

Requirement	Suitable starting point
Lower memory use and faster index builds	IVFFlat
Strong query performance and recall	HNSW
Small dataset, or exact results required	Search without an ANN index

Practical example: An AI support assistant stores 500,000 document embeddings. Start by testing HNSW for fast semantic search. If memory use or index-build cost becomes a concern, benchmark IVFFlat and compare query latency, recall, and resource consumption.

13.4 Manage the index lifecycle and embedding updates

Embeddings and vector indexes need ongoing maintenance as documents change, datasets grow, or embedding models are upgraded. A good migration strategy avoids unnecessary downtime and protects search quality.

Index maintenance

- **Monitor index usage:** Use the pg_stat_user_indexes view to check index scans and usage statistics. Check index size as well when assessing storage overhead.
- **Rebuild when necessary:** Significant changes in data distribution or index performance may justify a rebuild. Confirm this with query performance and resource metrics.
- **Use CREATE INDEX CONCURRENTLY:** Builds an index while normal inserts, updates, and deletes continue, which gives zero-downtime rebuilds. It reduces blocking but takes extra resources and has operational limitations.

Manage embedding updates

- **Batch updates:** Update embeddings in manageable transactions of about 1,000 to 5,000 rows, then tune the batch size by resource use and transaction duration.
- **Use a stale flag:** Mark documents whose source content changed and regenerate their embeddings asynchronously.
- **Use a parallel-column strategy:** When migrating to a new embedding model, add a separate vector column instead of overwriting the existing embeddings immediately.

Migration workflow

1. **Add a new vector column** for embeddings generated by the new model.
2. **Create an index on the new column,** using the chosen index strategy and deployment requirements.

3. **Backfill embeddings in batches** by processing existing documents with the new model.
4. **Validate search quality** by comparing relevance, latency, and resource use between the old and new approaches.
5. **Switch and clean up:** Point application queries at the new column once validation succeeds, then drop the old column and index after confirming they are no longer needed.

Practical example: An AI support application uses one embedding model, but a newer model gives better relevance. You create a new vector column, generate embeddings in batches, build the index, compare search results, and switch to the new column only after validation.

13.5 Run vector similarity search for semantic retrieval

Vector similarity search retrieves documents by meaning instead of exact keywords. In PostgreSQL, pgvector provides the vector search, and PostgreSQL full-text search can be combined with it to improve retrieval quality.

Refine vector search results

- **Metadata filtering:** Use WHERE clauses to restrict results by properties such as category, product, or publication date.
- **Distance thresholds:** Exclude results that are too far from the query embedding, so irrelevant documents are not returned.
- **B-tree indexes:** Index frequently filtered metadata columns to speed up filtering when the query plan can use them.

```
SELECT title, content
FROM documents
WHERE category = 'networking'
ORDER BY embedding <=> '[query_vector]'
LIMIT 5;
```

This query searches only the networking category and returns the five closest matches by cosine distance.

Hybrid search

- **Vector similarity search:** Finds documents that are semantically similar to the user's question.
- **Full-text search:** Finds documents that contain specific words or phrases. Especially useful for exact terms, product names, and error codes.
- **Reciprocal Rank Fusion (RRF):** Combines the ranked lists from both approaches into one result list, balancing semantic relevance with keyword matches. It also copes with the different scoring scales of the two methods.

Practical example: A user asks an AI support assistant, "Why am I getting Wi-Fi error E102?" Vector search retrieves articles about Wi-Fi connectivity problems even when they use different wording. Full-text search prioritizes articles that contain the exact error code E102. RRF merges both result lists so the answer is relevant both to the meaning of the question and to its specific technical identifier.

13.6 Implement retrieval patterns for RAG pipelines

Retrieval-Augmented Generation (RAG) combines information retrieval with an AI language model. Instead of relying only on what the model learned in training, the application searches its own documents, retrieves relevant content, and passes it to the model to generate a grounded response.

Design the RAG database schema

Separate the data into two tables:

- **source_documents:** The original document information, such as title, source URL, document type, and access permissions.
- **document_chunks:** Smaller sections of each document, including the chunk content, its embedding, and its position in the source document.

Key columns in document_chunks:

- **document_id:** A foreign key that links each chunk to its original document.
- **chunk_index:** The chunk's position in the document.
- **content:** The text that is sent to the AI model.
- **embedding vector(1536):** The numerical representation of the chunk. The dimensions must match the embedding model's output.
- **token_count:** Helps estimate how much text will be passed to the language model.

Separating source documents from chunks makes retrieval, citations, and document updates easier to manage.

Apply effective retrieval patterns

- **Fetch adjacent chunks:** If one chunk holds the answer, neighboring chunks can add context that would otherwise be missing.
- **Track token counts:** Combine relevant chunks while staying within the model's context-window limit. Leave room for the system instructions and the generated response.
- **Preserve source metadata:** Keep document IDs, titles, URLs, and chunk positions so the application can cite the original source.
- **Evaluate retrieval quality:** Measure precision, recall, and Mean Reciprocal Rank (MRR) to see whether the system retrieves the right information.

Retrieval metrics

- **Precision:** Of all the chunks retrieved, how many are relevant?
- **Recall:** Of all the relevant chunks available, how many did the system retrieve?
- **MRR:** Measures how highly the first relevant result appears in the ranked list. A higher MRR means the first relevant result tends to appear nearer the top.

Practical example: An AI support assistant answers a question about resetting Wi-Fi settings. PostgreSQL retrieves the most relevant document chunks with vector search, the application fetches neighboring chunks for context, sends the selected text to the language model, and uses the source document metadata to cite the troubleshooting guide.

Module 2 key takeaways

- **pgvector extension:** Adds vector data types and similarity operators to PostgreSQL so embeddings and business data are stored together.
- **Distance metric selection:** Choose cosine, Euclidean, or inner product based on the embedding model and search requirements. The index operator class must match the distance operator used in the query.
- **HNSW vs. IVFFlat:** HNSW often gives better recall and handles incremental inserts well, but uses more memory. IVFFlat generally builds faster and uses less memory, which suits relatively stable data.
- **Index lifecycle:** Monitor index usage and query performance. Rebuild indexes when measurements justify it, and use CREATE INDEX CONCURRENTLY where appropriate to reduce disruption.
- **Embedding migrations:** Use a parallel-column strategy when changing embedding models. Generate new embeddings, validate search quality, switch queries to the new column, and keep a rollback path until the migration is complete.
- **RAG retrieval quality:** Separate source documents from document chunks, track token budgets, preserve source references, and evaluate retrieval with precision, recall, and MRR.

Module 3: Optimize vector search in Azure Database for PostgreSQL

14.1 Introduction to optimizing vector search

Vector search lets AI applications find documents by semantic similarity instead of exact keyword matches. In Azure Database for PostgreSQL, the pgvector extension provides vector storage, similarity operators, and approximate nearest-neighbor indexes. Optimizing these features reduces query latency and improves search performance.

Key components

- **pgvector extension:** Stores embeddings and supports similarity comparisons directly in PostgreSQL, alongside relational data.
- **IVFFlat index:** Groups vectors into lists to speed up approximate search. It generally builds faster and uses less memory, but works best when built with representative existing data.
- **HNSW index:** Uses a graph structure for approximate nearest-neighbor search. It often gives strong query performance and recall, but typically needs more memory.
- **PgBouncer:** Pools database connections to reduce connection overhead when many application requests reach PostgreSQL at once.

What affects vector search performance?

- **Index selection:** Choose IVFFlat or HNSW based on dataset size, query latency, recall requirements, and available memory.
- **Configuration tuning:** Adjust index parameters and database resources according to measured workload performance.
- **Connection management:** Use connection pooling to handle concurrent application requests efficiently.
- **Query design:** Match the distance operator to the index operator class, and review query plans to confirm that the intended index is used.

Practical example: An AI support assistant stores 500,000 troubleshooting article embeddings. You create an HNSW index for fast similarity search, use PgBouncer to manage application connections, and measure query latency and recall. You then tune index parameters and metadata filters to improve results.

14.2 Tune PostgreSQL for pgvector

Vector search performance depends on database memory, query planning, storage, and the parameters of the pgvector index. Measure performance first, then tune settings for the actual workload.

Memory and query planner tuning

- **shared_buffers:** PostgreSQL's shared memory area for caching database pages. The right value depends on the server's memory and workload.
- **work_mem:** Memory available to individual sort and hash operations. Raising it can help complex queries, but excessive values can cause high memory use when many queries run at once.
- **random_page_cost:** Influences the planner's estimate of random disk reads. A lower value may suit SSD storage, but validate it with actual query plans.
- **effective_io_concurrency:** Helps PostgreSQL estimate and coordinate parallel I/O operations on supported storage.

Important: Values shown in course examples are illustrations, not universal recommendations. shared_buffers is a server-level setting that generally needs a server restart, so it cannot normally be changed with a session-level SET command. Configure server parameters through the supported Azure PostgreSQL configuration options.

Tune pgvector search parameters

- **ivfflat.probes:** Controls how many IVFFlat lists are searched. Increasing it generally improves recall but can increase latency.
- **hnsw.ef_search:** Controls how many candidate nodes HNSW explores during a search. Increasing it generally improves recall but raises query cost and latency.

```
-- For an IVFFlat index
SET ivfflat.probes = 10;

-- For an HNSW index
SET hnsw.ef_search = 100;
```

These are session-level settings. Tune the parameter that matches the index type in use and test it against representative queries.

Validate performance with EXPLAIN ANALYZE

EXPLAIN ANALYZE compares the planned query with its actual execution, including execution time and whether the expected index is used.

```
EXPLAIN ANALYZE
SELECT *
FROM products
ORDER BY embedding <=> '[query_vector]'
LIMIT 10;
```

Here <=> is cosine distance. Replace [query_vector] with a valid vector of the correct dimensions.

Practical example: An AI product-search application returns results slowly. You inspect the query plan, confirm whether the HNSW index is used, and test different hnsw.ef_search values. You compare latency and recall to find a good balance without unnecessarily raising memory or CPU use.

14.3 Choose and configure vector indexes

pgvector supports two commonly used approximate nearest-neighbor (ANN) indexes: IVFFlat and HNSW. Both speed up similarity searches by avoiding a full comparison with every stored vector, but they use different data structures and have different resource requirements.

IVFFlat: cluster-based index

- **How it works:** Groups vectors into clusters called lists. A search looks at selected lists instead of the entire dataset.
- **Faster builds:** Generally takes less time to build and uses less memory than HNSW.
- **Trade-off:** Search quality depends on how many lists are searched. Searching more lists can improve recall but increases latency.
- **Best for:** Limited memory budgets, frequent index rebuilds, or relatively stable datasets.

HNSW: graph-based index

- **How it works:** Creates a multi-layer graph that navigates efficiently toward vectors similar to the query.
- **Higher recall:** Often achieves better recall and strong query performance when configured properly.
- **Resource requirements:** Typically needs more memory and can take longer to build than IVFFlat.
- **Best for:** Read-heavy applications where low query latency and retrieval quality matter most.

How do you choose?

Requirement	Suitable starting point
Lower memory use and faster index builds	IVFFlat
High recall and fast similarity queries	HNSW
Exact results on a manageable dataset	Exact search without an ANN index

Any recall or latency percentages shown in course slides are illustrative, not guaranteed. Actual results depend on dataset size, vector distribution, index parameters, and query workload.

Practical example: An AI support application stores 500,000 document embeddings. Start with HNSW if fast retrieval and high recall are the priorities. If memory use or index-build cost becomes a concern, benchmark IVFFlat and compare recall, latency, and resource consumption.

14.4 Optimize data layout for vector search

Data layout affects how quickly PostgreSQL can filter records and retrieve similar vectors. Design the schema and indexes around your common search patterns so the database examines fewer irrelevant records.

Design the schema for filtered search

- **Structured columns:** Store frequently filtered attributes such as category, tenant_id, or status in dedicated columns, and add B-tree indexes where appropriate.
- **JSONB:** Use it for flexible attributes whose structure changes often or that are queried less frequently.
- **Composite indexes:** Create indexes on several columns when queries often filter by a combination of properties.

Optimize query execution

The intended workflow is:

1. **All rows:** The database holds documents and their embeddings.
2. **Metadata filter:** Apply conditions such as category or tenant to narrow the candidate set.
3. **Vector search:** Compare the query embedding with the candidate vectors.
4. **Top-K results:** Return the best matches, for example the top 10 documents.

Important: In PostgreSQL with pgvector, the actual execution order depends on the query plan and the index used. A metadata filter combined with an approximate vector index may be applied after the index scan, which can leave fewer results than requested. Check the query plan, and consider iterative index scans or a different indexing strategy when needed.

Handle large datasets

- **Selective filters:** Filters that eliminate many irrelevant records improve retrieval efficiency.
- **Post-filtering:** Can work for broad filters, but may return too few results when the filter excludes many of the vector candidates.
- **Table partitioning:** Consider it for very large datasets when there is a useful partitioning key, such as tenant or time period. Partitioning adds operational complexity, so use it when measurements justify it.

Practical example: An AI support application stores millions of knowledge-base chunks, and users search within a product category. You store category as a structured column, add a suitable B-tree index, and use vector search to retrieve the most relevant chunks within that category.

14.5 Scale for high-volume workloads

When an application has high traffic, you need to improve database performance without raising costs unnecessarily. Azure Database for PostgreSQL supports vertical scaling, read replicas, and caching to handle different kinds of workload pressure.

Vertical scaling (scale up)

- **Increase compute resources:** Move to a larger compute configuration when queries are slow because of CPU, memory, or I/O limits.
- **Memory Optimized tier:** Consider it for workloads with large working sets or indexes that benefit from staying in memory.
- **Monitor before scaling:** Review CPU utilization, memory pressure, storage I/O, query latency, and execution plans to find the real bottleneck.

Example: If vector similarity queries are slow because the database lacks memory, scaling up can improve index caching and reduce disk access.

Horizontal scaling (read replicas)

- **Offload read traffic:** Read replicas serve read-only queries, so reporting, analytics, and read-heavy workloads can use separate servers.
- **Improve scalability:** Send eligible read queries to replicas and keep writes on the primary server.
- **Replication lag:** Replicas use asynchronous replication, so recently written data might not be visible immediately.
- **Limit:** Azure Database for PostgreSQL Flexible Server supports up to five direct read replicas per primary server. Replicas scale reads; they do not increase the primary server's write capacity.

Caching

- **Cache frequently requested data:** Store commonly accessed results, such as popular product details or repeated AI retrieval results, in a low-latency cache.
- **Reduce database load:** Cache hits avoid repeated database queries, which lowers latency and PostgreSQL resource use.
- **Plan invalidation:** Update or expire cached data when its source changes, to avoid serving stale results.
- **Current Azure note:** Microsoft is retiring Azure Cache for Redis and recommends migrating to Azure Managed Redis. For new designs, check current service availability and evaluate Azure Managed Redis.

Practical example: An AI-powered support application has slow vector searches at peak traffic. First inspect query plans and resource metrics, then scale up if PostgreSQL is resource-constrained. Route suitable read-only workloads to read replicas, and cache frequently requested results to reduce repeated database work.

14.6 Connection optimization

When many application instances connect to PostgreSQL, opening a new database connection for every request wastes time and server resources. PgBouncer connection pooling lets applications reuse a smaller pool of database connections, which improves connection efficiency and scalability.

Understand PgBouncer connection pooling

- **Built-in pooling:** Azure Database for PostgreSQL Flexible Server supports PgBouncer on the General Purpose and Memory Optimized tiers. It may need to be enabled in the server configuration.
- **Transaction pooling:** A database connection is assigned to a client for the duration of a transaction, then returned to the pool for reuse.
- **Reduced connection overhead:** Especially useful for applications that run many short-lived transactions or keep many idle connections.

With PgBouncer, several applications share database connections instead of each keeping its own large set of PostgreSQL connections.

Application-level best practices

- **Balance pool sizes:** Size application connection pools according to the number of application instances and the database server's connection capacity. Do not let every instance create an excessively large pool.
- **Use SET LOCAL:** When setting PostgreSQL parameters for vector search, use transaction-scoped settings so they do not unintentionally affect another request after a pooled connection is reused.
- **Retry transient failures:** Use exponential backoff, which increases the delay between retries, to handle temporary connection failures without overwhelming the database.
- **Set timeouts:** Configure connection and query timeouts so requests do not wait indefinitely for database resources.

Practical example: An AI search application has 20 application instances, each serving many requests. Without pooling, each instance may create many connections and exhaust PostgreSQL's connection capacity. With PgBouncer, these client connections share a smaller pool of active database connections, which reduces overhead and lets the server handle concurrent requests more efficiently.

Module 3 key takeaways

- **Memory configuration:** Tune shared_buffers and work_mem according to available server memory and workload. Appropriate settings reduce disk I/O and improve query performance, but excessive memory allocation can hurt concurrent workloads.
- **Index selection:** Choose HNSW when high recall and fast searches are the priorities. Choose IVFFlat when faster index builds and lower memory use matter more.
- **Metadata filtering:** Create B-tree indexes on frequently filtered columns such as category or tenant_id. Check the query plan, because metadata filtering is not guaranteed to happen before approximate vector search.

- **Scaling strategy:** Scale up when CPU, memory, or I/O limits individual queries. For read-heavy workloads, add read replicas and caching to distribute demand and reduce repeated queries.
- **Connection pooling:** Use PgBouncer in transaction mode to reuse database connections and avoid connection exhaustion under high concurrency. Make sure the application is compatible with transaction pooling.

LP-6: Enhance AI solutions with Azure Managed Redis

Module 1: Implement data operations in Azure Managed Redis

15.1 Introduction to data operations in Azure Managed Redis

AI applications often need to retrieve data very quickly to give users a responsive experience. Azure Managed Redis is a managed, in-memory data service on Azure that lets applications access frequently used data with very low latency.

Why use in-memory caching?

- **Fast data access:** Redis keeps data in memory, so reads are much faster than repeatedly fetching the same data from a database or external service.
- **Reduced database load:** Frequently requested data is served from the cache instead of querying the primary database every time.
- **Better concurrency:** Caching helps keep applications responsive when many users send requests at once.

Common AI application use cases

- **User profiles:** Cache frequently accessed preferences and profile data to avoid repeated database queries.
- **Session data:** Store short-lived conversation state, session identifiers, and other temporary application data.
- **Cached model results:** Reuse results for repeated requests when the inputs and relevant context are equivalent. Apply expiration and invalidation rules to prevent stale answers.

Practical example: An AI chatbot retrieves a user's profile from PostgreSQL every time a message arrives. Instead, the profile is cached in Azure Managed Redis with a time-to-live (TTL). Later requests read the profile from Redis until the entry expires or is invalidated. This reduces repeated database queries and helps the chatbot respond faster under heavy traffic.

15.2 Explore Azure Managed Redis

Azure Managed Redis is a fully managed, in-memory data service. Two main decisions shape your design: how the application uses the cache, and which service tier provides the memory and performance you need.

Caching strategies

- **Data cache (cache-aside pattern):** The application checks Redis first. If the data is missing, it reads the database, stores the result in Redis, and returns it. When the underlying data changes, the cache entry must be invalidated or refreshed.
- **Content cache:** Stores frequently requested static content, such as page templates, headers, and configuration fragments, so the work is not repeated.
- **Session store:** Stores temporary user or chatbot session state under a session ID. Useful when different application instances may handle the same user's requests.

Service tiers

Azure Managed Redis offers four tiers with different memory and compute trade-offs.

- **Memory Optimized (8:1 memory-to-vCPU ratio):** More memory relative to compute. Suits development, testing, and workloads that need large cache capacity but moderate throughput.
- **Balanced (4:1 ratio):** A balance of memory and processing power. Suits general-purpose caching and session stores.
- **Compute Optimized (2:1 ratio):** More compute relative to memory. Best for high request rates, heavy command processing, and throughput-intensive applications.
- **Flash Optimized:** Keeps frequently accessed data in RAM and moves colder values to NVMe flash storage. Useful for very large datasets when lower cost matters more than consistently low latency.

Practical example: An AI chatbot caches user profiles, conversation sessions, and frequently requested responses.

- Choose Balanced for a typical workload that needs both memory and processing capacity.
- Choose Memory Optimized when the dataset is large but command throughput requirements are moderate.
- Choose Compute Optimized when request volume and command-processing performance are the main bottlenecks.
- Choose Flash Optimized when the dataset is very large and access patterns clearly separate frequently accessed data from rarely accessed data.

15.3 Client libraries and development best practices

Applications communicate with Azure Managed Redis through client libraries for different programming languages. To get low latency and handle high request volumes, choose the right client, reduce unnecessary network round trips, and avoid expensive commands.

Choose the right client library

- **Python:** redis-py
- **.NET:** StackExchange.Redis
- **Java:** Lettuce (supports efficient connections and asynchronous operations)
- **Node.js:** node-redis

Clustering consideration: The client must support the selected Redis clustering configuration. Cluster-aware clients can route commands to the correct nodes when you use Redis OSS Cluster-compatible configurations.

Development best practices

- **Keep values small:** Split large objects into smaller keys or fields where appropriate. This reduces network transfer and can improve response times.
- **Use pipelining:** Send several commands together and receive their responses in one batch, which reduces network round trips.
- **Avoid KEYS in production:** It can block Redis while scanning a large keyspace. Use SCAN for incremental, non-blocking iteration.
- **Co-locate the application and cache:** Deploy them in the same Azure region where possible to reduce network latency.
- **Monitor resource usage:** Track memory, CPU, client connections, and network bandwidth. Investigate sustained high utilization and scale when necessary.

Two patterns worth remembering

- **Pipelining multiple operations:** Instead of sending each command and waiting for its response, the client batches the commands and runs them together. This can reduce latency when retrieving several records.
- **Retrieving only the required fields:** Fetch just the needed field (for example, the name) from each user's Redis hash instead of the whole hash. This reduces the data transferred.

Practical example: An AI chatbot retrieves user profile information for every incoming request. Pipelining the commands that fetch several fields, and avoiding oversized values, reduces network overhead and improves response times during high concurrency.

15.4 Implement data operations: data types and connection

Azure Managed Redis stores data in memory and supports several data structures. Choosing the right data type simplifies application logic and improves performance.

Core data types

- **Strings (SET / GET):** Store and retrieve simple values such as names, tokens, cached API responses, or serialized JSON.
- **Hashes (HSET / HGET / HGETALL):** Store several field-value pairs under one key. Useful for user profiles and structured objects.
- **Lists:** Ordered collections of values. Useful for queues, recent activity, and ordered event data.
- **Numeric counters (INCR / DECR):** Atomically increase or decrease integer values. Useful for request counters, inventory tracking, and rate limiting.

Connect to Azure Managed Redis (Python, redis-py)

- **host:** The Redis endpoint of your Azure Managed Redis instance.
- **port:** The port configured for the endpoint. The course example uses 10000; verify the port shown for your own instance.
- **ssl=True:** Enables TLS encryption for traffic between the application and Redis.
- **decode_responses=True:** Returns text responses as Python strings instead of bytes.

Authentication details must also be configured, using the credentials or supported authentication method for your instance.

String and hash operations

- **String operation:** Store Alice's name under a key with SET and read it back later with GET.
- **Hash operation:** Store a name and an email as fields under one Redis key with HSET. Retrieve one field with HGET or all fields with HGETALL.

Practical example: An AI chatbot needs a user's name and email for each request. Instead of querying PostgreSQL repeatedly, the application caches the profile as a Redis hash. This reduces repeated database reads and improves response time.

15.5 Implement data operations: expiration and cache invalidation

Caching improves performance, but cached data must be managed correctly. Azure Managed Redis supports key expiration, key deletion, and the cache-aside pattern to keep cached data useful and reduce unnecessary database requests.

Manage key expiration

- **SETEX:** Stores a string value and sets its time-to-live (TTL) in seconds in a single operation.
- **EXPIRE:** Adds an expiration time to an existing key.
- **TTL command results:** -1 means the key exists but has no expiration; -2 means the key does not exist; a non-negative number is the remaining lifetime in seconds.

```
r.setex("user:1001:name", 300, "Alice")
```

This stores Alice's name for 300 seconds (five minutes). After that, Redis removes the key automatically.

Cache invalidation patterns

- **Time-based expiration:** Redis expires data after its TTL. Useful for temporary data such as session information or cached API responses.
- **Manual invalidation:** Delete or update cache keys when the underlying database record changes.
- **Cache-aside:** Check Redis first. On a cache miss, read the database, store the result in Redis with a TTL, and return it.
- **Pattern-based cleanup:** Use SCAN to iterate through matching keys and delete them in batches. Avoid KEYS on large production keyspaces because it can block Redis.

Cache-aside flow

1. **Check cache:** The application looks for the data in Redis.
2. **Cache hit:** If the data exists, return it immediately.
3. **Cache miss:** Fetch the data from the source database.
4. **Store in cache:** Save the result in Redis with an appropriate TTL.
5. **Return data:** Respond to the user with the result.

Practical example: An AI chatbot frequently reads user profile data from PostgreSQL. It checks Redis first. If the profile is cached, it returns immediately. Otherwise it reads PostgreSQL, caches the result for five minutes, and returns it. When the user updates the profile, the application invalidates or refreshes the cache entry so later requests do not receive stale data.

Module 1 key takeaways

- **Caching strategies:** Use cache-aside to reduce repeated database reads, content caching for frequently requested static data, and session storage for temporary user state.
- **Tier selection:** Choose a tier by memory, compute, and throughput needs. Memory Optimized favors memory capacity, Balanced suits general workloads, and Compute Optimized favors processing throughput.

- **Client libraries:** redis-py for Python, StackExchange.Redis for .NET, Lettuce for Java, and node-redis for Node.js. Make sure the client supports your clustering mode.
- **Connection configuration:** The course example uses port 10000 with TLS. Always verify the endpoint and port of your deployed instance.
- **Expiration management:** Use SETEX to store a value with a TTL in one step, or EXPIRE to add expiration to an existing key. Choose TTLs based on how often the source data changes.
- **Cache invalidation:** Combine expiration with explicit invalidation when the source data changes. Use SCAN for incremental key iteration instead of KEYS on large production keyspaces.

Module 2: Implement event messaging with Azure Managed Redis

16.1 Introduction to event messaging

Azure Managed Redis supports two messaging patterns: Pub/Sub and Redis Streams. Both let applications communicate through messages, but they differ in how messages are delivered and stored.

Pub/Sub: real-time notifications

- **How it works:** A publisher sends a message to a channel, and Redis delivers it to the clients that are currently subscribed.
- **Broadcast model:** Every active subscriber of the channel receives the message.
- **Fire-and-forget:** Messages are not retained for subscribers that are disconnected or unavailable.
- **Best for:** Real-time notifications, live status updates, and application events where missing an individual message is acceptable.

Redis Streams: durable message processing

- **How it works:** Messages are appended to a stream and stay available for consumers to read.
- **Persistence:** Consumers can read messages later, so they do not need to be connected when messages are published.
- **Consumer groups:** Several workers coordinate to process messages from the same stream.
- **Best for:** Background jobs, event-processing pipelines, and workloads that need message tracking and recovery.

Streams provide durable message storage within Redis, but they are not automatically a replacement for a fully managed event broker. Configure persistence, retention, acknowledgements, and recovery according to your reliability requirements.

Practical example: An AI chatbot processes user requests.

- When the chatbot updates a user's processing status, it publishes a Pub/Sub notification so connected clients can update their screens immediately.
- When a user uploads a document for AI analysis, the application adds a message to a Redis Stream. Background workers process the document and acknowledge completed messages through a consumer group.

16.2 Publish and subscribe to events with Redis Pub/Sub

Redis Pub/Sub lets applications send real-time messages to other applications through named channels. Publishers send messages, and subscribers receive them only while they are actively listening to the channel.

How Pub/Sub works

- **Publisher:** Sends a message to a named channel, such as ai:models:updated.
- **Channel:** A named communication path between publishers and subscribers.
- **Subscriber:** Listens to one or more channels and processes incoming messages.
- **At-most-once delivery:** Messages go to active subscribers, but Redis does not retain them for disconnected subscribers or guarantee redelivery.
- **Fire-and-forget:** The publisher does not wait for subscribers to confirm that they processed the message.

Code patterns

- **Publisher:** When a model is updated, the application publishes its name and version to the ai:models:updated channel.
- **Subscriber:** Listens for messages and clears the relevant model cache when an update notification arrives.
- **Pattern subscription:** Subscribing to the pattern ai:* receives messages from every matching channel, such as ai:models:updated and ai:jobs:completed.

When to use Pub/Sub

- **Cache invalidation:** Tell several API instances that cached data must be refreshed.
- **Real-time notifications:** Send status updates to connected clients.
- **Configuration broadcasts:** Tell services that application settings have changed.
- **Loose coupling:** Services publish events without needing to know which applications consume them.

Practical example: An AI application updates its embedding model. The deployment service publishes a notification to ai:models:updated. Multiple API instances receive

the event and clear their cached model information, so they can load the updated version.

16.3 Implement task queues with Redis Streams

Redis Streams is an append-only data structure that you can use to build task queues and event-processing pipelines. Unlike Pub/Sub, which delivers messages only to active subscribers, Streams keeps messages so workers can process them later.

What Redis Streams provide

- **Message storage:** Tasks are appended to a stream and remain available according to the stream's retention and Redis persistence configuration.
- **Consumer groups:** Distribute tasks among several workers, so different workers process different messages from the same stream.
- **Pending messages:** Messages that were delivered but not acknowledged stay in the pending entries list. They can be investigated and reassigned to another worker if the original worker fails.
- **Processing history:** Retained entries help with debugging and replay, provided they have not been trimmed or deleted.

Core commands

- **XADD:** Adds a new task to a stream and returns its unique message ID.
- **XREADGROUP:** Reads messages for a specific consumer group and consumer. The > ID requests messages not previously delivered to that group.
- **XACK:** Acknowledges successful processing and removes the message from the group's pending entries list.
- **XPENDING:** Inspects pending messages and their delivery information.
- **XCLAIM / XAUTOCLAIM:** Reassign eligible pending messages to another consumer, for example when a worker fails.

Walk through the code flow

1. **Add a task:** The producer adds a task containing a prompt and model name to the stream.
2. **Read tasks from a consumer group:** A worker requests up to five new messages and can wait up to five seconds for messages to arrive.
3. **Acknowledge completion:** After processing succeeds, the worker acknowledges the message so it is no longer pending for that consumer group.

Important: Redis Streams does not automatically retry every unacknowledged message. A message stays pending until it is acknowledged, or explicitly reclaimed and processed again. Implement retry limits and dead-letter handling for persistent failures.

Practical example: An AI application receives 10,000 documents for summarization. It adds each task to a Redis Stream. Multiple workers consume tasks through a consumer group, call the AI model, store the results, and acknowledge successful processing. If a worker crashes, another worker can reclaim its eligible pending tasks.

16.4 Choose between broadcast and coordinated distribution

Pub/Sub broadcasts events to everyone who is listening, while Streams distribute work among consumers. The choice depends on whether every subscriber needs the message, or whether exactly one worker should process each task.

Pub/Sub: broadcast messaging

- **Every active subscriber receives the message:** When a publisher sends an event to a channel, all clients currently subscribed to it receive it.
- **Fire-and-forget delivery:** Redis does not retain messages for disconnected subscribers and does not require processing acknowledgements.
- **Best for:** Cache invalidation, real-time notifications, and configuration updates.

Example: An AI application updates its model configuration and publishes an event. Every active API instance receives it and refreshes its local cache.

Redis Streams: coordinated work distribution

- **Consumer groups:** Several workers share the processing workload.
- **One consumer per message within a group:** A message is assigned to one consumer in a group instead of being broadcast to every worker. Separate consumer groups can independently process the same stream.
- **Acknowledgements:** After successfully processing a task, a worker calls XACK.
- **Failure recovery:** Unacknowledged messages stay pending. Workers can inspect and reclaim eligible messages with commands such as XPENDING and XAUTOCLAIM.

Example: An AI document-processing application receives 1,000 documents. It adds tasks to a Redis Stream, and several workers process different documents in parallel. Each worker acknowledges its task after processing succeeds.

Which pattern should you choose?

Requirement	Recommended pattern
Notify every active application instance	Pub/Sub
Broadcast cache invalidation events	Pub/Sub

Distribute background jobs among workers	Redis Streams
Track pending work and support recovery	Redis Streams
Retain messages for later processing	Redis Streams, with appropriate persistence and retention configuration

Module 2 key takeaways

The key decision is whether an event must be broadcast to multiple listeners or processed reliably as a task.

- **Pub/Sub for broadcast notifications:** Sends messages to all active subscribers. It is fire-and-forget: messages are not retained for disconnected subscribers and there are no processing acknowledgements. Best for cache invalidation, real-time status updates, and configuration-change notifications.
- **Streams for task processing:** Consumer groups distribute messages among workers so tasks run in parallel. Workers use XACK to mark messages as processed, XPENDING to inspect unacknowledged messages, and XCLAIM or XAUTOCLAIM to reassign eligible messages to another worker.
- **Broadcast vs. coordinated distribution:** In Pub/Sub every active subscriber receives the message. In Streams a message is normally assigned to one consumer within a consumer group, and separate groups can independently process the same stream.
- **Streams do not retry automatically:** The application must implement retry policies, reclaim pending messages, and handle persistent failures.
- **Combine both patterns:** A production AI application can use Streams to process document uploads, embedding generation, or AI inference jobs, and Pub/Sub to notify connected applications when processing status changes or cached data needs refreshing.

Module 3: Implement vector storage in Azure Managed Redis

17.1 Introduction to vector storage in Azure Managed Redis

Azure Managed Redis can act as a vector database by using the RediSearch module. Applications can store numerical embeddings, search for similar vectors, and filter results with metadata, all within Redis. This is useful when an AI application must find relevant information by meaning rather than by exact keywords.

Understand vector storage

- **Vector embeddings:** Numerical representations of text, images, or other data generated by an embedding model.
- **RediSearch:** Adds indexing and vector-search capabilities to Azure Managed Redis.
- **Low-latency search:** In-memory vector search can retrieve similar items quickly, depending on dataset size, index configuration, and workload.
- **Metadata filtering:** Store details such as document ID, category, title, or tenant ID with the embeddings so searches can be limited to relevant records.

Vector index types

- **FLAT:** Exact, brute-force similarity search. Suits smaller datasets or cases where exact results matter.
- **HNSW:** A navigable graph for approximate nearest-neighbor search. Generally suits larger datasets where fast retrieval matters, with some trade-off in recall.

Important deployment considerations

- **Enable RediSearch during provisioning:** You cannot add modules to an existing Azure Managed Redis instance after it is created.
- **Choose a supported tier:** RediSearch is available on the Memory Optimized, Balanced, and Compute Optimized tiers, but not on Flash Optimized.
- **Configure clustering correctly:** RediSearch requires the Enterprise clustering policy.
- **Use the appropriate eviction policy:** RediSearch requires NoEviction, so plan memory capacity carefully.

Consider these requirements before deploying a production vector-search workload.

Practical example: An AI support chatbot stores troubleshooting articles and their embeddings in Azure Managed Redis. When a user asks "My Wi-Fi keeps disconnecting", the application converts the question into an embedding and searches for similar article embeddings. Metadata filters can limit results to the relevant product category, and the retrieved articles are passed to an AI model to generate a grounded response.

17.2 Index and ingest vector data

Before you can search embeddings in Azure Managed Redis, you must create a vector index. The index tells Redis which fields to search, how vectors are represented, how many dimensions they have, and which distance metric to use.

Create a vector index

The Python example defines an index schema with RediSearch:

- **TextField("title"):** Makes the document title searchable as a text field.

- **VectorField("embedding", "HNSW", ...):** Defines a vector field and selects the HNSW indexing algorithm.
- **TYPE: FLOAT32:** Each vector element is stored as a 32-bit floating-point number.
- **DIM: 1536:** Each embedding must contain exactly 1,536 values, matching the embedding model's output.
- **DISTANCE_METRIC: COSINE:** Cosine distance compares query embeddings with stored vectors.
- **prefix=["doc:"]:** Limits the index to Redis keys that begin with doc: when you use the HASH data model.

The index automatically includes matching documents as they are added or updated.

Ingest vector data

Embedding models return vectors as numerical arrays, but Redis vector fields need the binary representation that matches the configured data type.

- **Convert embeddings:** In Python, tobytes() converts a NumPy array to bytes. Make sure the array uses float32 when the index is configured with TYPE FLOAT32.
- **Use pipelines:** Batch several Redis commands to reduce network round trips and improve bulk-ingestion throughput.
- **Keep schemas consistent:** The embedding dimension and data type used when storing vectors must match the index definition.

Practical example: An AI support application stores troubleshooting articles in Redis. Each article has a title and a 1,536-dimensional embedding. You create an HNSW index over keys with the doc: prefix, store each embedding as FLOAT32 bytes, and later use vector search to find articles similar to a user's question.

17.3 Query vector data

After creating a vector index, you can query it to find similar documents. Redis supports different search patterns depending on whether you want a fixed number of results, results filtered by metadata, or all vectors within a distance threshold.

The three query types

- **K-Nearest Neighbors (KNN):** Returns a fixed number of the closest vectors. For example, the five documents most similar to a user's question.
- **Hybrid search:** Combines metadata filtering with vector similarity. For example, search only documents in the docs category, then return the three closest matches.
- **Range search:** Finds vectors within a specified distance threshold. Useful when you want results that are similar enough, rather than always returning a fixed number.

KNN search: find the five nearest neighbors

- **KNN 5:** Requests the five nearest matching documents.
- **@embedding:** Specifies the vector field to search.
- **$vec:** The query embedding, supplied as a query parameter.
- **AS score:** Names the computed distance score.
- **sort_by("score"):** Orders the results by score.
- **dialect(2):** Enables the query dialect required by this vector-query syntax.

Hybrid search: filter by category

The query restricts the search to documents whose category tag matches docs, then retrieves the three nearest vectors among the eligible results. The category field must be indexed as a compatible TAG field for this filter syntax.

Important considerations

- **Matching dimensions:** The query vector must have the same number of dimensions as the indexed vectors.
- **Distance scores:** For cosine and Euclidean distance, lower scores mean closer vectors. Interpret scores according to the configured distance metric.
- **Query parameters:** Pass $vec as a parameter when executing the query. Do not embed dynamically generated vector values directly in query strings.
- **Result quality:** KNN always tries to return the requested number of results, but they may not all be relevant. Consider range thresholds or metadata filters where appropriate.

Practical example: An AI support chatbot receives "How do I fix Wi-Fi connection issues?" It converts the question into an embedding, searches for the five closest troubleshooting articles, and optionally filters by product category. The retrieved articles provide context for the AI-generated answer.

17.4 Optimize Redis data structures for vector storage

Azure Managed Redis can store application data and vector embeddings in different data structures. Choosing between HASH and JSON depends on the complexity of the data, memory use, and how the application needs to query it.

HASH storage

- **Structure:** Flat field-value pairs under a single Redis key.
- **Vector storage:** Embeddings are stored as binary bytes. For a FLOAT32 vector index, convert the embedding to its 32-bit floating-point byte representation (for example, a NumPy array with the correct data type and dimensions).
- **Advantages:** Simple structure and often lower memory overhead for flat records.

- **Best for:** User profiles, product metadata, and documents with a small number of fields.

JSON storage

- **Structure:** Supports nested objects, arrays, and hierarchical data.
- **Vector storage:** Embeddings are stored as numeric arrays, for example with tolist().
- **Advantages:** Complex data is easier to represent and work with.
- **Best for:** AI documents with nested metadata, product specifications, or many related attributes. For example, the embedding is stored as a JSON array and the specifications as a nested object.

Which structure should you choose?

Requirement	Suitable option
Flat fields and simple records	HASH
Nested objects and arrays	JSON
Vector stored as binary data	HASH
Vector represented as a numeric array	JSON

For vector search, make sure the index schema matches the stored representation, and that the vector dimensions and data type are correct.

Practical example: An AI product-recommendation application stores a product name, price, and embedding in Redis. HASH suits these flat fields. If the product also has nested specifications, reviews, and many metadata fields, JSON may be more convenient.

Module 3 key takeaways

- **Vector index schema:** VectorField defines the indexing algorithm, data type, dimensions, and distance metric. The dimensions must match the embedding model's output (for example, 1,536). Cosine distance is common for text embeddings, but the right metric depends on the model and requirements.
- **Query types:** KNN returns a fixed number of nearest neighbors, hybrid search combines vector similarity with metadata filters, and range search finds vectors within a distance threshold, where supported.
- **Data type and index selection:** FLOAT32 is a common choice for embeddings. Reduced-precision formats can lower memory needs, but validate search quality first. FLAT gives exact search for smaller datasets; HNSW gives approximate

search, and EF_RUNTIME controls the search effort at query time (higher values can improve recall but may raise latency).

- **HASH or JSON:** HASH is best for flat field-value data and embeddings stored as binary bytes. JSON is best for nested objects, arrays, and flexible metadata.
- **Consistency:** The stored embedding representation, dimensions, and data type must match the vector index definition.

LP-7: Integrate backend services for AI solutions

Module 1: Queue and process AI operations with Azure Service Bus

18.1 Introduction to messaging with Azure Service Bus

Azure Service Bus is a fully managed enterprise message broker on Azure. It lets applications communicate reliably without calling each other directly: the sender places a message in the messaging service, and the receiving application processes it when ready. This is especially useful for AI applications with long-running operations such as document processing, model inference, and background tasks.

Queues and topics

- **Queues (point-to-point):** A producer sends a message to a queue, and a competing consumer processes it. Suitable for distributing background jobs among workers.
- **Topics (publish-subscribe):** A producer sends a message to a topic, and multiple subscriptions each receive their own copy. Subscription filters decide which messages each subscriber receives.

Reliable message delivery

- **AMQP 1.0:** A standard messaging protocol for reliable binary message transfer between applications.
- **Message ordering:** Supported through features such as sessions, when messages must be handled in sequence.
- **Duplicate detection:** Identifies and discards duplicate messages within a configured time window.
- **Transactions:** Group supported messaging operations into one atomic operation.

These features help build robust, loosely coupled applications. Applications should still handle retries and, where possible, process messages idempotently.

Service Bus tiers

- **Standard:** Shared infrastructure and messages up to 256 KB.

- **Premium:** Dedicated resources for more predictable performance. Supports messages up to 100 MB over AMQP when large-message support is configured. The default maximum per entity is 1 MB.

Practical example: An AI application accepts documents for summarization. Instead of processing each document during the user's HTTP request, it sends a message to a Service Bus queue. A background worker retrieves the message, calls the AI model, stores the result, and updates the document status. To broadcast a model deployment notification to several independent services, a Service Bus topic with separate subscriptions is more appropriate.

18.2 Service Bus concepts and messaging in AI architectures

AI workloads often take unpredictable amounts of time to process. If an application calls an AI model synchronously for every request, users may wait a long time and the application can struggle when many requests arrive together. Service Bus separates request submission from background processing: producers send messages to a queue, and workers process them when capacity is available.

Why do AI applications need messaging?

- **Unpredictable inference latency:** AI model calls take different amounts of time depending on input size, model load, and service capacity. Synchronous calls block under load.
- **Decoupling:** The API that receives a request does not have to wait for the worker to finish.
- **Resilience:** Messages can stay in the queue while workers are temporarily unavailable, subject to message expiration and queue configuration.

Core messaging patterns

- **Load leveling:** Producers may send messages rapidly while workers process them at a steady rate. The queue absorbs temporary traffic spikes.
- **Competing consumers:** Several workers read from the same queue and share the workload. A message is typically locked for one consumer at a time, but it may be redelivered if processing fails or the lock expires.
- **Temporal decoupling:** Producers and consumers do not need to run at the same time. A message waits in the queue until a worker is available.

The architecture

- **Producers:** APIs or applications that submit AI tasks at a variable rate.
- **Queue:** Azure Service Bus stores messages until workers can process them.
- **Workers:** Background services that retrieve messages and perform tasks at a steady rate their capacity supports.

Practical example: 1,000 users submit documents for AI summarization within a short period. The API enqueues the jobs instead of processing every document immediately. Several workers then consume the queue and call the AI model as resources become available.

18.3 Choose between queues and topics with subscriptions

Service Bus provides two main messaging patterns: queues for point-to-point communication and topics with subscriptions for publish-subscribe communication. The choice depends on whether a message should be processed by one consumer or delivered to several independent systems.

Queues: point-to-point messaging

- **One message, one processing path:** Each message is delivered to exactly one consumer among the competing consumers.
- **Competing consumers:** Several workers receive messages from the same queue, so the workload scales horizontally.
- **Sessions and ordering:** Enable sessions when related messages must be processed in FIFO order within a session group.
- **Best for:** Background jobs, document processing, AI inference requests, and task queues.

Example: An application sends 1,000 documents to a queue. Several AI workers retrieve different messages and summarize the documents in parallel.

Topics with subscriptions: publish-subscribe messaging

- **Message fan-out:** A message published to a topic is made available to every matching subscription.
- **Independent subscriptions:** Each subscription acts like its own virtual queue, with its own messages and processing state.
- **Easy extensibility:** New subscribers can be added without changing the producer. Subscription filters control which messages each subscriber receives.
- **Best for:** Notifications, audit events, order updates, and broadcasting AI processing results to several systems.

Example: An AI document-processing service publishes a DocumentProcessed event to a topic. Separate subscriptions let the notification service inform users, the audit service record the event, and the analytics service update its metrics.

The Python code

```
# Queue: point-to-point
sender = client.get_queue_sender("requests")
receiver = client.get_queue_receiver("requests")
```

```
# Topic: fan-out
sender = client.get_topic_sender("results")
receiver = client.get_subscription_receiver(
  topic_name="results",
  subscription_name="notifications"
)
```

- **Queue example:** The sender and receiver both connect to the requests queue. The producer sends work and a worker receives and processes it.
- **Topic example:** The producer publishes messages to the results topic, and the receiver reads from the notifications subscription.

18.4 Structure messages for AI workloads: anatomy and correlation

In AI pipelines, messages should carry not only the data needed for processing but also metadata that helps you track, route, and troubleshoot requests. A well-structured message makes it easier to follow an AI operation from the API, through background processing, to the final result.

Message anatomy

- **Body:** The main payload. For AI workloads this is often a serialized JSON document with a prompt, model name, inference parameters, or a document-processing request.
- **Application properties:** Custom key-value metadata used for routing, filtering, and prioritization, such as model_name or priority.
- **System properties:** Built-in metadata such as message_id, correlation_id, content_type, and time_to_live.

Example: a message with a JSON payload and tracking properties

```
message = ServiceBusMessage(
  body=json.dumps({
    "prompt": "Extract parties...",
    "model": "gpt-4o",
    "temperature": 0.1
  }),
  content_type="application/json",
  message_id=str(uuid.uuid4()),
  correlation_id="req-abc-12345",
  application_properties={
    "model_name": "gpt-4o",
    "priority": "standard"
```

```
    }
)
```

- **body:** Contains the prompt and inference settings the worker needs.
- **content_type:** Identifies the payload format as JSON.
- **message_id:** Uniquely identifies the message. When duplicate detection is enabled on the queue or topic, Service Bus uses it to detect duplicate sends within the configured detection window.
- **correlation_id:** Connects related messages and operations across pipeline stages.
- **application_properties:** Metadata that consumers can use to filter or route work.

Correlation tracking

Imagine a document-extraction request that passes through an API, a Service Bus queue, an AI worker, and a result-processing service.

- Assign a correlation ID at the start of the request.
- Propagate it through related messages and log entries.
- Use it to trace the request end to end when investigating failures or latency.
- Use a unique message ID for each message when duplicate detection is required.

Practical example: A document-processing API sends a message to Service Bus. The worker uses the correlation ID to link its logs with the original request. If the sender retries the same message, duplicate detection can prevent a duplicate enqueue, provided the same message ID is reused and duplicate detection is enabled.

18.5 Structure messages for AI workloads: claim-check, TTL, and batching

AI workloads often involve large payloads such as documents, images, prompts, or extracted text. Instead of sending the whole payload through Service Bus, use the claim-check pattern: store the payload in Azure Blob Storage and send only a reference to it.

The claim-check pattern

1. **Producer uploads the payload:** Store the large document or file in Azure Blob Storage.
2. **Producer sends a message:** Put the blob URI and the required metadata in a Service Bus queue.
3. **Consumer receives the message:** The worker reads the blob reference from the message.
4. **Consumer retrieves the payload:** The worker securely downloads the file from Blob Storage and processes it.

Why use it?

- Keeps Service Bus messages small.
- Avoids message-size limits for large documents or AI inputs, and works within any tier's size limits.
- Separates message delivery from payload storage.
- Allows large files to be processed asynchronously.

Security note: The consumer must have authorized access to the blob, for example through a managed identity or a securely managed SAS token. Avoid exposing blob URLs that contain credentials in logs.

Message TTL and batching

- **Time to live (time_to_live):** Defines how long a message stays valid. It prevents stale AI requests from being processed after they are no longer useful.
- **Message batching:** ServiceBusMessageBatch groups several messages into one send operation, which reduces sending overhead and improves throughput.
- **Batch size limits:** Make sure a batch fits within the maximum message size supported by the Service Bus entity and tier.

Practical example: An AI application accepts large PDF files for summarization. The API uploads each PDF to Blob Storage and sends a Service Bus message containing the blob reference, document ID, and correlation ID. A background worker receives the message, downloads the PDF, calls the AI model, and stores the summary. If the request expires before processing begins, the message's TTL prevents it from staying valid indefinitely.

18.6 Process messages reliably: receive modes and settlement

When an AI application receives a message from Service Bus, it must decide when that message can be removed from the queue. If processing succeeds, the message should be completed. If it fails, the application decides whether to retry it or move it to the dead-letter queue.

Receive modes

- **Receive-and-delete:** The message is removed from the queue as soon as it is received. This gives the highest throughput with lower overhead, but there is no delivery guarantee: if the worker crashes before processing the message, the message is lost.
- **Peek-lock (default):** The message is temporarily locked while the worker processes it and stays in the queue until the worker settles it. It gives at-least-once delivery and supports retries and redelivery.

Example: An AI worker receives a message to summarize a PDF. With peek-lock the message is locked while the AI operation runs. If the worker crashes before completing the message, Service Bus makes it available for redelivery after the lock expires.

Settlement operations

- **Complete:** Processing succeeded. Service Bus removes the message from the queue.
- **Abandon:** A temporary problem prevented processing. The lock is released so the message can be delivered again.
- **Dead-letter:** The message cannot be processed successfully (for example, a malformed JSON payload). It moves to the dead-letter queue, with a reason, for investigation.
- **Defer:** Postpones processing while keeping the message in the entity. It can be retrieved later by its sequence number.

Example: peek-lock receive with settlement

```
for msg in receiver:
  try:
    result = run_inference(msg)
    receiver.complete_message(msg)
  except json.JSONDecodeError:
    receiver.dead_letter_message(
      msg, reason="MalformedPayload"
    )
  except TransientServiceError:
    receiver.abandon_message(msg)
```

- **Successful inference:** complete_message(msg) removes the processed message.
- **Malformed payload:** dead_letter_message() moves the message to the dead-letter queue with a reason such as MalformedPayload.
- **Transient service failure:** abandon_message(msg) releases the lock so the message can be retried.

Configure the receiver for manual settlement when you call these settlement methods explicitly.

Important reliability practice: Peek-lock provides at-least-once delivery, so a message may be delivered more than once. For example, a worker may save an AI result but crash before completing the message, and the redelivered message is processed again. Design message processing to be idempotent so repeated deliveries do not create duplicate records, payments, or other unintended side effects.

18.7 Process messages reliably: dead-letter queue and lock management

Sometimes a message cannot be processed because its payload is invalid, a dependency is unavailable, or the maximum delivery count has been exceeded. The dead-letter queue (DLQ) isolates these problem messages so they can be investigated without blocking normal processing.

The dead-letter queue (DLQ)

- **What it is:** An automatic subqueue that exists for every queue and topic subscription.
- **Why messages enter it:** The application dead-letters them explicitly, they exceed the maximum delivery count, or other configured conditions apply.
- **Diagnostic information:** Messages can carry dead_letter_reason and dead_letter_error_description, which help identify why processing failed.
- **Expired messages:** They enter the DLQ only when dead-lettering on message expiration is enabled. Otherwise they may be discarded.

Inspect and process dead-letter messages

```
with client.get_queue_receiver(
  queue_name="inference-requests",
  sub_queue=ServiceBusSubQueue.DEAD_LETTER,
  max_wait_time=10
) as dlq_receiver:
  for msg in dlq_receiver:
    print(msg.dead_letter_reason)
    print(msg.correlation_id)
    dlq_receiver.complete_message(msg)
```

- **ServiceBusSubQueue.DEAD_LETTER:** Selects the queue's dead-letter subqueue instead of its active messages.
- **dead_letter_reason:** The recorded reason for dead-lettering.
- **correlation_id:** Traces the message back to the original AI request or workflow.
- **complete_message(msg):** Removes the message from the DLQ after it has been inspected and handled.

In production, do not complete a DLQ message until you have recorded the required diagnostics or decided how to handle it. Depending on the cause, the message may need correcting and resubmitting, or it may be safe to discard.

Message lock management

- **Lock duration:** In peek-lock mode the message is locked while a consumer processes it. The default lock is 1 minute and the maximum is 5 minutes per lock period.

- **Automatic renewal:** AutoLockRenewer extends the lock for long-running AI operations, subject to its configured renewal period and service behavior.
- **Two-phase processing:** For very long operations, split the work into stages (for example, accept the request first and process it asynchronously) instead of holding a message lock for the whole operation.

Practical example: An AI document-processing worker receives an invalid JSON message that it cannot parse. Instead of retrying indefinitely, it dead-letters the message with a reason such as MalformedPayload. An operator later inspects the error details, corrects the source payload if possible, and resubmits the task.

Module 1 key takeaways

- **Messaging decouples AI components:** Queues absorb traffic spikes, producers can send faster than workers process, and background workers consume messages at a rate their capacity supports.
- **Queues vs. topics:** Queues distribute work among competing consumers; each message is generally processed by one consumer at a time, with possible redelivery on failure. Topics deliver messages to several independent subscriptions, so different services can process the same event.
- **Message structure matters:** A JSON body carries the payload, application properties support filtering and routing, the correlation ID tracks a request across services, and the message ID supports duplicate detection when it is enabled.
- **Claim-check for large payloads:** Upload the document or image to Azure Blob Storage and send only its URI in the message.
- **Peek-lock ensures reliability:** At-least-once delivery with Complete (remove), Abandon (release the lock for redelivery), Dead-letter (isolate unprocessable messages), and Defer (retrieve later).
- **Monitor the dead-letter queue:** Messages arrive there because of malformed payloads, exceeded delivery limits, or application-defined failures. Watch DLQ counts to detect broken deployments, investigate the reasons, and fix recurring problems instead of repeatedly resubmitting bad messages.

Module 2: Develop event-driven AI workflows with Azure Event Grid

19.1 Introduction to event-driven AI workflows

Azure Event Grid is a fully managed event-routing service that helps Azure services and applications react automatically when something happens. Instead of one application continuously checking another system for changes, Event Grid routes events to the services that need to respond. For AI applications, events might be a document being uploaded, a model deployment completing, or a data-processing job finishing.

Why use Azure Event Grid?

- **Event-driven architecture:** Services respond to events instead of repeatedly polling for changes.
- **Low-latency routing:** Subsecond latency to the configured destinations, with per-event pricing.
- **Scalable delivery:** Supports millions of events per second, so independent services can process events as they arrive.
- **Loose coupling:** The publisher does not need to know the implementation details of every subscriber.

Topics and event types

- **System topics:** Route events generated by Azure services, such as notifications when a storage resource changes.
- **Custom topics:** Let your applications publish their own events, such as DocumentUploaded, ModelUpdated, or InferenceCompleted.
- **Event subscriptions:** Define which events are delivered and where they go. Filters can limit delivery to specific event types or properties.

Push vs. pull delivery

- **Push delivery:** Event Grid sends events to a destination such as a webhook or Azure Function. Use it when the receiving service should be notified automatically.
- **Pull delivery:** The consumer connects to Event Grid and reads events when it is ready. Useful when the application needs more control over its processing rate. Pull delivery is supported by Event Grid namespace topics.

CloudEvents support: Event Grid supports the CloudEvents 1.0 standard, a common format for describing events across platforms and applications. This makes it easier to integrate Azure services with other systems without a proprietary event format.

Practical example: A user uploads a document to Azure Blob Storage for AI summarization.

1. Blob Storage emits a document-created event.
2. Event Grid routes the event to an Azure Function.
3. The Function places a processing request on Azure Service Bus.
4. A background worker retrieves the request, processes the document with an AI model, and saves the summary.

In this architecture Event Grid reacts to events, Service Bus distributes the work, and the worker performs the long-running AI operation.

19.2 Event Grid concepts and event-driven patterns for AI solutions

Event Grid lets applications and Azure services react when something happens. Instead of continuously checking whether data has changed, an application publishes or receives an event and triggers the next step automatically.

Core components

- **Event:** A notification that something happened, such as DocumentUploaded or ModelUpdated.
- **Event source:** The Azure service or application that generates the event.
- **Topic:** The endpoint that receives events from a source and makes them available for routing.
- **Event subscription:** Defines which events are delivered to a particular destination. Filters can limit delivery by event type or properties.
- **Event handler:** The destination that processes the event, such as an Azure Function, webhook, Service Bus, or queue.

How routing works

One topic can distribute events to several independent subscriptions.

1. An application publishes an event to the topic.
2. Event Grid evaluates the subscriptions and their filters.
3. Subscription A routes matching events to an Azure Function.
4. Subscription B routes matching events to a webhook.
5. Subscription C routes matching events to a queue for background processing.

Each subscription can handle the event for a different purpose, and the publisher does not need to know the details of every consumer.

Benefits for AI applications

- **Event-driven processing:** Trigger work automatically when new data arrives. It replaces polling with subsecond reactive triggers.
- **Independent scaling:** Different stages of an AI pipeline scale according to their own workload.
- **Fault isolation:** A downstream processing component can be handled independently of the component that generates the event.
- **Loose coupling:** New event handlers can be added without major changes to the event producer.

Practical example: A user uploads a PDF to Azure Blob Storage for AI analysis. Event Grid detects the blob-created event, an Azure Function validates the document, a Service Bus queue receives a processing request, and a background worker extracts the

text and calls an AI model. Event Grid routes the event, Service Bus distributes the work, and the worker performs the AI processing.

19.3 Work with event schemas and properties

Applications that communicate through Event Grid exchange events that describe something that has happened. The event schema defines how this information is structured, and event properties let Event Grid filter and route events to the correct destination.

The CloudEvents 1.0 schema

CloudEvents is an open CNCF standard for describing events consistently across applications and platforms. It is recommended for new implementations. The example below shows an event for a completed AI inference operation.

```
{
  "specversion": "1.0",
  "type": "com.contoso.ai.InferenceCompleted",
  "source": "/services/content-moderation",
  "id": "a1b2c3d4-e5f6-7890",
  "subject": "/pipelines/moderation/batch-42",
  "data": {
    "modelName": "content-classifier-v3",
    "resultLocation": "https://.../results.json",
    "status": "success"
  }
}
```

- **specversion:** The CloudEvents specification version, such as 1.0.
- **type:** Describes the event, such as com.contoso.ai.InferenceCompleted.
- **source:** The application or component that generated the event.
- **id:** Uniquely identifies the event within its source.
- **subject:** The specific resource or operation associated with the event, such as /pipelines/moderation/batch-42. It enables hierarchical, path-based filtering.
- **data:** The event payload, such as the model name, result location, and processing status.

The required context attributes are specversion, type, source, and id. The subject attribute is optional, but it is useful for filtering events by resource or processing path.

Event filtering options

Event subscriptions can filter events so a handler receives only the events it needs.

- **Type filtering:** Exact match on the event type, such as com.contoso.ai.InferenceCompleted.

- **Subject filtering:** Prefix or suffix path matching. For example, target events whose subject begins with /pipelines/moderation/.
- **Advanced filtering:** Operators on fields of the event data, such as data.status = "success" or data.modelName = "content-classifier-v3".

Filtering avoids unnecessary processing and lets separate components respond to different events.

Practical example: An AI moderation pipeline processes thousands of documents. When an inference operation completes, it publishes a CloudEvent with the model name, result location, and status.

- A notification handler receives events where data.status is success.
- An audit service receives all InferenceCompleted events.
- A moderation dashboard filters events under the /pipelines/moderation/ subject path.

19.4 Configure delivery and retry policies for reliable event processing

Event Grid delivers events to endpoints such as Azure Functions and webhooks. A destination may be temporarily unavailable or fail to process an event, so delivery policies tell Event Grid how to retry and where to send events that cannot be delivered.

Event delivery

- **HTTP POST:** Event Grid sends an HTTP POST request with the event to the configured endpoint.
- **Successful response:** HTTP status codes 200 to 204 within 30 seconds indicate successful delivery.
- **At-least-once delivery:** An event may be delivered more than once, so handlers must be idempotent: processing the same event again must not cause duplicate side effects.
- **Automatic retries:** Event Grid retries failed deliveries with exponential backoff (for example 10 seconds, 30 seconds, 1 minute, and so on, up to 24 hours).

Retry policy options

- **Maximum delivery attempts (1 to 30):** How many times Event Grid tries to deliver an event before giving up.
- **Event time to live (1 to 1440 minutes):** How long Event Grid keeps trying to deliver an event.
- **Exponential backoff:** Retry delays grow over time, which reduces pressure on a temporarily unavailable endpoint.
- **Dead-lettering:** Undeliverable events can be stored in Azure Blob Storage for later investigation and recovery.

The retry policy should reflect the workload. A time-sensitive AI notification may have a short useful lifetime, while an important document-processing event may need a longer delivery window.

The delivery flow

1. **Deliver event:** Event Grid sends the event to the destination.
2. **Check response:** If the endpoint returns a success response (200 to 204 within 30 seconds), delivery is complete.
3. **Retry:** If delivery fails, Event Grid retries according to the configured policy with exponential backoff.
4. **Check limits:** Event Grid checks whether the maximum attempts or the TTL has been reached.
5. **Dead-letter:** If delivery cannot succeed within the policy, Event Grid can route the event to Blob Storage, provided dead-lettering is configured.

High-throughput workloads

- **Output batching:** Group several events into one delivery request when the destination supports batching, to reduce per-request overhead.
- **Monitor failures:** Track delivery failures and dead-letter counts to spot unavailable endpoints, incorrect authentication, or broken deployments.
- **Implement recovery:** Inspect dead-lettered events, fix the underlying issue, and reprocess events when appropriate.

Practical example: An AI document-processing pipeline publishes a DocumentUploaded event. If the Azure Function that handles it is temporarily unavailable, Event Grid retries delivery. If it still cannot deliver the event within the configured policy, it stores the event in Blob Storage for investigation and recovery.

19.5 Publish custom events from AI applications

An application can publish its own events to an Event Grid custom topic. Other services subscribe to those events and react independently, which connects the stages of an AI pipeline without tightly coupling the applications.

Create a custom topic

- **Custom topic:** Provides an endpoint where an application publishes events such as InferenceCompleted, DocumentUploaded, or ModelUpdated.
- **CloudEvents 1.0:** The standard event format recommended for interoperable event-driven applications.
- **Input schema:** Create the topic so it accepts the CloudEvents schema, using cloudeventschemav1_0.

Authenticate securely

- **Microsoft Entra ID:** Lets the publisher authenticate without storing an Event Grid access key in application code.
- **Managed identity:** When the application runs on an Azure resource that supports managed identity, it obtains credentials automatically.
- **Permissions:** Grant the publisher identity the required Event Grid publishing role, such as EventGrid Data Sender, at the appropriate scope.

Example: publish a CloudEvent with the Python SDK

```
from azure.core.messaging import CloudEvent
from azure.eventgrid import EventGridPublisherClient

client = EventGridPublisherClient(endpoint, credential)

event = CloudEvent(
  type="com.contoso.ai.InferenceCompleted",
  source="/services/content-moderation",
  data={
    "requestId": "req-78901",
    "modelName": "classifier-v3",
    "status": "completed"
  },
  subject="/pipelines/moderation/classifier"
)

client.send(event)
```

- **type:** What happened, such as com.contoso.ai.InferenceCompleted.
- **source:** The component that generated the event.
- **subject:** The specific pipeline or resource.
- **data:** Application-specific information, such as the request ID, model name, and completion status.
- **client.send(event):** The application must call this after creating the event to actually publish it.

Event design best practices

- **Include correlation IDs:** Use the request ID to trace a workflow across services and logs.
- **Keep payloads small:** Include status and metadata in the event. Store large results in Blob Storage and publish a secure reference to them.
- **Publish at natural pipeline checkpoints:** Emit events when a document is accepted, inference completes, or a workflow fails.

- **Design for duplicate delivery:** Event handlers should process repeated events safely without creating unintended duplicate results.

Practical example: An AI content-moderation service completes an inference operation and publishes an InferenceCompleted CloudEvent with the model name, request ID, result location, and status. Event Grid routes it to a notification Function, an audit service, and a downstream processing workflow.

Module 2 key takeaways

- **Event Grid architecture:** Topics receive events from Azure services or custom applications, event subscriptions define which events go to which destinations, and event handlers (Azure Functions, webhooks, and other supported destinations) process them.
- **CloudEvents schema:** type identifies what happened, source identifies where the event originated, subject identifies the affected resource or operation, data carries application-specific information, and id uniquely identifies the event within its source.
- **Event filtering:** Type filtering selects specific event types such as InferenceCompleted, subject filtering matches prefixes or suffixes, and advanced filtering applies conditions to event properties such as data.status = "success". Filtering reduces unnecessary processing.
- **Retry and dead-letter policies:** Failed deliveries are retried according to the retry policy with exponential backoff. Undeliverable events can be stored in Azure Blob Storage when dead-lettering is configured. Because delivery can happen more than once, handlers must be idempotent.
- **Custom event publishing:** Use the Event Grid SDK or REST API. Publish at meaningful workflow checkpoints, include correlation IDs, keep payloads small, and keep large result files in Blob Storage with the event carrying a reference.

Module 3: Build serverless AI backends with Azure Functions

20.1 Introduction to serverless AI backends with Azure Functions

Azure Functions is a serverless compute service that runs code in response to events without requiring you to manage servers. It suits backend components that respond to HTTP requests, process messages, or react to changes in Azure services.

Key features

- **Serverless and event-driven:** Functions run when triggered by an event, such as an HTTP request, a Service Bus message, or a file uploaded to Blob Storage. Depending on the hosting plan, the app can scale automatically, including down to zero.

- **Triggers and bindings:** A trigger starts a function. Bindings simplify reading from or writing to supported services without writing all the connection-handling code yourself.
- **Cost efficiency:** Consumption-based plans can charge by executions and resource usage, which reduces cost for intermittent workloads. Other plans may charge for provisioned or always-ready capacity.
- **Language support:** Python, JavaScript/TypeScript, C#, and Java, subject to the chosen runtime and hosting configuration.
- **Hosting plans:** The selected plan affects scaling behavior, cold-start latency, available memory and compute, and cost.

Why use Azure Functions?

Azure Functions works well for short-running event handlers, API backends, scheduled jobs, and orchestration between Azure services. For long-running or compute-intensive AI tasks, choose an appropriate hosting plan or offload the work to a background-processing service.

Practical example: AI document processing. A user uploads a PDF for AI summarization.

1. The PDF is uploaded to Azure Blob Storage.
2. An event triggers an Azure Function.
3. The function validates the file and sends a processing message to Azure Service Bus.
4. A background worker processes the document with an AI model.
5. Another function can store the summary or notify the user when processing completes.

This design separates file uploads from long-running AI processing, which makes the application easier to scale and maintain.

20.2 Azure Functions hosting and scaling for AI workloads

Azure Functions can run AI-related tasks in response to events and scale with demand. AI workloads often need more initialization time and memory than ordinary functions, because they may load large libraries, initialize SDKs, open database connections, or load models. The hosting plan determines how instances scale, how resources are allocated, and how much cold-start latency to expect.

Cold start

- **What it means:** A cold start happens when Azure must allocate or initialize an instance before running a function.
- **Why AI workloads are affected:** Large dependencies, model initialization, and connection warm-up increase startup latency.

- **How to reduce it:** Use always-ready instances where supported, minimize unnecessary dependencies, and reuse clients and connections appropriately.

Flex Consumption vs. Premium

- **Flex Consumption:** A serverless plan that can scale to zero, supports configurable instance memory, and provides per-function scaling for supported trigger groups. You can configure always-ready instances to reduce startup delays. A strong option for event-driven workloads whose usage varies significantly.
- **Premium:** Provides pre-warmed instances and automatic scaling, which helps reduce cold starts. It also supports custom container deployment. Consider it when the workload needs consistently warm capacity and more predictable performance.

Scaling and concurrency

- **Trigger-based scaling:** Scaling behavior depends on the hosting plan and trigger type. Flex Consumption supports per-function scaling for many triggers, while certain triggers (HTTP, Blob, and Durable Functions orchestration) scale in defined groups.
- **Concurrency:** Lower concurrency can isolate resource-intensive invocations, but it may need more instances and raise cost.
- **Memory and CPU:** In Flex Consumption the instance sizes include 512 MB, 2,048 MB, and 4,096 MB, with typical allocations of 0.25, 1, and 2 vCPU respectively.

Handle long-running AI tasks

- **HTTP timeout:** An HTTP-triggered function has a maximum response time of about 230 seconds because of the Azure Load Balancer timeout, regardless of the configured function execution timeout.
- **Asynchronous request-reply:** Accept the request, enqueue the work, and return HTTP 202 Accepted with a way to check the status.
- **Queue-triggered processing:** A background function processes the task without keeping the original HTTP request open. Its execution is still subject to the hosting plan, function timeout configuration, and other runtime limits.

Practical example: An AI document-processing API receives a large PDF that takes several minutes to analyze.

1. An HTTP-triggered function accepts the document and returns 202 Accepted.
2. It sends a processing message to Azure Service Bus.
3. A queue-triggered function processes the document and calls the AI model.
4. The application stores the result and updates the job status.

5. The client checks a status endpoint to retrieve the completed result.

This avoids holding the HTTP request open during the whole AI operation.

20.3 Set up the local development environment for Azure Functions

Before deploying to Azure, you can build, run, and debug Azure Functions on your own machine. Azure Functions Core Tools provides the local Functions runtime, and Visual Studio Code makes development and debugging easier.

Core Tools and project structure

- **Azure Functions Core Tools:** Runs the Functions host locally so you can test triggers and bindings before deployment.
- **Python v2 programming model:** Uses a function_app.py file with decorators that define functions, triggers, and bindings directly in Python code. This is the recommended model for new Python applications.
- **host.json:** Contains configuration settings that apply to the Functions host.
- **local.settings.json:** Stores local application settings, including values used during development. These settings are not published to Azure automatically.

Local development workflow

- **Azurite:** Emulates Azure Storage services locally. Useful for testing Blob, Queue, and Table Storage triggers without a real Azure Storage account. The Python v2 worker may also need storage configuration even when your functions do not use a storage trigger directly.
- **F5 in Visual Studio Code:** Starts the local Functions host and attaches the debugger, so you can set breakpoints and inspect variables.
- **Run locally:** Use func start from the project directory to launch the Functions runtime with Core Tools.

Synchronize settings with Azure

```
func azure functionapp fetch-app-settings <function-app-name>
```

Run this command from the project root. It downloads the Function App's application settings into local.settings.json. Existing settings can be overwritten, so review the file afterward.

Security tip: local.settings.json may contain connection strings, keys, or other secrets. Do not commit it to source control.

Practical example: You are building a Python Azure Function that receives an AI document-processing request. You define the function in function_app.py, configure local settings, start Azurite if needed, and press F5 to debug. Once it works locally, you deploy it to Azure and configure the matching application settings in the Function App.

20.4 Create triggers and bindings for AI integration patterns: HTTP and Service Bus triggers

Azure Functions uses triggers to decide when a function runs. For AI applications, an HTTP trigger can accept an inference request directly, while an Azure Service Bus trigger can process queued jobs in the background. The key design decision is whether the caller needs an immediate response or the work can be processed asynchronously.

HTTP triggers for AI inference

- **@app.route() decorator:** Defines the HTTP endpoint, route, allowed methods, and authentication level.
- **Request and response:** The function reads the incoming request, runs the AI processing logic, and returns an HTTP response to the caller.
- **Function authentication:** AuthLevel.FUNCTION requires a function key, which helps restrict access to the endpoint. For production systems, also consider Microsoft Entra ID authentication and appropriate authorization.
- **Timeout consideration:** HTTP-triggered functions have a response limit of about 230 seconds imposed by the Azure Load Balancer. Long-running inference should generally use an asynchronous pattern rather than keeping the HTTP connection open.

Service Bus triggers for background processing

- **@app.service_bus_queue_trigger():** Automatically invokes a function when a message arrives in the configured Service Bus queue.
- **Concurrency control:** The maxConcurrentCalls setting in host.json controls how many messages a function instance processes concurrently. Tune it according to CPU, memory, downstream AI API limits, and database capacity.
- **Dead-letter queue (DLQ):** Messages that repeatedly fail can move to the DLQ after exceeding the queue's configured maximum delivery count. Inspect the DLQ to investigate malformed messages or persistent failures.
- **Automatic message handling:** With the normal Service Bus trigger behavior, a successful function execution lets the runtime complete the message. An unhandled exception causes the message to be abandoned for retry.

The HTTP trigger code

The example defines an endpoint at /api/classify that accepts HTTP POST requests.

- **Receive request:** req.get_json() reads the incoming JSON payload.
- **Run classification:** perform_classification() passes the input text to the AI processing logic.
- **Return a result:** func.HttpResponse() sends a JSON response with HTTP status 200.

- It uses auth_level=func.AuthLevel.FUNCTION, so callers need a valid function key unless another supported authorization configuration is used.

When to use each trigger

- **HTTP trigger:** When a caller needs an immediate result, such as a short classification or validation operation.
- **Service Bus trigger:** For queued workloads such as document processing, batch inference, and long-running AI tasks.
- **Bindings:** Simplify connections to supported services by declaring how a function receives or sends data. They reduce integration code, although complex operations may still need the relevant SDK.

Practical example: An AI document-classification application exposes an HTTP endpoint for quick classification requests. For large batches, the API places jobs in Service Bus and a queue-triggered function processes them in the background. If a message repeatedly fails, the DLQ lets the team inspect and recover it.

AI-200 exam tip: HTTP triggers serve direct requests, Service Bus triggers process queued work, maxConcurrentCalls controls per-instance concurrency, and the DLQ isolates repeatedly failing messages. Use asynchronous queue processing for long-running AI jobs instead of holding an HTTP request open.

20.5 Create triggers and bindings for AI integration patterns: output bindings and SDK clients

Azure Functions supports input and output bindings that simplify moving data between services. For AI applications, an output binding can save processing results to Blob Storage without you writing all the storage connection and upload code. For services that have no suitable binding, use the Azure SDK client directly.

Output bindings

- **Purpose:** Let a function send data to a supported destination such as Azure Blob Storage, Cosmos DB, or Service Bus.
- **Automatic handling:** The Functions runtime manages the binding connection and serialization according to the binding configuration.
- **Less boilerplate:** You can focus on business logic instead of manually managing every storage operation.

The Python example

The example processes a document job from a Service Bus queue and saves the result to Blob Storage.

- **Service Bus trigger:** @app.service_bus_queue_trigger() invokes the function when a message arrives in the document-jobs queue.
- **Blob output binding:** @app.blob_output() configures where the result is stored. The {rand-guid} expression generates a unique blob name.
- **Read the message:** msg.get_body().decode("utf-8") reads the message body and converts it to text.
- **Process the document:** extract_and_classify() performs the AI-related operation using the document URL.
- **Write the result:** output_blob.set(json.dumps(result)) passes the serialized result to the output binding for storage.

Declare the output parameter as func.Out[str] in the Python function signature, and make sure the configured Blob Storage connection is available in the Function App settings.

When to use SDK clients

Not every Azure service has a suitable Functions binding. For services such as Azure AI Document Intelligence, Azure OpenAI, and Azure AI Search, use the appropriate SDK client.

- **Reuse clients:** Initialize SDK clients at module level so invocations can reuse them. This reduces connection overhead and improves performance.
- **Use DefaultAzureCredential:** It supports local development and Azure-hosted authentication, depending on the available credentials and configuration.
- **Prefer managed identity in production:** Grant the Function App identity the required permissions instead of storing long-lived credentials in application code.

Practical example: A document-processing application receives a job through Service Bus. An Azure Function reads the document URL, calls an AI document-extraction service with its SDK client, and writes the extracted fields to Blob Storage through an output binding. Service Bus provides the queued work, the SDK client does the AI processing, and the Blob output binding stores the result.

20.6 Manage secrets and configuration in Azure Functions

Azure Functions apps need configuration values such as service endpoints, feature flags, database connection strings, and API keys. Instead of hardcoding them in application code, store them externally so they can be managed securely and changed independently of the code.

Application settings and Azure Key Vault

- **Application settings:** Exposed as environment variables. In Python, read them with os.environ.get("SETTING_NAME").

- **Key Vault references:** Store sensitive values in Azure Key Vault and configure a Function App setting that references the secret. The Azure platform resolves the reference, so your code reads it like a normal setting.
- **Versionless references:** When the reference does not specify a secret version, Azure uses the latest version. Updated values are normally refreshed within 24 hours; a configuration change or an explicit refresh can trigger an earlier update.

```
import os
database_url = os.environ.get("DATABASE_URL")
```

The application reads the setting without needing to know whether the value comes directly from an app setting or from a Key Vault reference.

Security tip: Enable managed identity for the Function App and grant it the required Key Vault permissions. Do not store API keys or passwords directly in source code.

Azure App Configuration

Azure App Configuration is a centralized store for non-secret configuration shared across applications and services.

- **Centralized management:** Manage common settings, service endpoints, and feature flags in one place.
- **Labels:** Keep different configuration values for environments such as dev, test, and prod.
- **Separation of responsibilities:** Use App Configuration for ordinary settings and feature flags, and Azure Key Vault for sensitive credentials.

The application must be configured to load the appropriate settings and labels. Refresh behavior depends on whether it uses an application configuration provider or platform-level configuration references.

Practical example: An AI document-processing Function connects to PostgreSQL and an AI endpoint.

- Store the AI endpoint and feature flags in Azure App Configuration.
- Store the database password or API key in Azure Key Vault.
- Configure the Function App to access these values securely at runtime.
- Use different configuration labels for development and production, so each environment uses the correct settings.

20.7 Configure identity and access for Azure Functions

Azure Functions often communicates with services such as Azure Service Bus, Blob Storage, Cosmos DB, and Key Vault. Configure authentication and authorization so the function can access only the resources it needs. For supported Azure services, the

recommended approach is managed identity, which avoids storing passwords or connection-string secrets in application settings.

Identity-based connections

- **Enable managed identity:** Turn on a system-assigned or user-assigned managed identity for the Function App.
- **Assign the required role:** Give the identity the minimum permissions needed on the target resource. For example, a Service Bus queue-triggered function typically needs the Azure Service Bus Data Receiver role.
- **Configure the endpoint:** For an identity-based Service Bus connection, use an application setting such as the one below. The double underscore (__) separates configuration properties.

```
ServiceBusConnection__fullyQualifiedNamespace
  = mynamespace.servicebus.windows.net
```

- **Runtime authentication:** Once the identity, permissions, and connection settings are configured correctly, the supported binding authenticates without a stored Service Bus connection-string password.

Azure Functions access keys

For HTTP-triggered functions, the authorization level controls which Functions key is required.

- **anonymous:** No Functions access key is required. Protect the endpoint with other appropriate security controls when necessary.
- **function:** Requires a function key, which grants access to that specific function endpoint.
- **admin:** Requires the master key, which also grants access to administrative runtime APIs. Treat this key as highly privileged.

Other key types:

- **Host keys:** Can authorize calls to all functions in the Function App.
- **System keys:** Used by specific extensions to protect extension-managed endpoints, such as certain webhook endpoints.

Important: Functions keys are access secrets, not a replacement for full user authentication and authorization. Use Microsoft Entra ID or another suitable authentication mechanism when your application needs stronger identity-based access control.

Configuration workflow

1. Enable the Function App's managed identity.

2. Assign the minimum required role on the target resource.
3. Configure the endpoint and connection settings in the Function App.
4. Run the function and verify that it can access the resource successfully.

Practical example: An Azure Function processes AI inference requests from a Service Bus queue.

- Enable the Function App's managed identity.
- Assign Azure Service Bus Data Receiver at the appropriate scope.
- Set ServiceBusConnection__fullyQualifiedNamespace to the Service Bus namespace.
- Configure the Service Bus trigger to use ServiceBusConnection.
- Verify that the function can receive and process messages without a connection-string secret.

Module 3 key takeaways

- **Choose the right hosting plan:** Flex Consumption suits many event-driven AI backends (automatic scaling, scale-to-zero, configurable always-ready instances). Consider Premium when reduced cold starts, pre-warmed instances, or custom container support matter. Decide based on workload duration, memory requirements, latency, scaling behavior, and cost.
- **Use asynchronous processing for long-running AI tasks:** HTTP-triggered functions have a response limit of about 230 seconds. For longer tasks, use the asynchronous request-reply pattern: an HTTP function accepts the request, places a message on a Service Bus queue, a queue-triggered function processes the AI workload, and the client checks a status endpoint for the result.
- **Triggers and bindings:** Triggers decide when a function runs (for example, an HTTP request or an available Service Bus message). Bindings simplify reading from or writing to supported Azure services. Supported trigger types can scale independently according to their workloads and the hosting plan.
- **Manage secrets with Key Vault references:** Store API keys and database credentials in Key Vault and reference them from Function App settings. Versionless references point to the latest secret version, but updates may take time to refresh, so plan secret rotation around that behavior.
- **Use identity-based connections:** Enable a system-assigned or user-assigned managed identity, assign the minimum required Azure RBAC roles, and configure supported triggers, bindings, and SDK clients to use identity-based authentication wherever possible. Avoid hardcoding credentials or storing long-lived secrets in code.

LP-8: Manage application secrets and configuration for AI solutions

Module 1: Manage application secrets with Azure Key Vault

21.1 Introduction to Azure Key Vault

Azure Key Vault is a managed Azure service that securely stores sensitive application information such as API keys, passwords, database connection strings, cryptographic keys, and certificates. Instead of hardcoding credentials in application code or configuration files, applications retrieve the secrets they need from Key Vault.

Core features

- **Centralized secret storage:** Keeps credentials and sensitive configuration in one secure service, separate from application code.
- **Microsoft Entra ID authentication:** Applications and users authenticate with Microsoft Entra ID. Azure RBAC decides who can access secrets, keys, and certificates.
- **Versioning:** Updating a secret creates a new version. Applications can reference a specific version or use a versionless reference to get the latest one.
- **Event Grid integration:** Key Vault can publish events when secrets or certificates change or approach expiration. Azure Functions or other event handlers can use them to automate rotation and renewal.
- **SDK support:** SDKs are available for Python, .NET, Java, JavaScript, and Go.

Security best practices

- **Apply the principle of least privilege:** Grant access only to the secrets an identity requires.
- **Prefer managed identity** over storing credentials used to access Key Vault.
- Enable appropriate logging and monitoring for secret access.
- Plan secret rotation carefully and make sure applications can refresh credentials when required.

Practical example: An AI-powered application needs an API key to call an AI service and a connection string for PostgreSQL.

- Store the API key and connection string in Azure Key Vault.
- Enable a managed identity for the application.
- Grant that identity the required Key Vault permissions.
- Let the application retrieve the secrets through the SDK or a supported Key Vault reference.

- When a credential needs rotation, update the secret and configure the application to pick up the new version.

This approach reduces the risk of credentials being exposed in source control or copied across configuration files.

21.2 Store and organize secrets, keys, and certificates

Key Vault stores sensitive information securely and controls who can access it. It supports three main object types: secrets, cryptographic keys, and certificates. Knowing their differences helps you choose how to protect application credentials and manage encryption.

The three object types

- **Secrets:** Sensitive strings such as API keys, passwords, and database connection strings. Each secret can be up to 25 KB.
- **Keys:** Used for cryptographic operations such as encryption, decryption, signing, and verification. Applications can use Key Vault cryptographic operations without directly accessing the protected key material.
- **Certificates:** X.509 certificates used for TLS, application identity, and secure communication. Key Vault supports certificate versioning, certificate authority integration, and automatic renewal for supported issuers.

Organize vaults and control access

- **Separate vaults by application and environment:** For example, keep distinct vaults for development and production. This isolates permissions and limits the impact of accidental exposure.
- **Key Vault Secrets User:** Grants permission to read secret contents. Appropriate for application identities that only need to retrieve secrets.
- **Key Vault Secrets Officer:** Lets operators manage secrets, including creating and updating them, but does not grant permission to manage access permissions on the vault.

Apply the principle of least privilege: applications read only the secrets they need, while authorized operators manage secret values and their lifecycle.

Soft delete and purge protection

- **Soft delete:** Deleted vaults and objects stay recoverable for a configured retention period. New Key Vaults have soft delete enabled by default.
- **Purge protection:** Prevents permanent deletion of a soft-deleted vault or object before its retention period expires. Strongly recommended for production. Once enabled, it cannot be disabled.

Together these features protect against accidental deletion and malicious attempts to permanently destroy secrets or keys.

Practical example: An AI application needs an Azure AI API key, a PostgreSQL credential, and a TLS certificate.

- Store the API key and database credential as secrets.
- Use keys for cryptographic operations when encryption or signing is required.
- Store and manage the TLS certificate as a certificate.
- Give the application's managed identity the Key Vault Secrets User role if it only needs to read secrets.
- Enable purge protection for production and keep production secrets separate from development credentials.

21.3 Retrieve secrets using Azure SDK client libraries

Key Vault lets applications retrieve secrets at runtime instead of storing credentials in source code or configuration files. The Azure SDK client libraries provide methods to authenticate, retrieve secret values, inspect metadata, and handle errors.

SecretClient operations

- **SecretClient:** The Python SDK client that communicates with Key Vault. It is initialized with the vault URL and a credential object.
- **get_secret():** Retrieves a secret's value together with metadata such as its version, creation date, and content type.
- **list_properties_of_secrets():** Lists secret properties without retrieving the secret values. Useful for discovering which secrets exist.

The code example

```
secret = client.get_secret("openai-api-key")
api_key = secret.value
version = secret.properties.version
created = secret.properties.created_on
```

- **get_secret("openai-api-key"):** Retrieves the named secret from the vault.
- **secret.value:** The actual secret value, such as an API key.
- **secret.properties.version:** Identifies the specific version retrieved.
- **secret.properties.created_on:** The creation timestamp of that version.

The application can then use the API key to authenticate to an AI service. Never print the secret value in logs or expose it in an error response.

Handle errors correctly

- **ResourceNotFoundError:** The secret or version does not exist. Check the secret name, vault URL, and version. Retrying without correcting the configuration will not help.
- **HttpResponseError:** The service returned an HTTP error. Check the status code. For example, 403 may indicate missing permissions, while server-side errors may be transient.
- **ServiceRequestError:** A network or connection problem prevented the request from reaching the service. Retry transient failures with bounded retries and exponential backoff.

Authentication best practice

Use DefaultAzureCredential so the application can authenticate with a supported local development credential or with its managed identity when running in Azure. Grant the identity the required Key Vault permissions, such as the Key Vault Secrets User role when it only needs to read secrets.

Practical example: An Azure Function needs an API key to call an AI model.

- The Function authenticates to Key Vault using managed identity.
- SecretClient.get_secret() retrieves the API key.
- The Function uses the key to call the AI service.
- Errors are handled according to whether they are configuration, authorization, or transient network failures.

21.4 Handle secret versioning and rotation

Key Vault versioning lets applications update credentials without changing the secret's name. When a secret value changes, Key Vault creates a new version. Applications can retrieve the latest version or request a specific version when needed.

Secret versioning

- **set_secret():** Creates a new version of a secret when a value is set or updated.
- **get_secret():** Retrieves the latest enabled version by default.
- **Specific versions:** Applications can retrieve an older version for validation, troubleshooting, or controlled rollback.

```
# Create two versions of the same secret
v1 = client.set_secret("db-key", "old-value")
v2 = client.set_secret("db-key", "new-value")

# Retrieve the latest version
latest = client.get_secret("db-key")

# Retrieve a specific version
```

```
old = client.get_secret(
  "db-key",
  version=v1.properties.version
)
```

Both versions share the name db-key but each has its own version identifier. The default retrieval returns the latest enabled version.

Secret rotation strategies

- **Manual rotation:** An operator updates a credential and creates a new Key Vault secret version.
- **Automated rotation:** Azure Event Grid notifies an Azure Function when a secret approaches expiration. The function runs custom logic to rotate the underlying credential and update Key Vault. Event Grid provides the trigger; your automation performs the rotation.
- **Dual-credential rotation:** When a service supports two active credentials, rotate one while the other stays valid. Applications get time to adopt the new credential before the old one is disabled.

Zero-downtime credential updates

- **Refresh credentials:** Applications should be able to obtain the latest secret version instead of caching an old value permanently.
- **Retry after authentication failure:** If a credential becomes invalid, refresh it and retry the operation where it is safe.
- **Use expiration events:** Configure expires_on and an Event Grid subscription to start rotation before the credential expires. Key Vault emits a SecretNearExpiry event 30 days before expiration.

Important: Creating a new Key Vault version does not automatically update the underlying database password or API credential. Your rotation workflow must update the target service and make sure applications can retrieve and use the new value.

Practical example: An AI application connects to PostgreSQL with a password stored in Key Vault. Before the password expires, an Azure Function creates a new password, updates PostgreSQL, and stores the new value as a new Key Vault secret version. The application refreshes its credential and reconnects without a full redeployment.

21.5 Implement caching strategies to reduce Key Vault calls

Retrieving secrets from Key Vault on every request adds network latency and can lead to throttling. Caching secrets in application memory reduces repeated calls, improves response time, and helps the application stay resilient during temporary service disruptions. Microsoft recommends caching secrets and reusing them whenever possible.

Why cache secrets?

- **Reduce throttling:** Key Vault has transaction limits. For example, the documented limit for secret operations and other vault transactions is generally 4,000 transactions per 10 seconds per vault per region, subject to operation-specific limits.
- **Improve performance:** Reading a cached secret from memory avoids a network request.
- **Handle traffic spikes:** The application can keep using a valid cached credential during temporary Key Vault connectivity issues.
- **Control freshness:** A cache TTL decides how long the application uses a cached value before refreshing it.

When Key Vault returns HTTP 429 because of throttling, clients should use exponential backoff instead of retrying immediately.

The caching example

The Python code implements a simple per-process in-memory cache.

- **_cache:** A dictionary that stores retrieved secret values and their timestamps.
- **_ttl = 900:** A cache lifetime of 900 seconds (15 minutes).
- **time.monotonic():** Measures elapsed time reliably for TTL calculations.

The function works like this:

1. Check whether the requested secret exists in the cache.
2. If the cached entry is still within its TTL, return the cached value.
3. If the entry is missing or expired, call client.get_secret(name).
4. Store the retrieved value and the current timestamp in the cache.
5. Return the secret to the application.

Choose the right cache strategy

- **Per-process cache:** The simplest approach. Each application instance keeps its own copy of the secret. A good starting point for many applications.
- **Shared distributed cache:** Can reduce duplicate Key Vault reads across many instances, but adds infrastructure and security considerations. Secret values must be protected appropriately.
- **Event-based invalidation:** Azure Event Grid can notify applications when a secret gets a new version. An event handler can refresh or invalidate cached entries, which reduces reliance on periodic polling.

Choose an appropriate TTL

A longer TTL means fewer Key Vault requests, but a cached secret can stay stale for longer. A shorter TTL improves freshness but increases Key Vault traffic. For example,

with a 15-minute TTL the cache can keep using an old value until it expires, so if a credential rotates during that time the application may need to refresh the secret sooner. A one-hour TTL for a 90-day rotation schedule is only an illustrative configuration. Select the TTL based on rotation frequency, security requirements, and acceptable staleness.

Important: If authentication fails because a credential was rotated, refresh the cached secret and retry the operation when it is safe. Creating a new Key Vault version does not automatically invalidate existing in-memory cache entries.

Practical example: An AI application retrieves an API key from Key Vault whenever it calls an AI service. With no caching, 500 requests could produce 500 Key Vault reads. With a 15-minute cache, the application reuses the secret across those requests and greatly reduces Key Vault traffic. If the credential changes, the application refreshes the cache when it expires or when a configured rotation event triggers invalidation.

Module 1 key takeaways

- **Three object types:** Secrets store sensitive values such as API keys, passwords, and database connection strings. Keys support cryptographic operations such as encryption, decryption, and signing. Certificates manage digital certificates used for TLS and secure application communication.
- **Organize vaults and control access:** Separate vaults by application and environment to isolate development from production. Use Azure RBAC: Key Vault Secrets User for applications that only read secrets, and Key Vault Secrets Officer for authorized operators who manage them. Apply least privilege.
- **Versioning for rotation:** Updating a secret creates a new version and keeps the previous ones. get_secret() returns the latest enabled version when no version is specified. With dual-credential rotation, introduce the new credential before retiring the old one to minimize downtime. Creating a new secret version does not change the credential in the underlying service, so the rotation workflow must update both sides.
- **Cache carefully:** Store retrieved secrets in application memory for a limited time to reduce Key Vault calls. Choose the TTL by security requirements, rotation frequency, and acceptable staleness. Use Event Grid secret events to trigger refresh or rotation workflows when needed.

Module 2: Manage application settings with Azure App Configuration

22.1 Introduction to Azure App Configuration

Azure App Configuration is a managed service that centralizes application settings outside the application code. Several services can share configuration values and update them without a new deployment. For AI applications these settings might

include model endpoints, feature flags, retry limits, logging levels, and service-specific options.

Core features

- **Centralized key-value store:** Keeps application settings separate from source code, so configuration is easier to manage across services.
- **Hierarchical keys:** Organize settings with naming patterns such as AI:ModelName, AI:Temperature, and Database:Timeout.
- **Labels:** Keep different values for development, testing, and production without changing application code.
- **Feature flags:** Enable or disable application features dynamically, such as rolling out a new AI model to selected users.
- **Key Vault references:** Store sensitive values in Azure Key Vault and reference them from App Configuration instead of storing the actual secret there.

Dynamic refresh

Dynamic refresh lets an application pick up updated configuration values without restarting.

1. The application loads its settings from Azure App Configuration.
2. An operator updates a setting, such as the model name or retry limit.
3. The application checks for updates and refreshes its configuration.
4. Later operations use the new values.

Important: With the Python provider, refresh is activity-driven. The application must call config.refresh(); the provider does not refresh settings automatically in the background.

The Python provider

The Python configuration provider loads settings into a dictionary-like object, so the application accesses values by key:

```
model_name = config["AI:ModelName"]
retry_limit = config["AI:RetryLimit"]
```

This keeps configuration separate from business logic. The same code can be used across environments while loading different values through labels.

Practical example: An AI classification service uses a model endpoint, a model name, and a retry limit.

- Store AI:ModelName and AI:RetryLimit in Azure App Configuration.
- Use labels such as dev and prod to keep environment-specific settings.
- Use a Key Vault reference for any sensitive API key.

- Enable dynamic refresh so the application picks up configuration changes without a restart.

22.2 Connect to Azure App Configuration from application code

App Configuration lets an application load settings from a centralized store instead of hardcoding them in source code. The Python provider exposes the settings through a dictionary-like object, so the application reads values by their keys.

Data model and connection

- **Key-value pairs:** Each setting has a key and a value, with optional labels and content types.
- **Labels:** Allow different values for environments such as Development, Testing, and Production.
- **Python provider load():** Connects to App Configuration and loads the selected settings into an object the application can query.
- **Microsoft Entra ID:** Use DefaultAzureCredential to authenticate securely, including through managed identity when the application runs in Azure.

Dynamic refresh with a sentinel key

A sentinel key is a special configuration setting that signals that a group of settings has changed. For example, after changing several pipeline settings, an operator updates the sentinel key. The application checks the sentinel and reloads the configuration when it detects a change.

Important: In this Python provider, refresh is activity-driven. The application must call config.refresh(); the provider does not refresh settings in the background. The configured refresh interval limits how often it checks for changes.

Practical example: An AI document-processing application uses a batch size and a retry limit stored in App Configuration. An operator changes the production batch size from 10 to 20 and updates the sentinel key. On a later refresh, the application picks up the new configuration without restarting.

22.3 Organize settings with labels and feature flags

App Configuration helps teams manage environment-specific settings and control application features without changing code or redeploying. Labels organize configuration variants, and feature flags decide whether particular functionality is enabled.

Labels and configuration composition

- **Labels:** The same configuration key can have different values for Development, Testing, and Production.

- **Default settings:** Store common values without a label so they can be shared across environments.
- **Override behavior (selector stacking):** Load the default settings first and the environment-specific settings afterward. When the same key is selected more than once, the value loaded later wins.

Example: AI:ModelName might be model-dev under the Development label and model-prod under the Production label. The application uses the appropriate value without any code change.

Feature flags

- **Feature flags:** Enable or disable functionality through configuration instead of code changes.
- **FeatureManager.is_enabled():** Checks whether a named feature is enabled.
- **Runtime control:** Operators can change feature flags in App Configuration, so applications can introduce or disable features without redeployment.
- **Controlled rollout:** Test a new model with selected users before enabling it more broadly, or quickly disable a feature if problems occur.
- **Independent refresh:** Feature flags can have refresh settings separate from ordinary configuration values, depending on how the provider and feature-management library are configured.

The Python example

The code loads configuration and feature flags, then selects which model to use.

- **feature_flag_enabled=True:** Tells the Python provider to load feature flags. The documented parameter is singular (feature_flag_enabled), not feature_flags_enabled.
- **FeatureManager(config):** Makes the loaded feature flags available for evaluation.
- **is_enabled("UseNewModel"):** Checks the flag and selects the processing path.
- **Fallback behavior:** When the flag is disabled, the application keeps using the current model.

Practical example: An AI document-classification service wants to introduce a newer model. The team creates a UseNewModel feature flag in App Configuration. At first the flag is disabled, so all documents use the current model. After testing, the team enables the flag and the application starts using the new model according to its feature-management configuration.

22.4 Reference Key Vault secrets from App Configuration

App Configuration can store references to secrets in Azure Key Vault. The application retrieves ordinary settings and sensitive values through one configuration provider, while the actual secrets stay securely in Key Vault.

How Key Vault references work

- **Reference, not secret:** App Configuration stores a URI that points to a Key Vault secret, not the secret value itself.
- **Automatic resolution:** The Python configuration provider recognizes the Key Vault reference and retrieves the secret using the configured credentials.
- **Unified access:** Once resolved, the secret is available through the same dictionary-like configuration object as other settings.
- **Version handling:** A reference can point to a specific secret version, or use a versionless URI to retrieve the latest version when refreshed.

Configure authentication and permissions

The application needs permission to read from both services:

- **App Configuration Data Reader:** Lets the application identity read configuration settings.
- **Key Vault Secrets User:** Lets the identity read secret values from Key Vault.
- **Managed identity:** In Azure, use the Function App's managed identity where possible, so no credentials are stored for accessing either service.

The Python provider can use DefaultAzureCredential for authentication and a Key Vault credential to resolve the referenced secrets.

Secret refresh

The secret_refresh_interval setting controls how often the provider can refresh resolved Key Vault secret values, independently of ordinary App Configuration settings.

Practical example: An AI application needs a database password and a model endpoint.

- Store the database password in Azure Key Vault.
- Create a Key Vault reference in Azure App Configuration.
- Grant the application's managed identity permission to read both configuration settings and secrets.
- Load the settings through the Python provider.
- Refresh the configuration periodically so a newly rotated secret can be picked up.

The application reads the database password through its configuration object without the actual password being stored in App Configuration.

22.5 Decide what to store in App Configuration vs. Key Vault

App Configuration and Key Vault are complementary services with different purposes. App Configuration manages application settings; Key Vault protects sensitive information. Using them together makes configuration easier to manage without exposing credentials in application code.

What goes into Azure App Configuration?

Non-sensitive configuration values that control how the application behaves:

- **Endpoints:** URLs of AI services, APIs, and other dependencies.
- **Batch sizes:** The number of documents or tasks processed in each batch.
- **Feature flags:** Settings that enable or disable features, such as switching to a new AI model.
- **Key Vault references:** Pointers to secrets stored in Key Vault, not the secret values themselves.

What goes into Azure Key Vault?

Sensitive values that would create a security risk if exposed:

- **API keys:** Credentials used to access AI services or third-party APIs.
- **Connection strings:** Database or storage credentials that contain secrets.
- **Certificates:** Digital certificates used for TLS and application identity.
- **Encryption keys:** Keys used for cryptographic operations.

Key Vault also supports secret versioning, expiration, rotation workflows, and access auditing.

Classification criteria

The simplest rule is to ask: *does this value grant access, or does it control application behavior?* Grants access means Key Vault. Controls behavior means App Configuration. Three questions help:

- **Sensitivity:** Could someone gain unauthorized access if this value were exposed? If yes, store it in Key Vault.
- **Rotation:** Does the value need secret lifecycle management, expiration, or rotation? Key Vault is the right place.
- **Access control:** Does the value need tightly controlled, audited access? Use Key Vault and grant only the necessary permissions.

How the two services work together

1. The application reads settings from Azure App Configuration.
2. App Configuration provides ordinary settings, such as an AI endpoint and batch size.
3. For sensitive values, App Configuration provides a reference to a secret in Key Vault.
4. The application resolves the reference using an identity that has permission to read the secret.

This gives the application one consistent way to access configuration while keeping sensitive values in a dedicated secret store.

Practical example: An AI document-processing application needs an AI endpoint, a batch size, and an API key.

Setting	Where to store it
AI endpoint URL	App Configuration
Batch size	App Configuration
Feature flag for a new model	App Configuration
AI API key	Key Vault
Database password	Key Vault
Reference to the AI API key	App Configuration

Module 2 key takeaways

- **Centralize application settings:** Store settings outside the code so values can be changed without rebuilding or redeploying. Common examples are AI model names, service endpoints, batch sizes, retry limits, and logging levels. Several applications can share configuration, which makes settings easier to maintain across environments.
- **Labels and configuration composition:** Labels hold different values of the same key for development, testing, and production. With selector stacking, load the default settings first and the environment-specific settings afterward. The last-loaded value wins. For example, the default AI:ModelName could be model-standard while the production-labelled value is model-production.
- **Feature flags:** FeatureManager evaluates whether a feature is enabled. Enable or disable functionality through configuration instead of code changes. This is useful for introducing a new AI model gradually or disabling a feature quickly if problems occur. Feature flags can have their own refresh settings.
- **Key Vault references:** App Configuration stores a reference to a secret, not the secret value. The Python provider can resolve it when loading configuration if Key

Vault credentials are configured, and the application identity needs access to both App Configuration and the referenced secrets.

- **Classification rule:** Values that grant access (API keys, database passwords, credentials) go in Key Vault. Values that control behavior (model names, endpoints, batch sizes, retry limits, feature flags) go in App Configuration. Use Key Vault references to connect the two.

LP-9: Observe and troubleshoot apps on Azure

Module 1: Instrument an app with OpenTelemetry

23.1 Introduction to instrumenting an application with OpenTelemetry

OpenTelemetry (OTel) is an open-source, vendor-neutral observability framework maintained by the Cloud Native Computing Foundation (CNCF). It helps developers understand how an application behaves by collecting telemetry data such as traces, metrics, and logs.

The three types of telemetry

- **Traces:** Show the journey of a request across several services. For example, an AI request travelling from an API to PostgreSQL, an AI model, and a background worker.
- **Metrics:** Numerical measurements such as request count, response latency, error rate, CPU usage, and memory consumption.
- **Logs:** Records of individual events, warnings, and errors that help developers investigate failures.

Together these signals show where an application is slow, why a request failed, and whether the system is running out of resources.

How OpenTelemetry works

- **Instrumentation:** OpenTelemetry APIs, SDKs, and supported instrumentation libraries generate telemetry from the application and its dependencies.
- **Collection and processing:** An OpenTelemetry Collector can receive, process, and forward telemetry to suitable destinations.
- **Export:** Telemetry can be sent to compatible monitoring backends without rewriting the application's instrumentation.

Important: OpenTelemetry is not itself a monitoring dashboard or a telemetry storage backend. It is the framework for generating, collecting, and exporting telemetry.

Use OpenTelemetry with Azure Monitor

- The Azure Monitor OpenTelemetry distribution simplifies instrumentation and exports telemetry to Azure Monitor and Application Insights. It includes Azure-specific integration and supported automatic instrumentation.
- Microsoft's current OpenTelemetry packages and setup instructions vary by language. Python and Node.js use different package names, and Java has its own integration approach. Follow the documentation for your language and application type instead of assuming one installation method works everywhere.

Practical example: monitor an AI application. An AI document-processing application receives an HTTP request, retrieves document content from storage, calls an AI model, and saves the result to a database. OpenTelemetry helps you trace the request across these stages, measure how long the model call takes, and record errors when the database connection fails. Azure Monitor and Application Insights then help you analyze the telemetry to find bottlenecks and diagnose failures.

23.2 Explore OpenTelemetry and its role in observability

OpenTelemetry provides standardized APIs, SDKs, instrumentation libraries, and exporters to collect telemetry and send it to monitoring platforms such as Azure Monitor and Application Insights. It is particularly useful for AI applications, because a single request may involve an API, a database, an embedding service, an AI model, and several background workers.

The three observability pillars

- **Distributed tracing:** Tracks a request across several services. Each operation is a span, and related spans form a trace. This helps identify which component adds latency.
- **Metrics:** Numerical measurements over time, such as request counts, error rates, response latency, CPU usage, and token consumption.
- **Logs:** Timestamped events, warnings, and errors that help developers investigate specific failures.

The signals complement one another. Traces show where a request is slow, metrics show whether the problem is widespread, and logs help explain what happened.

OpenTelemetry components

- **APIs:** Define the interfaces that application code and instrumentation use to create telemetry.
- **SDKs:** Implement telemetry processing, sampling, resource configuration, and export behavior.
- **Instrumentation libraries:** Automatically capture telemetry from supported frameworks and dependencies, which reduces the need for manual tracing code.

- **Exporters:** Send telemetry to compatible destinations such as Azure Monitor or an OpenTelemetry Collector.

These components work together rather than forming a strictly sequential pipeline. Instrumentation generates telemetry through the OpenTelemetry APIs, the SDK processes it, and the configured exporter sends it to the backend.

How the flow works (AI document-processing application)

1. A user submits a document through an API.
2. Instrumentation captures the incoming request and starts a trace.
3. The application retrieves document content, calls an AI model, and writes results to a database. Each operation can contribute spans and metrics.
4. The SDK processes the telemetry and the exporter sends it to Application Insights.
5. Developers inspect traces, metrics, and logs to find performance bottlenecks or failures.

Why use OpenTelemetry with Azure Monitor?

Developers instrument applications with standardized interfaces instead of tying all telemetry generation to one monitoring vendor. Azure Monitor and Application Insights provide the destination where the exported telemetry is analyzed.

Practical example: An AI response takes eight seconds. A distributed trace shows that seven seconds were spent waiting for the model API, while the rest went to database access and application processing. Optimization effort goes to the real bottleneck.

23.3 Add the OpenTelemetry SDK to an application

You can add OpenTelemetry to an application with minimal code changes or with explicit SDK instrumentation. For Azure-hosted applications, the Azure Monitor OpenTelemetry distribution simplifies setup by combining OpenTelemetry SDK capabilities, supported instrumentation libraries, and Azure Monitor exporters.

Instrumentation approaches

- **Automatic instrumentation:** Captures telemetry from supported frameworks and libraries with minimal manual work. Easy to configure, but gives less control over application-specific spans and attributes.
- **SDK-based instrumentation:** Developers explicitly create spans, add attributes, and control sampling. Use it when you need detailed visibility into custom AI operations.
- **Azure Monitor OpenTelemetry distribution:** Packages supported instrumentation and Azure Monitor integration into one convenient installation and configuration path.

Configure OpenTelemetry in Python

Install the Azure Monitor OpenTelemetry distribution:

```
pip install azure-monitor-opentelemetry
```

Set the Application Insights connection string through an environment variable:

```
APPLICATIONINSIGHTS_CONNECTION_STRING="<your-connection-string>"
```

In production, configure this environment variable through the hosting platform instead of hardcoding the connection string in source code.

Configure the application identity

Resource attributes let Application Insights identify each service in Application Map.

- **service.name:** Identifies the application or service, such as embedding-service.
- **service.namespace:** Groups related services under a common namespace, such as rag-pipeline.
- **service.instance.id:** Identifies a particular running instance of the service.

Azure Monitor uses service.namespace and service.name to determine the Cloud Role Name, and service.instance.id to determine the Cloud Role Instance. These attributes help you tell individual services and instances apart when investigating a distributed application.

Practical example: observe an AI pipeline. An application generates embeddings for uploaded documents: it calls an embedding model, stores vectors in a database, and returns the result to the caller. With OpenTelemetry configured, supported instrumentation captures the relevant requests and dependencies, and you can add custom spans around embedding generation or vector storage to measure individual operations. Application Insights might then show that most of the latency comes from the embedding API rather than database storage, so the team knows where to focus.

23.4 Configure spans and traces

Distributed tracing shows how a request moves through the components of an application. With OpenTelemetry you create spans for individual operations and combine related spans into a trace, which makes latency and failures in AI pipelines easier to identify.

Custom spans and attributes

- **Tracer:** Obtained with trace.get_tracer() and used to create spans for a particular instrumentation scope.
- **Span:** Represents an operation, such as generating an embedding, querying PostgreSQL, or calling an AI model.

- **Context manager:** start_as_current_span() starts and ends a span automatically and sets up its parent-child relationship with the current trace context.
- **Attributes:** Add business context such as model name, token count, document ID, and result count, so telemetry can be filtered and analyzed.

OpenTelemetry supports nested spans, so a database query span can appear as a child of a broader document-processing span.

Execution flow of the Python example

1. **Get a tracer:** trace.get_tracer() provides the tracer used to create spans.
2. **Start the span:** A span named GenerateEmbedding records how long the embedding operation takes.
3. **Add attributes:** Record which model was used and how many tokens were processed.
4. **Run the operation:** generate_embedding() performs the actual work.
5. **End the span:** Leaving the with block closes the span automatically.

These attributes make it possible to check whether certain models or larger inputs are associated with higher latency.

Span kinds and error handling

- **SERVER:** An operation that handles an incoming request.
- **CLIENT:** An outgoing request to a dependency, such as an AI API or a database.
- **Nested spans:** Build a parent-child hierarchy that shows how time is distributed across operations.
- **Exceptions:** In the Python SDK, an uncaught exception that escapes a start_as_current_span() context is recorded by default and the span status is set to ERROR. If your code catches an exception internally, record it and set the error status explicitly when appropriate.

Practical example: trace an AI request. An AI document-processing service receives a request, generates an embedding, searches Azure Database for PostgreSQL, and calls a language model. You create spans for each operation. The trace shows that embedding generation took 200 ms, database retrieval 100 ms, and the language-model call 3 seconds, which identifies the language-model call as the main latency contributor.

23.5 Export telemetry to Azure Monitor

After instrumenting an application with OpenTelemetry, the next step is exporting its telemetry to a monitoring backend. Azure Monitor and Application Insights let developers analyze traces, metrics, logs, and exceptions to understand application behavior and diagnose issues in distributed AI workflows.

The export pipeline

- **Direct export:** The Azure Monitor OpenTelemetry exporter sends telemetry to the Application Insights ingestion endpoint.
- **Telemetry mapping:** Application Insights stores different telemetry types in tables such as requests, dependencies, traces, and exceptions.
- **Offline storage:** If the application temporarily loses connectivity, the exporter can cache telemetry locally and retry sending it for up to 48 hours. This is not an unconditional guarantee: storage limits and prolonged outages can still cause telemetry loss.

Sampling strategies

Sampling reduces the amount of telemetry exported, which helps control ingestion costs and overhead while preserving useful diagnostic information.

- **Fixed-percentage sampling:** Collects a configurable fraction of traces. For example, a sampling ratio of 0.1 attempts to keep about 10% of traces. In the Python Azure Monitor OpenTelemetry distribution, this is configured with a sampling ratio setting.
- **Rate-limited sampling:** Limits the number of traces collected per second. The current Azure Monitor OpenTelemetry Python distribution defaults to about five traces per second when no other sampling configuration is provided. Defaults can vary by package and version.

Important: Trace sampling does not mean metrics are sampled by the same mechanism. Metrics remain available independently of trace sampling, which makes them useful for aggregate request counts, error rates, and latency. Logs associated with unsampled traces can be affected by trace-based log sampling, depending on configuration.

Choose the right sampling strategy

- **Fixed-percentage sampling:** Useful when you want a predictable fraction of traces retained as traffic volume changes.
- **Rate-limited sampling:** Useful when you want to cap trace volume per second as request traffic grows.
- **Metrics for alerting:** Use metrics to monitor key signals such as failure rates and response latency, because trace sampling can make sampled trace counts incomplete.

Practical example: An AI application processes thousands of requests per minute, and exporting every trace would create unnecessary ingestion costs. With rate-limited sampling, the application keeps a controlled number of traces while metrics still provide aggregate performance data. When a failure occurs, the retained traces and exception telemetry help developers investigate.

23.6 Debug distributed flows with trace data

When an AI application calls several services, finding the source of a delay can be difficult. Application Insights helps developers investigate the complete request journey using distributed traces, dependency telemetry, exceptions, and custom attributes. For example, a single request might pass through an API, an embedding service, Azure Database for PostgreSQL, and a language model, and tracing shows which component adds the most latency.

Application Insights diagnostic tools

- **Application Map:** Shows application components, their dependencies, response times, and failure rates. Use it to find slow or failing services.
- **End-to-end transaction details:** Shows a timeline of operations and dependency calls within a trace, so you can pinpoint where a request spends its time.
- **Span details:** Show attributes, timestamps, status, and exception information for diagnosing individual operations.

Query requests and dependencies with KQL

The requests table holds incoming requests handled by the application, and the dependencies table holds outgoing calls to external services such as AI endpoints and databases. The operation_Id field correlates telemetry records that belong to the same distributed operation, and duration measures the duration of a request or dependency call.

A readable query to find slow requests and their dependency calls:

```
requests
| where duration > 3s
| project operation_Id, name, duration
| join kind=inner (
  dependencies
  | project operation_Id, dependencyName = name, dependencyDuration
= duration
) on operation_Id
| order by duration desc
```

- Filter requests that take longer than three seconds.
- Select the operation ID, request name, and request duration.
- Join the related dependency calls through operation_Id.
- Show the slowest requests first, along with their dependency details.

The query uses 3s because Application Insights records duration as a time span, not as a plain number of milliseconds.

Investigate AI-specific performance issues

- **Embedding timeouts:** Examine dependency duration, failures, and exception details for embedding API calls.
- **Vector search latency:** Check database dependency durations, query performance, and resource utilization.
- **LLM rate limiting:** Investigate HTTP response codes, retry activity, and model API latency.
- **Token consumption:** Query custom telemetry dimensions to find out whether input or output token counts correlate with slow requests.

Practical example: An AI support API takes eight seconds to answer a question. Application Insights shows that the database query takes 200 ms, embedding generation 300 ms, and the language-model dependency seven seconds. The trace shows the LLM call is the main bottleneck, so the team can focus on model latency, request size, or retry behavior instead of unnecessarily tuning the database.

Module 1 key takeaways

- **Three observability pillars:** Traces show the complete journey of a request across services and help identify latency and failures. Metrics give numerical measurements such as request counts, response times, error rates, and resource use. Logs record timestamped events, errors, and diagnostic details. Together they give complementary views of application health.
- **Vendor neutrality:** Use OpenTelemetry APIs and SDKs to generate telemetry without tying the application to one monitoring vendor. The Azure Monitor OpenTelemetry distribution simplifies instrumentation and export to Application Insights, and the same instrumentation can send telemetry to other compatible backends through suitable exporters.
- **Custom spans:** Automatic instrumentation captures many common framework and dependency operations, but application-specific operations may need custom spans, for example for embedding generation, vector retrieval, prompt preparation, and model inference. Custom attributes such as model name and token count add diagnostic context. Do not record sensitive prompts, credentials, or personal information in telemetry.
- **Balance sampling and cost:** Sampling reduces the trace data exported and helps control ingestion costs. Fixed-percentage sampling keeps a configured fraction of traces, and rate-limited sampling limits trace volume over time. Metrics continue to provide aggregate measurements independently of trace sampling. Choose a strategy that balances cost, performance, and the diagnostic detail you need.
- **Correlate traces for troubleshooting:** In Application Insights, operation_Id correlates requests and dependencies that belong to the same distributed

operation, so you can investigate an entire request across several services. For example, if an AI response takes eight seconds, a trace shows whether the delay comes from embedding generation, a database query, or the language-model API.

Module 2: Analyze app telemetry with logs and metrics

24.1 Introduction to analyzing application telemetry with logs and metrics

Azure Monitor and Application Insights help developers monitor application health, investigate failures, and find performance bottlenecks. They collect telemetry from instrumented applications and distributed services, and Kusto Query Language (KQL) lets you analyze that telemetry and build monitoring solutions.

Core concepts

- **Application Insights:** Collects application telemetry, including requests, dependencies, exceptions, traces, and performance information. Distributed tracing connects related operations across services.
- **KQL:** The query language for exploring telemetry in Azure Monitor Logs, Log Analytics workspaces, and Azure Data Explorer.
- **Pipe model:** A KQL query usually starts with a table name, followed by operators separated by the pipe character (|). Each operator transforms the results of the previous step.
- **Dashboards:** Show important operational metrics and visualizations in one place.
- **Workbooks:** Interactive reports with filters, parameters, charts, and tables for investigating incidents.
- **Alert rules:** Evaluate metrics or log queries and notify teams when defined conditions are met.

Important: Telemetry collection depends on appropriate application instrumentation and configuration. Application Insights does not automatically capture every detail of every service without the necessary setup.

Learn KQL with a practical example

An AI application has slow responses. Query the Application Insights requests table to find requests that take longer than three seconds:

```
requests
| where duration > 3s
| project timestamp, name, duration, success
| order by duration desc
| take 10
```

- **requests:** Reads request telemetry.
- **where:** Filters requests lasting more than three seconds.
- **project:** Selects the columns needed for the investigation.
- **order by:** Sorts the slowest requests first.
- **take 10:** Returns ten of the matching requests (here, the first ten after sorting).

Dashboards, workbooks, and alerts

- **Dashboard:** Monitor request volume, failure rates, latency, and availability at a glance.
- **Workbook:** Use interactive parameters to investigate a particular time range, operation, service, or error category.
- **Alert rule:** Configure an alert when a log query detects repeated failures or when a metric crosses a threshold. For an AI service, an alert might detect a sustained increase in failed inference requests or unusually high response latency.

Practical example: An AI document-processing application suddenly takes eight seconds to respond instead of two. The team queries request telemetry with KQL, checks dependency calls to see whether the AI endpoint or database is slow, and checks exceptions for related failures. A workbook compares the incident with earlier periods, and an alert rule notifies the team if latency stays above the defined threshold.

24.2 Write basic KQL queries: filtering and searching

KQL lets you investigate slow HTTP requests, failed dependencies, application exceptions, and log messages without manually inspecting individual records.

Application Insights tables

- **requests:** Incoming requests handled by the application, including duration, response code, and success status.
- **dependencies:** Outgoing calls to databases, APIs, and AI model endpoints, including duration and success information.
- **exceptions:** Exception telemetry and diagnostic details, such as exception messages and stack traces.
- **traces:** Application log messages emitted as telemetry, including severity and message text.

Core KQL operators

- **where:** Filters rows by a condition.
- **project:** Selects columns and can rename them for easier analysis.
- **take:** Returns a specified number of rows without guaranteeing their order.
- **top:** Returns the top N rows according to a specified sort expression.
- **has:** Searches for a term using term-based matching.

- **contains:** Searches for a substring within text. It can be more expensive than term-based matching.

take vs. top vs. has vs. contains

Operator	What it does
take	Returns N rows without guaranteeing their order
top	Returns the top N rows according to a sort expression
has	Term-based matching (generally more efficient)
contains	Substring matching (can be more expensive)

Practical KQL examples

Example 1: find failed HTTP requests. Shows the ten most recent failed requests with their timestamp, name, response code, and duration.

```
requests
| where success == false
| top 10 by timestamp desc
| project timestamp, name, resultCode, duration
```

Example 2: find slow AI dependencies. Identifies the ten slowest dependency calls that exceeded three seconds. Use it to investigate slow AI endpoints, database queries, or external API calls.

```
dependencies
| where duration > 3s
| top 10 by duration desc
| project timestamp, name, target, duration
```

Example 3: search application log messages. Searches log messages for the term timeout and returns up to 20 recent matching records.

```
traces
| where message has "timeout"
| top 20 by timestamp desc
| project timestamp, message, severityLevel
```

Practical example: An AI application starts responding slowly. Query requests to identify slow incoming requests, dependencies to investigate downstream calls, and exceptions or traces to find related failures and diagnostic messages.

24.3 Write basic KQL queries: aggregation and visualization

Once you can filter individual telemetry records, KQL lets you aggregate large volumes of data, identify trends, and visualize application performance. This is especially useful for monitoring AI workloads across several services.

Aggregate data with summarize

The summarize operator groups records and calculates aggregate values.

- **count():** Counts the number of records.
- **avg():** Calculates an average, such as average request duration.
- **percentile():** Calculates a percentile, such as p95 latency, to understand slower requests.
- **bin():** Groups timestamps or numeric values into fixed intervals for trend analysis.

Example: group requests by hour and service over the last 24 hours

```
requests
| where timestamp > ago(24h)
| summarize count(), avg(duration)
  by bin(timestamp, 1h), cloud_RoleName
| render timechart
```

- **where timestamp > ago(24h):** Filters requests from the last 24 hours.
- **summarize:** Calculates the total request count and average duration.
- **bin(timestamp, 1h):** Groups requests into one-hour intervals.
- **cloud_RoleName:** Separates results by application role or service name.
- **render timechart:** Displays the aggregated results as a time-series chart.

The query shows when request volume increases and whether average response times change across services.

Use percentiles for AI latency analysis

Average latency can hide occasional slow requests. For AI applications, p95 latency often reflects the experience of the slower requests.

```
requests
| where timestamp > ago(24h)
| summarize avg(duration), percentile(duration, 95)
  by bin(timestamp, 1h)
| render timechart
```

This query calculates the average and 95th-percentile request duration for each hour. Use it to find periods when a small proportion of requests become unusually slow.

Visualize results with render

- **render timechart:** Shows trends over time, such as request latency or error counts.
- **render barchart:** Compares categories, such as request volume by service.
- **render piechart:** Shows proportions, such as the distribution of outcomes across categories.

Including render helps preserve the intended visualization when the query is used in supported dashboards or workbooks.

Practical example: An AI application is slow during peak hours. The development team groups requests by hour and service, compares request counts with average and p95 latency, and finds when performance deteriorates. They then examine dependency telemetry for the same period to determine whether the bottleneck is the model endpoint, the database, or the application itself.

24.4 Explore logs for errors

By analyzing exception telemetry and correlating requests with their dependencies, you can determine not only which operation failed but also where in the processing pipeline the failure occurred.

Investigate errors with KQL

- **Exceptions table:** Stores exception telemetry, including exception type, timestamp, and diagnostic details.
- **Sampling-aware counting:** When telemetry sampling is enabled, one stored record may represent several telemetry items. Use sum(itemCount) instead of count() to estimate the total number of exceptions represented.
- **Group by exception type:** Identify recurring failures such as timeouts, authentication errors, or invalid input.
- **Group by operation:** Find which application operation or request path is associated with the exceptions.

Example: exception trend over 24 hours

```
exceptions
| where timestamp > ago(24h)
| summarize sum(itemCount)
  by bin(timestamp, 1h), type
| render timechart
```

- **where timestamp > ago(24h):** Selects exceptions from the last 24 hours.
- **summarize:** Aggregates the represented exception count.
- **bin(timestamp, 1h):** Groups records into hourly intervals.

- **type:** Separates counts by exception type.
- **render timechart:** Shows exception trends over time.

The chart shows when failures increased and which exception types contributed to the spike.

Correlate errors across services

A failure in an AI application may originate in one service but appear as a failed request in another. The operation_Id field connects related telemetry records from the same distributed operation.

Apply this to AI workloads

If an AI document-processing service starts reporting failures, you can investigate:

- **Embedding timeouts:** Look for exceptions related to embedding API calls.
- **Vector-search failures:** Check database errors and the associated request duration.
- **Model API failures:** Examine exception types, messages, and correlated dependencies.
- **Traffic spikes:** Compare hourly exception totals with incoming request counts.

24.5 Explore logs for performance: dependencies

In an AI application, slow responses often come from a downstream dependency rather than the application itself. Application Insights records outgoing calls in the dependencies table, so you can investigate latency, failures, and the services responsible for performance bottlenecks.

Analyze dependency latency and failures

- **dependencies table:** Captures instrumented outgoing calls to databases, APIs, AI model endpoints, and other services.
- **Percentile calculations:** Use p95 or p99 latency to reveal slow requests that averages may hide.
- **Group failures by target and result code:** Find out whether errors come from a particular AI endpoint, a database, or another downstream service.
- **Correlate with requests:** Use operation_Id to connect dependency calls to incoming requests and investigate the full transaction.

Interpret dependency result codes

- **HTTP 429 (Too Many Requests):** The downstream service is throttling requests. Respect Retry-After when provided, use bounded retries with exponential backoff, and consider reducing concurrency or request volume.

- **HTTP 500 (Internal Server Error):** The downstream service returned a server-side error. Examine its logs, request payloads, dependency health, and retry behavior before deciding whether to retry.
- **Timeout with no result code:** The call may have timed out before a response arrived. Investigate network connectivity, client timeouts, downstream overload, and dependency duration.

A result code is a diagnostic clue, not necessarily the complete root cause. Use correlated traces and dependency details to determine what happened.

Practical example: An AI document-processing pipeline calls an embedding API, PostgreSQL for vector search, and a language-model endpoint. If p95 latency suddenly increases, dependency telemetry shows whether the embedding API is slow, PostgreSQL queries are taking longer, or the language-model endpoint is returning 429 responses. The team can apply a targeted fix instead of scaling every component unnecessarily.

24.6 Build dashboards for application telemetry

Azure dashboards give a consolidated view of application health and performance. Instead of opening several Azure resources one by one, a team brings important metrics, charts, and query results into one place to monitor an AI application. Azure dashboards can combine resources across different resource groups and subscriptions.

Azure dashboards

- **Customizable tiles:** Combine metrics charts, application health indicators, and log-query results in one dashboard.
- **At-a-glance monitoring:** Track request volume, response time, and failures without navigating between resources.
- **Cross-resource visibility:** View telemetry from different application components and Azure resources together.
- **Team monitoring:** Share dashboards with the appropriate users so teams monitor the same operational indicators.

Pin Application Insights metrics charts

You can create a metric chart in the Application Insights Metrics pane and pin it to an Azure dashboard:

1. Open the Application Insights resource in the Azure portal.
2. Select Metrics and choose the metric to monitor.
3. Apply filters and split the data by dimensions, such as request success or service role.
4. Select the time range and aggregation.

5. Choose Pin to dashboard, select the target dashboard, and save your changes.

Select the correct aggregation

- **Average:** Shows typical behavior, such as average server response time.
- **Sum:** Shows the total across the selected time interval, such as total failed requests when using an appropriate count metric.
- **Maximum:** Shows the highest observed value. Useful for finding peaks in resource utilization or latency.

Choose the aggregation based on what the metric represents. Averaging request duration shows typical latency, while summing request counts measures volume.

Which metrics should you monitor?

For an AI application, start with these indicators:

- **Server response time:** How long the application takes to respond.
- **Failed requests:** Reveals application errors and unsuccessful operations.
- **Dependency failures:** Identifies problems with databases, AI model endpoints, or external APIs.
- **Request rate:** Shows incoming workload and traffic spikes.

Practical example: An AI document-processing service becomes slow during peak hours. A dashboard shows request volume, average response time, failed requests, and dependency failures together. The team can quickly tell whether the slowdown coincides with increased traffic or a failing downstream service.

24.7 Set alerts for application failures and anomalies: alert rules

Azure Monitor alerts help teams detect application failures and unusual behavior automatically. Instead of checking dashboards continuously, developers configure alert rules that evaluate metrics or log queries and notify the responsible team when a defined condition occurs. For AI applications, alerts can detect high failure rates, slow model responses, dependency outages, and sudden increases in resource consumption.

Components of an alert rule

- **Scope:** The Azure resource being monitored, such as Application Insights, a Function App, or a database.
- **Condition:** The signal and threshold that trigger the alert, such as failed requests exceeding a defined limit.
- **Action group:** What happens when the alert fires, such as sending an email or SMS, calling a webhook, or invoking an automated response.
- **Severity:** The urgency of the problem. Azure Monitor uses severity levels from 0 (critical) to 4 (verbose) for log alerts.

The two alert types

- **Metric alerts:** Evaluate numerical metrics at regular intervals. Suitable for conditions such as high CPU usage, increased response latency, or a high number of failed requests.
- **Log search alerts:** Run KQL queries against collected telemetry. Useful when detection requires grouping, filtering, joining, or analyzing several types of records.

Example: a KQL alert query

This query detects when a service has more than ten failed requests.

```
requests
| where success == false
| summarize failedCount = count() by cloud_RoleName
| where failedCount > 10
```

- **requests:** Reads incoming request telemetry.
- **where success == false:** Keeps only failed requests.
- **summarize:** Counts failures for each application role or service.
- **where failedCount > 10:** Returns only services whose failure count exceeds ten.

Important: This query is simplified. In a real alert rule, configure the evaluation frequency and the lookback period so the count covers a meaningful time window, for example more than ten failures in five minutes. Also consider sampling, because stored telemetry may not represent every request.

Practical example: detect AI service failures. An AI document-classification service starts returning frequent errors. A log search alert evaluates failed requests over a five-minute window. When the failure count exceeds the threshold, Azure Monitor triggers an action group that notifies the operations team. The team then investigates Application Insights traces and dependency telemetry to find out whether the failures come from the AI endpoint, the database, or the application code.

24.8 Set alerts for application failures and anomalies: action groups and smart detection

Once an alert rule detects a failure or abnormal behavior, Azure Monitor must notify the right people or start an automated response. Action groups define what happens when an alert fires, and smart detection helps identify unusual patterns that manually configured thresholds may not cover.

Action groups

An action group is a reusable collection of notifications and automated actions that you can attach to several alert rules. Azure Monitor allows up to five action groups on a single alert rule.

- **Email, SMS, voice, and push notifications:** Inform developers or operations teams about incidents.
- **Azure Functions:** Trigger custom remediation logic, such as collecting diagnostics or starting a recovery workflow.
- **Logic Apps and webhooks:** Connect alerts to workflows, incident-management systems, or collaboration tools.
- **Automation runbooks and Event Hubs:** Automate operational tasks or forward alert events to other processing systems.

For example, a critical AI inference failure could notify the operations team by email and trigger a Function that collects diagnostic information.

Smart detection

Application Insights smart detection uses machine learning to identify unusual patterns in application telemetry, without requiring a fixed threshold for every condition. Detection works only when the application provides enough telemetry.

- **Failure anomalies:** Detects when the failure rate rises outside its expected range, taking factors such as application load and historical behavior into account.
- **Performance anomalies:** Identifies when operation response times or dependency durations degrade compared with their historical baseline.
- **Reduced manual tuning:** Surfaces unexpected problems that a fixed threshold might miss.

Smart detection vs. manual alert rules

The two approaches complement one another. Use manual alert rules for known operational limits, and smart detection to find unexpected changes.

Practical example: monitor an AI inference service. An AI classification API normally has a low failure rate, but after a deployment its dependency latency starts to increase gradually.

1. Application Insights collects request and dependency telemetry.
2. Smart detection identifies the unusual latency increase relative to the historical baseline.
3. Azure Monitor raises an alert.
4. An action group notifies the engineering team and can invoke an automated diagnostic workflow.

5. The team uses distributed traces and dependency details to find out whether the issue comes from the AI endpoint, the database, or the application code.

Module 2 key takeaways

- **Application Insights tables:** requests (incoming requests), dependencies (outgoing calls), exceptions (errors and stack traces), and traces (application log messages). Correlate them with operation_Id.
- **KQL basics:** Queries start with a table and flow through operators separated by pipes. Use where to filter, project to choose columns, top to get ranked rows, take for an arbitrary sample of rows, has for efficient term matching, and summarize with count(), avg(), percentile(), and bin() to aggregate. Add render to visualize results.
- **Prefer percentiles to averages:** p95 latency reveals slow requests that averages hide. When sampling is enabled, use sum(itemCount) instead of count() to estimate the true number of events.
- **Read dependency result codes as clues:** HTTP 429 means throttling (use Retry-After and exponential backoff), HTTP 500 means a downstream server-side error, and a timeout with no result code points to connectivity, client timeout, or overload problems.
- **Dashboards:** Pin Application Insights metric charts to Azure dashboards. Track server response time, failed requests, dependency failures, and request rate, and choose the aggregation (average, sum, or maximum) that fits the metric.
- **Alerts:** An alert rule has a scope, a condition, an action group, and a severity. Use metric alerts for numeric thresholds and log search alerts for KQL-based detection, with a sensible evaluation frequency and lookback window.
- **Action groups and smart detection:** Action groups are reusable (up to five per alert rule) and can send notifications or trigger Functions, Logic Apps, webhooks, runbooks, and Event Hubs. Smart detection finds failure and performance anomalies against a historical baseline. Use manual alerts for known limits and smart detection for unexpected changes.

Module 2 key takeaways

- **Application Insights tables:** requests (incoming requests), dependencies (outgoing calls), exceptions (errors and stack traces), and traces (application log messages). Correlate them with operation_Id.
- **KQL basics:** Queries start with a table and flow through operators separated by pipes. Use where to filter, project to choose columns, top to get ranked rows, take for an arbitrary sample of rows, has for efficient term matching, and summarize with count(), avg(), percentile(), and bin() to aggregate. Add render to visualize results.

- **Prefer percentiles to averages:** p95 latency reveals slow requests that averages hide. When sampling is enabled, use sum(itemCount) instead of count() to estimate the true number of events.
- **Read dependency result codes as clues:** HTTP 429 means throttling (use Retry-After and exponential backoff), HTTP 500 means a downstream server-side error, and a timeout with no result code points to connectivity, client timeout, or overload problems.
- **Dashboards:** Pin Application Insights metric charts to Azure dashboards. Track server response time, failed requests, dependency failures, and request rate, and choose the aggregation (average, sum, or maximum) that fits the metric.
- **Alerts:** An alert rule has a scope, a condition, an action group, and a severity. Use metric alerts for numeric thresholds and log search alerts for KQL-based detection, with a sensible evaluation frequency and lookback window.
- **Action groups and smart detection:** Action groups are reusable (up to five per alert rule) and can send notifications or trigger Functions, Logic Apps, webhooks, runbooks, and Event Hubs. Smart detection finds failure and performance anomalies against a historical baseline. Use manual alerts for known limits and smart detection for unexpected changes.